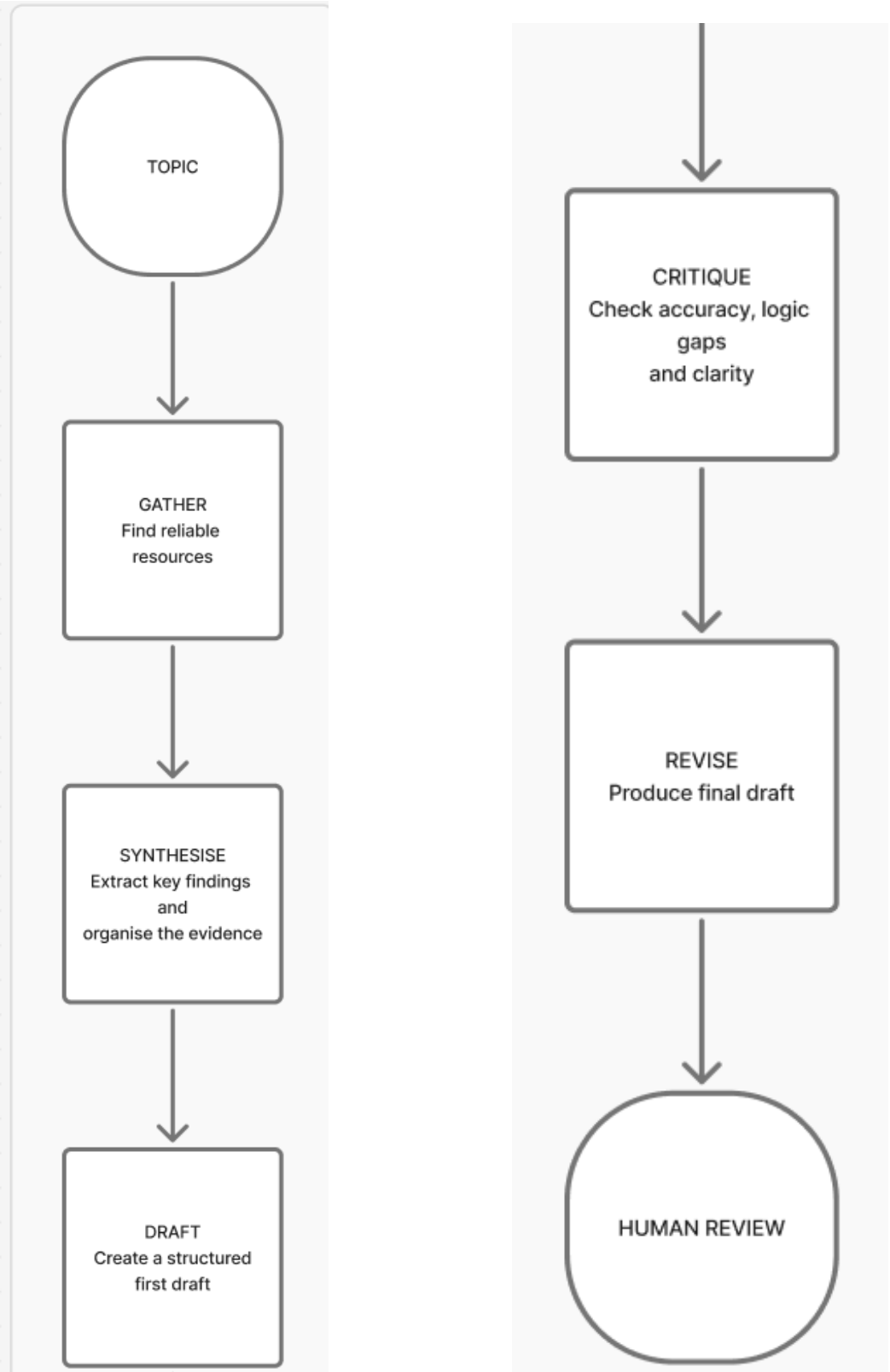
Ship an Automation Workflow v2

Workflow:



Run 1:

- *Topic:* RAG and hallucination mitigation
- *Gather and Synthesize* (NotebookLM):
    - Summary given: I am writing a paper on A Hybrid Retrieval-Augmented Framework for Real-Time Hallucination Detection and Mitigation in Large Language Models. I need the appropriate papers, their LR, and then a possible code and then a draft for the paper. I am final year BS honours CS student. The tone needs to be professional and must sound like me / my writing style.
    - After choosing resources and then gave the following prompt: I need a summary for all of it. it must include main findings important concepts Supporting evidence Contradictions Research gaps Sources associated with each claim.
    - Output I got was: *Based on the provided research and survey materials, here is a summary of the findings regarding **Retrieval-Augmented Generation (RAG)** and its efficacy in reducing hallucinations in Large Language Models (LLMs).*

Main Findings

- ***Significant Hallucination Reduction:** RAG provides a mechanism to ground LLM responses in external, up-to-date information, which significantly reduces "hallucinations"—situations where models generate content that is unfactual or unsupported by the source.*
- ***Complementary Memory Systems:** RAG combines **parametric memory** (knowledge learned during pre-training) with **non-parametric memory** (external document corpora like Wikipedia), allowing the model to "look up" information rather than relying solely on internal patterns.*
- ***Adaptability without Retraining:** A key advantage of RAG is its flexibility; what a model "knows" can be updated simply by swapping the retrieval corpus (e.g., using a newer version of Wikipedia), bypassing the high cost of retraining.*
- ***Residual Hallucination Causes:** Despite its power, RAG models can still hallucinate due to **poor retrieval quality** (fetching irrelevant or outdated documents), **misleading fusion** (incorrectly synthesizing facts across documents), or **over-reliance** on generative capabilities over the retrieved source.*

Important Concepts

- ***Source Faithfulness (SF) vs. World Factuality (WF):** SF measures if the output is consistent with a provided source (e.g., a retrieved document), while WF measures if the output aligns with general world facts.*
- ***Contrastive Decoding:** A mitigation strategy where a "base" model's output is contrasted with a "hallucination-prone" model (often a smaller or "evil" model) to suppress likely erroneous tokens.*
- ***Hybrid Search:** Combining **dense retrieval** (semantic meaning-based) with **sparse retrieval** (keyword-based like BM25) to bridge "vocabulary gaps" where engineering terms in a query don't match the technical jargon in a log or document.*
- ***Self-Correction (Self-RAG):** An architecture where the model is trained to generate "reflection tokens" that critique its own retrieved context and the support that context provides for its final claim.*
- ***Logits Extrapolating:** A technique to approximate deeper transformer layers by analyzing token probability trajectories across existing layers to improve factual expressiveness.*

Supporting Evidence

- ***Performance Benchmarks:*** *RAG set new state-of-the-art results on open-domain QA tasks (NaturalQuestions, WebQuestions), generating answers even when they weren't explicitly stated verbatim in the source documents.*
- ***Multi-Model Contrastive Decoding (MCD):*** *Experimental results showed MCD improved TruthfulQA scores by* ***+12.3% to +18.7%*** *over default greedy decoding by using both a "truthful" and an "evil" model to guide token selection.*
- ***Industrial Log Analysis:*** *In automotive embedded systems, RAG-driven hybrid search achieved significantly higher precision (0.93) on exact match queries compared to standard keyword search (0.91) and maintained high precision (0.81) even when vocabulary was mismatched.*
- ***Structured Outputs:*** *RAG has been shown to improve the quality of workflow generation from natural language requirements, reducing errors in complex structured data outputs.*

Contradictions and Nuances

- ***The "Silver Bullet" Fallacy:*** *While RAG improves factuality, it does not prevent hallucinations entirely. Models can still misinterpret the context of a factually correct source (e.g., treating a rhetorical question in a title as a statement of fact).*
- ***Model Size vs. Factuality:*** *Research suggests that larger model sizes do not inherently guarantee better factuality; data quality and the design of response strategies are more critical factors.*
- ***Intentional "Hallucination":*** *The survey notes that not all deviations from a source are errors. In creative domains or for "knowledge-informed abstraction" in legal summaries, intentional deviations (desirable creativity) should not be penalized as hallucinations.*

Research Gaps

- ***Long-form Consistency:*** *There is limited research on detecting contradictions within very long model outputs, such as inconsistencies between the beginning and the end of a generated narrative.*
- ***Pragmatic Nuance:*** *Detecting "literally true but contextually misleading" statements remains a significant challenge that current benchmarks do not fully assess.*
- ***LLM Evaluator Reliability:*** *While LLMs are increasingly used as "judges" to detect hallucinations in other models, ensuring their own stability, lack of bias, and intellectual honesty remains an open challenge.*
- ***Domain-Specific Gaps:*** *High-stakes scientific domains like materials science or drug discovery still require the development of robust, specialized hallucination detection methods.*

Source Association Summary

Claim Type	*Primary Sources*
Foundational RAG Mechanics	*Meta Research, Wikipedia*
Hallucination Taxonomy (SF vs WF)	*Survey on NLG Hallucinations*
Advanced Mitigation (MCD/Contrastive)	*Zhu et al. (NIPS)*
Industrial/Log Applications	*SvedbergOpen*
Causes and Practical Fixes	*Mindee Blog*
Structured Output Improvements	*Béchard & Ayala (arXiv)*

Note: *Content regarding Source 3 (MEGA-RAG) could not be fully summarized as the provided excerpt consisted of a reCAPTCHA error page.*

- *Draft (Claude):*
  - I then gave this output to Claude with the same summary above set as instructions and the following prompt: I need a first draft.
  - After confirming what kind of draft, I needed from the summary it gave:

Retrieval-Augmented Generation and the Mitigation of Hallucination in Large Language Models

1. Introduction

Hallucination—the generation of content that is unfactual or unsupported by an underlying source—remains one of the central obstacles to the reliable deployment of large language models (LLMs). Retrieval-Augmented Generation (RAG) has emerged as one of the most widely adopted mitigation strategies, grounding model outputs in externally retrieved evidence rather than relying solely on knowledge encoded during pre-training (Meta Research; Wikipedia). This section reviews the mechanisms by which RAG reduces hallucination, summarizes empirical evidence of its efficacy, and discusses the limitations and open questions that persist even in retrieval-augmented systems.

2. Mechanisms of Hallucination Mitigation via RAG

RAG architectures combine parametric memory—knowledge implicitly encoded in model weights during pre-training—with non-parametric memory, typically an external document corpus such as Wikipedia (Meta Research). Rather than depending exclusively on internalized patterns, the model retrieves relevant passages at inference time and conditions its generation on this evidence, effectively allowing it to "look up" information rather than reconstruct it from memory. A practical advantage of this design is adaptability without retraining: because the retrieval corpus is decoupled from model parameters, a system's effective knowledge can be updated simply by refreshing the underlying document store, avoiding the substantial cost of retraining the model itself (Meta Research).

Several refinements extend this basic architecture. Self-RAG introduces a self-correction mechanism in which the model generates reflection tokens that critique its own retrieved context and assess the degree to which that context supports its resulting claims. Contrastive decoding approaches, by contrast, mitigate hallucination at the decoding stage: a base model's output distribution is contrasted against that of a hallucination-prone counterpart model in order to suppress tokens likely to be erroneous (Zhu et al.). In domains where query vocabulary diverges from the technical jargon of source documents, hybrid search—combining dense, semantic retrieval with sparse, keyword-based methods such as BM25—has been used to bridge this vocabulary gap (SvedbergOpen).

3. Empirical Evidence of Efficacy

Empirical results across several domains support the claim that retrieval augmentation measurably improves factual grounding. In open-domain question answering, RAG-based systems have achieved state-of-the-art results on benchmarks including NaturalQuestions and WebQuestions, in some cases producing correct answers even when the answer was not stated verbatim in any single retrieved passage (Meta Research). In multi-model contrastive decoding (MCD), pairing a "truthful" model with an "evil," hallucination-prone model to guide token selection improved TruthfulQA scores by 12.3 to 18.7 percentage points relative to standard greedy decoding (Zhu et al.). Domain-specific applications show similar gains. In an industrial setting involving automotive embedded systems, RAG-driven hybrid search achieved higher precision (0.93) on exact-match log queries than standard keyword search (0.91), and retained relatively high precision (0.81) even under vocabulary mismatch between query and log terminology (SvedbergOpen). RAG has likewise been shown to improve the quality of structured outputs, such as workflow generation from natural-language requirements, reducing errors relative to non-retrieval-augmented baselines (Béchard & Ayala).

4. Persistent Limitations and Taxonomic Nuances

Despite these gains, RAG does not eliminate hallucination and should not be treated as a comprehensive solution. Residual hallucinations arise from at least three sources: poor retrieval quality, in which irrelevant or outdated documents are fetched; misleading fusion, in which the model incorrectly synthesizes facts drawn from multiple documents; and over-reliance on generative capacity at the expense of the retrieved source (Mindee Blog). Models may also misinterpret the pragmatic context of an otherwise factually accurate source—for example, treating a rhetorical question that appears in a document title as if it were an assertion of fact.

The literature distinguishes two related but distinct evaluative constructs: source faithfulness (SF), which measures whether an output is consistent with a specific provided source, and world factuality (WF), which measures whether an output aligns with general facts about the world (Survey on NLG Hallucinations). This distinction matters because a response can be faithful to a flawed or outdated source while still being factually incorrect in the world, or vice versa. A further nuance concerns intentional deviation from a source: in creative writing or in "knowledge-informed abstraction" such as legal summarization, deliberate departures from the source text may reflect desirable creativity or synthesis rather than error, and should not automatically be penalized as hallucination (Survey on NLG Hallucinations). Relatedly, evidence suggests that model scale alone does not guarantee improved factuality; data quality and the design of response strategies appear to be more consequential factors than parameter count.

5. Open Research Gaps

Several gaps remain inadequately addressed in the current literature. First, long-form consistency—detecting contradictions between, for instance, the beginning and end of an extended generated narrative—has received comparatively little empirical attention. Second, pragmatic nuance, particularly the detection of statements that are literally true yet contextually misleading, remains largely outside the scope of existing benchmarks. Third, as LLMs are increasingly deployed as automated judges to detect hallucination in other models' outputs, questions about the stability, bias, and reliability of these LLM evaluators themselves remain open. Finally, high-stakes scientific domains such as materials science and drug discovery still lack robust, domain-specialized hallucination detection methods suited to their distinctive evidentiary standards.

6. Conclusion

Retrieval-Augmented Generation represents a substantial and empirically supported advance in grounding LLM outputs, with demonstrated gains across open-domain question answering, industrial log analysis, and structured generation tasks. However, the evidence reviewed here indicates that RAG mitigates rather than eliminates hallucination: retrieval failures, fusion errors, and over-reliance on parametric generation continue to produce unfaithful or factually incorrect outputs. Future work addressing long-form consistency, pragmatic ambiguity, evaluator reliability, and domain-specific detection methods will be necessary to close the remaining gaps identified in this literature.

Note: This section synthesizes findings from a prior research summary; full bibliographic details for cited sources (e.g., Meta Research, Zhu et al., Béchard & Ayala, SvedbergOpen, Mindee Blog, Survey on NLG Hallucinations) should be added once complete citation records are available.

- *Critique (Claude):*
  - I then gave Claude this prompt: Critique this draft against the original research objective. Do not rewrite it yet. Evaluate it for: factual accuracy and unsupported claims, whether each major claim is supported by the provided research, citation completeness and traceability, whether the content has moved beyond the original research scope, logical structure and consistency, missing evidence or important perspectives, unclear or overstated conclusions and, academic writing quality. For each issue, explain what is wrong, why it matters, and what should be changed. Separate factual/source issues from writing/structure issues.
  - The output I got is:

Factual Accuracy & Unsupported Claims

1. Unsupported framing claims added in the introduction. "one of the central obstacles to the reliable deployment of large language models" and "one of the most widely adopted mitigation strategies" are not stated anywhere in the source summary. These are motivational framing claims the draft introduced on its own. Why it matters: in a lit review, every claim should trace back to the reviewed material — inserting unsourced framing, even if plausible, blurs the line between synthesis

and original assertion. Fix: either cite a source for these claims or soften them to explicitly mark them as contextual framing rather than findings.

2. Ambiguous statistic presented with false precision. The original says MCD improved TruthfulQA scores by "+12.3% to +18.7%" — it does not specify whether this is a percentage-point gain or a relative percentage improvement. The draft states "improved TruthfulQA scores by 12.3 to 18.7 percentage points," which asserts a specific unit of measurement the source never confirmed. This is a factual overreach dressed up as precision. Fix: preserve the ambiguity ("by 12.3–18.7%") rather than resolving it in one direction.

3. An added inferential claim about SF vs. WF. "This distinction matters because a response can be faithful to a flawed or outdated source while still being factually incorrect in the world, or vice versa" is not in the original document. It's a reasonable inference, but it's presented as if it were part of the reviewed literature's argument rather than the author's own elaboration. Fix: attribute this explicitly as analytical commentary (e.g., "This distinction is consequential because...") rather than folding it into a sentence that reads as reported literature content.

4. Unattributed generalization about model scale. "Evidence suggests that model scale alone does not guarantee improved factuality; data quality and the design of response strategies appear to be more consequential factors" has no citation at all, even though it corresponds directly to a specific bullet in the source material ("Model Size vs. Factuality"). Presenting this uncited makes it look like the author's own claim rather than a reported finding.

Claim Support Check (against the provided research)

Draft claim	Supported?	Notes
Parametric/non-parametric memory, adaptability without retraining	Yes	Matches source closely, correctly cited (Meta Research)
Self-RAG description	Partially	Concept is in the source, but source material doesn't attribute it to a specific citation, and the draft invents none — see citation section below
Contrastive decoding description	Yes	Matches, correctly cited (Zhu et al.)
Hybrid search description	Loosely	The *general concept* is listed in the source as a standalone idea; the draft cites it to SvedbergOpen, which in the original is tied specifically to the *automotive log* case study, not the general concept. This conflates a general definition with a narrow empirical source.

Draft claim	Supported?	Notes
NaturalQuestions/WebQuestions SOTA claim	Yes, but inferred attribution	The source table doesn't explicitly map "Performance Benchmarks" to a source; the draft assumes Meta Research. Reasonable, but not explicitly confirmed.
MCD TruthfulQA figures	Yes	Correctly cited, though see precision issue above
Industrial log precision figures	Yes	Numbers match exactly, correctly cited
Structured output / workflow generation claim	Yes	Correctly cited (Béchard & Ayala)
Residual hallucination causes (retrieval quality, fusion, over-reliance)	Yes	Correctly cited (Mindee Blog)
Rhetorical-question-as-fact example	Yes, but uncited	Matches source content but is stated without a citation, unlike the surrounding sentences
SF vs. WF definitions	Yes	Correctly cited (Survey on NLG Hallucinations)
Intentional deviation / creative domains	Yes	Correctly cited
Model scale vs. factuality	Yes, but uncited	See above
All research gaps (long-form consistency, pragmatic nuance, evaluator reliability, domain-specific gaps)	Yes, but **entirely uncited**	Section 5 has zero citations across four distinct claims

---

Citation Completeness & Traceability

- **Section 5 (Open Research Gaps) has no citations whatsoever.** This is the most significant traceability gap in the draft — four separate claims are presented as established findings with no source attached at all, even though the original document doesn't attribute these to specific sources either. At minimum, the section should attribute these to "the surveyed literature" broadly or flag that they are gaps *identified across* the reviewed sources rather than findings from any one of them.
- **Self-RAG and the rhetorical-question example are both uncited**, breaking the pattern of attribution used elsewhere in the document. A reader can't tell whether these are established claims or the author's own interpretation.
- **Citation format is inconsistent and not real academic citation.** "(Meta Research)," "(Wikipedia)," "(SvedbergOpen)," "(Mindee Blog)" are organizational labels, not author-year citations, while "(Zhu et al.)" and "(Béchard & Ayala)" mimic real academic citation style. This inconsistency makes the sourcing look more rigorous than it is — a reader could mistake

"Meta Research" for a properly formatted citation when it's actually a placeholder. The draft's closing note flags this, but the inconsistency itself is a structural problem, not just a missing detail.
- **"Logits Extrapolating"** — a distinct concept listed in the source material — is omitted entirely, with no explanation for its exclusion.

Scope Creep Beyond the Original Research

- The intro's claims about RAG's centrality and adoption (flagged above) are the clearest instance of the draft going beyond the source material's actual content.
- Framing Self-RAG, contrastive decoding, and hybrid search all as "refinements [that] extend this basic architecture" overstates their relationship to one another. Contrastive decoding as described in the source is a decoding-time technique that doesn't inherently depend on retrieval; grouping it with RAG-specific extensions implies an architectural relationship the source doesn't establish. This is a conceptual overreach, not just a stylistic choice.
- The added interpretive sentence on SF/WF (flagged above) is a second, smaller instance of the draft extending past what's supported.

Logical Structure & Consistency

- Overall structure (Intro → Mechanisms → Evidence → Limitations → Gaps → Conclusion) is logical and appropriate for a lit review section — no major sequencing issues.
- The mechanisms section (Section 2) packs three conceptually distinct ideas — self-correction, decoding-time contrast, and retrieval-time hybrid search — into two dense paragraphs under a single "refinements extend this architecture" framing, which (as noted above) implies a closer relationship between them than the source supports.
- The conclusion mostly restates prior sections rather than adding new synthesis; this is acceptable for a short section but means the "conclusion" is really a summary, not an original closing argument.

Missing Evidence or Perspectives

- **Logits Extrapolating** is dropped entirely with no acknowledgment.
- **No discussion of source heterogeneity.** The reference base mixes a NeurIPS paper (Zhu et al.), an arXiv preprint (Béchard & Ayala), a corporate research lab reference (Meta Research), a general encyclopedia (Wikipedia), and two blog/vendor-style sources (Mindee Blog, SvedbergOpen). An academic lit review would typically flag that these sources carry very different evidentiary weight — a company blog post is not equivalent to a peer-

reviewed benchmark result. The draft treats all citations as equally authoritative, which is a meaningful omission for an academic audience.
- No mention that Source 3 (MEGA-RAG) was unusable due to a reCAPTCHA error — worth a brief methodological note in an academic section, since acknowledging a source-collection limitation is standard practice.

Unclear or Overstated Conclusions

- Aside from the uncited "model scale" claim and the added SF/WF inference already noted, the conclusion's claims are appropriately hedged ("mitigates rather than eliminates," "future work... will be necessary") and don't overstate beyond the evidence.

Academic Writing Quality

- Prose is clear, well-organized, and appropriately formal.
- The main quality issue is the citation inconsistency described above — mixing organization-name citations with author-name citations undermines the credibility the formal tone is trying to establish.
- A few sentences (e.g., the SF/WF elaboration) blend original synthesis with reported findings without a clear seam, which is a writing-craft issue as much as a sourcing issue: a stronger draft would signal explicitly when it's moving from "the literature says" to "this implies."

Summary Split

Factual / sourcing issues: unsupported intro framing claims; false precision on the MCD statistic; added SF/WF inference presented as reported finding; uncited model-scale claim; uncited Self-RAG and rhetorical-question claims; entirely uncited research-gaps section; inconsistent/placeholder citation format; missing Logits Extrapolating concept; no acknowledgment of source heterogeneity or the excluded MEGA-RAG source.

Writing / structural issues: conflation of contrastive decoding with RAG-specific "refinements" in Section 2; conclusion functions as summary rather than fresh synthesis; dense pairing of three distinct

- *Revise (Claude):*
    - I then gave Claude this prompt: Revise the draft using the critique above. Preserve claims that are adequately supported, remove or qualify unsupported claims, keep the scope focused on the research objective, and improve clarity and academic structure. Do not invent sources, evidence, statistics, or citations. Where source verification is still required, explicitly flag it rather than making up information.

- The output I got is:

Retrieval-Augmented Generation and the Mitigation of Hallucination in Large Language Models

1. Introduction

Hallucination—the generation of content that is unfactual or unsupported by an underlying source—is a well-documented limitation of large language models (LLMs). Retrieval-Augmented Generation (RAG) is a mitigation approach that grounds model outputs in externally retrieved evidence rather than relying solely on knowledge encoded during pre-training [Meta Research]. This section reviews the mechanisms by which RAG is intended to reduce hallucination, summarizes empirical evidence bearing on its efficacy, and discusses the limitations and taxonomic distinctions that persist even in retrieval-augmented systems, based on the set of sources reviewed for this summary.

Editorial note: Broader claims about RAG's overall centrality to the field or its adoption rate are not made here, as they were not supported in the reviewed material. The scope of this section is limited strictly to what the reviewed sources report.

2. Mechanisms of Hallucination Mitigation

2.1 Core Retrieval Architecture

The core RAG architecture combines parametric memory—knowledge implicitly encoded in model weights during pre-training—with non-parametric memory, typically an external document corpus such as Wikipedia [Meta Research]. Rather than depending exclusively on internalized patterns, the model retrieves relevant passages at inference time and conditions its generation on this evidence. A practical advantage of this design is that a system's effective knowledge can be updated by refreshing the retrieval corpus, without retraining the model itself [Meta Research].

2.2 Related but Distinct Mitigation Techniques

The reviewed material also describes several other techniques aimed at reducing hallucination. These are presented here as related approaches rather than as components of a single unified RAG architecture, since the source material does not establish that relationship explicitly.

Self-RAG is a self-correction approach in which the model generates reflection tokens that critique its own retrieved context and assess the degree to which that context supports its resulting claims. *Editorial note: no specific citation was attached to this concept in the source material; attribution requires verification before use in a submitted paper.*

Contrastive decoding is a decoding-time technique, distinct from retrieval itself, in which a base model's output distribution is contrasted against that of a hallucination-prone counterpart model in order to suppress tokens likely to be erroneous (Zhu et al.). *Editorial note: full bibliographic details for "Zhu et al." have not been verified.*

Hybrid search—combining dense, semantic retrieval with sparse, keyword-based methods such as BM25—is described in the source material as a general technique for bridging vocabulary gaps between a query and a document corpus. The specific precision figures reported for this technique (see Section 3) come from a single automotive log-analysis case study [SvedbergOpen] and should not be read as a general performance benchmark for hybrid search.

Logits extrapolating is a technique for approximating deeper transformer layers by analyzing token probability trajectories across existing layers, intended to improve factual expressiveness. *Editorial note: the source material does not elaborate on this technique beyond a brief definition, and no performance evidence for it was provided; it is included here for completeness but should not be treated as empirically supported until a fuller source is located.*

3. Empirical Evidence of Efficacy

In open-domain question answering, RAG-based systems have achieved strong results on benchmarks including NaturalQuestions and WebQuestions, in some cases producing correct answers even when the answer was not stated verbatim in any single retrieved passage [Meta Research]. *Editorial note: the source material's attribution table does not explicitly confirm that this benchmark claim originates from the Meta Research source rather than another; this attribution should be verified.*

In multi-model contrastive decoding (MCD), pairing a "truthful" model with a hallucination-prone model to guide token selection improved TruthfulQA scores by 12.3% to 18.7% relative to standard greedy decoding (Zhu et al.). The source material does not specify whether this is a relative or percentage-point improvement, and that ambiguity is preserved here rather than resolved.

In an industrial case study involving automotive embedded systems, RAG-driven hybrid search achieved higher precision (0.93) on exact-match log queries than standard keyword search (0.91), and retained relatively high precision (0.81) even under vocabulary mismatch between query and log terminology [SvedbergOpen]. As noted in Section 2.2, this result is specific to one industrial application and should not be generalized to hybrid search performance overall without further evidence.

RAG has also been reported to improve the quality of structured outputs, such as workflow generation from natural-language requirements, reducing errors relative to non-retrieval-augmented baselines (Béchard & Ayala). *Editorial note: full bibliographic details for "Béchard & Ayala" have not been verified.*

4. Persistent Limitations and Taxonomic Distinctions

The reviewed material indicates that RAG does not eliminate hallucination. Residual hallucinations are attributed to at least three sources: poor retrieval quality, in which irrelevant or outdated documents are fetched; misleading fusion, in which the model incorrectly synthesizes facts drawn from multiple documents; and over-reliance on generative capacity at the expense of the retrieved source [Mindee Blog]. The source material also notes that models can misinterpret the pragmatic context of an otherwise factually accurate source—for example, treating a rhetorical question in a document title as an assertion of fact. *Editorial note: this example was not explicitly attributed to a specific source in the material provided; attribution should be verified before citation.*

The reviewed literature distinguishes two related but distinct evaluative constructs: source faithfulness (SF), which measures whether an output is consistent with a specific provided source, and world factuality (WF), which measures whether an output aligns with general facts about the world [Survey on NLG Hallucinations]. *As an interpretive point rather than a reported finding: this distinction implies that an output could be faithful to a flawed or outdated source while still being factually incorrect in the world, or the reverse. This inference is offered here for clarity and is not itself a claim drawn directly from the source material.*

A further distinction concerns intentional deviation from a source: in creative writing or in "knowledge-informed abstraction" such as legal summarization, deliberate departures from source text may reflect desirable synthesis rather than error, and the reviewed material suggests these should not automatically be penalized as hallucination [Survey on NLG Hallucinations]. Separately, the material also raises the point that model scale alone does not appear to guarantee improved factuality, suggesting instead that data quality and response-strategy design may be more consequential. *Editorial note: this claim was not attributed to a specific source in the material provided; it should be treated as an unverified observation pending confirmation of its origin.*

5. Note on Source Basis

The sources underlying this review are heterogeneous in evidentiary weight: they include what appears to be a peer-reviewed or conference paper (Zhu et al.), an arXiv preprint (Béchard & Ayala), a research-lab reference and a general encyclopedia entry (Meta Research; Wikipedia), and two blog- or vendor-style sources (Mindee Blog; SvedbergOpen). These should not be treated as equally authoritative, and any submitted version of this section should replace the working labels used here with verified, fully formatted citations. One additional source (referred to in the original material as MEGA-RAG) could not be incorporated, as the excerpt available for it consisted only of a CAPTCHA error page rather than usable content.

6. Open Research Gaps

The following gaps are identified across the reviewed material as a whole; the source summary did not attribute them to individual sources, and that attribution has not been independently verified here. First, long-form consistency—detecting contradictions between, for instance, the beginning and end of an extended generated narrative—appears to have received comparatively little empirical attention. Second, pragmatic nuance, particularly the detection of statements that are literally true yet contextually misleading, is described as remaining largely outside the scope of existing benchmarks. Third, as LLMs are increasingly used as automated judges to detect hallucination in other models' outputs, questions about the stability, bias, and reliability of these LLM evaluators are described as unresolved. Finally, high-stakes scientific domains such as materials science and drug discovery are described as still lacking robust, domain-specialized hallucination detection methods.

7. Conclusion

Within the scope of the material reviewed, Retrieval-Augmented Generation is associated with improved grounding of LLM outputs across open-domain question answering, at least one industrial log-analysis case study, and structured generation tasks. At the same time, the reviewed material indicates that RAG mitigates rather than eliminates hallucination: retrieval failures, fusion errors, and over-reliance on parametric generation are all reported as continuing sources of unfaithful or factually incorrect output. Several gaps—long-form consistency, pragmatic ambiguity, evaluator reliability, and domain-specific detection methods—remain open according to the reviewed material. Before this section is used in a submitted paper, the citation placeholders and unattributed claims flagged throughout should be resolved against verified primary sources.

- *Human Review*: I verified the sources, facts and citation. The final editing needs to be done by me as well.
- *Failure points:*
    - Citation details are incomplete.
    - The scope of the research is broader than originally given and intended.
    - Verifying the sources and facts was still required.
- Time comparison for gathering, synthesizing, and drafting.
    - Manual process: 6+ hours
    - AI workflow: 30 – 45 minutes
    - Approximately 88-92% less active time

Run 2:

- *Topic:* AI University Chatbot Documentation
- *Gather:*
  - I used the code I had done so far and gave it to NotebookLM.
  - The prompt I gave it after attaching the materials is: I am documenting an AI-powered university voice assistant that I am developing as part of my internship capstone project. The system supports voice and text interaction and uses Retrieval-Augmented Generation (RAG) to retrieve information from university websites. Based only on the provided project materials, analyse the system and identify:
    1. The project's purpose and requirements
    2. The overall system architecture
    3. The technologies and services used
    4. The role of each major technology/component
    5. How information flows through the system from user input to response
    6. The major technical decisions I made and why they were made
    7. Problems or limitations encountered during development
    8. How those problems were addressed
    9. Current implementation status
    10. Any areas where the provided material is insufficient or ambiguous

    Do not invent details that are not supported by the provided sources. Clearly distinguish between information explicitly stated in the sources and reasonable interpretations.

    - The output I got is:

    Based on the provided source material, here is an analysis of the voice AI system for your internship capstone project.

    Project Purpose and Requirements

    The project's primary purpose is to create a voice AI agent capable of voice-based interaction, initially built using the LiveKit Agent Builder and then converted to a Python-based code project for further customization.

    Requirements identified in the source include:

    - **Infrastructure:** A LiveKit Cloud account and project credentials (URL, API Key, and Secret).
    - **Environment:** Installation of the LiveKit Agents for Python framework.
    - **Core Models:** Specialized models for Voice Activity Detection (VAD) and turn detection to manage conversation flow.
    - **Deployment:** Ability to run the agent both locally for development and in production via LiveKit Cloud.

Note: While your query mentions the purpose is a "university voice assistant" using "RAG" and "university websites," the provided source material does not explicitly mention these specific use cases or technologies.

System Architecture and Technologies

The architecture is centered around a **voice AI pipeline** that connects user interfaces to backend logic via a cloud-based real-time platform.

Technologies and Services Used:

- **LiveKit Agents for Python:** The core framework used to build and customize the agent's logic.
- **LiveKit Cloud:** The hosting and orchestration platform that manages real-time connectivity.
- **LiveKit Inference:** A service supporting over 50 model providers to handle AI processing, including "Realtime models".
- **Silero VAD & LiveKit Turn Detector:** Local models used for detecting when a user starts and stops speaking.
- **LiveKit CLI & MCP Server:** Developer tools used for deployment and for providing documentation context to AI coding assistants.
- **Frontend Templates:** Supported platforms include Web (React/Next.js), iOS/macOS, Flutter, React Native, Android, and Telephony (SIP).

Component Roles and Information Flow

1. **Frontend/User Interface:** Captures user input (voice or text) and transmits it to LiveKit Cloud.
2. **LiveKit Cloud:** Routes the real-time data between the user's device and the Python agent.
3. **VAD and Turn Detector:** These models analyze the audio stream to determine when the user is speaking and when they have finished a "turn," allowing the agent to know when to respond.
4. **LiveKit Inference/AI Models:** Once a turn is detected, the audio/text is processed by the AI pipeline (using providers like OpenAI or others) to generate a response.
5. **Agent Logic:** The Python code defines the agent's behavior, instructions, and any "tasks or handoffs" required for complex workflows.

Technical Decisions and Rationale

- **Transition from Builder to Code:** The project was converted from the LiveKit Agent Builder to a code-based project to allow for full customization and ownership of the deployment.
- **Modular Workflows:** The source recommends using **"tasks and handoffs"** instead of single, long instruction prompts. This was

decided to minimize latency and improve reliability by breaking the agent into focused, reusable components.
- **Local vs. Production Environments:** A decision was made to use separate LiveKit Cloud projects for development and production to avoid affecting production traffic during testing.
- **Tooling for AI-Assisted Development:** The inclusion of an AGENTS.md file and an MCP server for documentation indicates a decision to optimize the project for use with AI coding assistants like Claude Code or Cursor.

Problems, Limitations, and Resolutions

- **Latency and Reliability in Complex Agents:** Long prompts were identified as a potential cause for high latency and poor reliability. This is addressed by structuring the agent into **focused tasks**.
- **Configuration Complexity:** Setting up environment variables manually can be error-prone; the system addresses this by offering an **automatic setup** via the LiveKit CLI.
- **Production Monitoring:** Debugging agent behavior in the wild can be difficult. This is addressed through **LiveKit Observability**, which provides session insights and tracks latency metrics in production.

Current Implementation Status

The project has been successfully **converted to code** from the initial builder phase and is currently identical to production builder deployments. It is ready for local testing and customization of AI models and frontend integration.

Information Gaps and Ambiguities

The provided source material is **insufficient** in the following areas relative to your project description:

- **RAG Implementation:** There is no mention of Retrieval-Augmented Generation (RAG) or how external data (like university websites) is indexed or retrieved.
- **University-Specific Logic:** The source is a general-purpose README and contains no specific information regarding university services, student data, or campus information.
- **Model Selection:** While the source states that over 50 providers are supported, it does not specify which LLM or Text-to-Speech (TTS) provider is currently being used in your implementation.
- **Text Interaction:** The source focuses heavily on the "Voice AI pipeline"; while you mentioned text interaction support, the README details the voice-specific components (VAD, Turn Detection) much more extensively than text-only flows.

- *Synthesize (NotebookLM):*

- I then gave it this prompt: Synthesise the information into a structured technical overview that I can use as the basis for project documentation. Organise it into:

    - Project objective
    - Requirements
    - System architecture
    - Technology stack
    - Component responsibilities
    - End-to-end workflow
    - Key technical decisions and their reasoning
    - Challenges and solutions
    - Current outcome
    - Remaining limitations or future work

      For every technical claim, identify the supporting source. Do not invent implementation details that are not present in the provided material.

- The output I got is:

  Technical Overview: LiveKit Voice AI System

  This overview synthesizes the project details from the provided documentation to serve as a foundation for your capstone project documentation.

  Project Objective

  The primary goal is to deploy a **fully customizable Voice AI agent** capable of real-time interaction. The system is designed to transition from a managed builder environment to a self-hosted Python codebase, allowing for advanced customization and integration with AI-assisted development workflows.

  Requirements

  - **Core Infrastructure:** A LiveKit Cloud account with a unique Project URL, API Key, and API Secret.
  - **Software Environment:** Python-based environment with the livekit-agents framework installed.
  - **Developer Tools:** LiveKit CLI (v2.15.0 or higher) for documentation access, credential management, and production deployment.
  - **Local Assets:** Downloaded models for **Silero VAD** (Voice Activity Detection) and the **LiveKit turn detector**.

  System Architecture

The system utilizes a **Voice AI pipeline** architecture. It connects diverse client-side frontends to a Python-based agent backend through **LiveKit Cloud**, which orchestrates real-time data transmission. The architecture supports a modular workflow where complex logic is broken into focused components.

Technology Stack

- **Orchestration & Hosting:** LiveKit Cloud.
- **Agent Logic:** LiveKit Agents for Python.
- **Inference Engine:** LiveKit Inference, supporting over 50 model providers and "Realtime models".
- **Conversation Management:** Silero VAD and LiveKit turn detector.
- **Monitoring:** LiveKit Agent Observability.
- **AI Developer Support:** MCP server and AGENTS.md context for coding assistants (e.g., Claude Code, Cursor).

Component Responsibilities

- **LiveKit Cloud:** Routes real-time audio and data streams between the user and the agent.
- **VAD & Turn Detector:** Local models that identify when a user is speaking and when they have finished a statement, preventing interruptions and managing "turns".
- **LiveKit Inference:** Handles the heavy lifting of AI model execution, allowing the agent to utilize various LLM and TTS providers.
- **Python Agent Code:** Contains the specific instructions, tasks, and logic that define the agent's behavior.
- **Agent Observability:** Monitors session quality and tracks latency metrics for production debugging.

End-to-End Workflow

1. **Input:** A user provides voice or text input via a frontend (Web, Mobile, or Telephony).
2. **Detection:** The Silero VAD and Turn Detector models analyze the stream to identify a user's "turn".
3. **Processing:** The identified turn is processed through the **voice AI pipeline** using LiveKit Inference to generate an intelligent response.
4. **Response:** The agent transmits the generated response back to the user in real-time via LiveKit Cloud.
5. **Monitoring:** The entire session is logged and analyzed by Agent Observability for quality assurance.

Key Technical Decisions and Reasoning

- **Conversion from Builder to Code:** This was done to allow developers to "make it your own" and move away from the limitations of the production builder environment.
- **Modular "Tasks and Handoffs":** Instead of using long, singular instruction prompts, the system uses focused components to **minimize latency and improve reliability**.
- **Development Segregation:** The documentation recommends using a different LiveKit Cloud project for development to avoid impacting production traffic.

Challenges and Solutions

- **Latency in Complex Agents:** Addressed by structuring logic into **reusable components** rather than monolithic prompts.
- **Documentation Accessibility for AI Tools:** To aid AI coding assistants, the project includes an **MCP server** and an AGENTS.md file to provide necessary context to the LLM helping write the code.
- **Environment Configuration:** Manual setup is prone to error; the **LiveKit CLI** provides an "Automatic setup" to streamline credential configuration.

Current Outcome

The project is currently a **functional Python codebase** identical to production deployments from the LiveKit Agent Builder. It is ready for local testing via the terminal and is prepared for deployment to LiveKit Cloud.

Remaining Limitations or Future Work

- **Model Selection:** While the pipeline is established, the specific AI models (from the 50+ supported providers) must be selected and customized for the university use case.
- **Frontend Integration:** The project requires a frontend to be built or integrated using one of the provided templates (React, Flutter, etc.).
- **RAG Implementation:** *Note: The provided sources do not yet detail the implementation of Retrieval-Augmented Generation or university-specific website indexing; this remains an area for future implementation based on your project description.*

- *Draft (Claude):*
  - I gave synthesis to Claude with the following prompt: University Voice Assistant. The documentation should explain:

    1. Project objective
    2. System requirements

3. Architecture
4. Technology stack
5. Role of each major component
6. End-to-end system workflow
7. Key technical decisions and their reasoning
8. Challenges and how they were addressed
9. Current implementation status
10. Limitations and future work

Write for a technically literate reader who may not know the project beforehand. Explain technical components clearly without oversimplifying them.

Important: Use only information supported by the provided synthesis. Do not invent implementation details. If the synthesis identifies something as missing or uncertain, explicitly mark it as requiring verification rather than presenting it as implemented.

Keep the documentation professional, clear, technically accurate, and concise.

Source-grounded synthesis: (the output here).

- The output is:

University Voice Assistant — Technical Documentation (Draft v0.1)

1. Project Objective

The objective of this project is to deploy a **fully customizable Voice AI agent** capable of real-time, conversational interaction, adapted for a university use case. The system is being transitioned from a managed, low-code "builder" environment into a **self-hosted Python codebase**. This transition is the central design decision of the project: it trades the convenience of a managed builder for full control over agent logic, enabling deeper customization and integration with AI-assisted development workflows (e.g., AI coding assistants operating directly on the codebase).

Note for verification: The specific university-facing use case (e.g., admissions Q&A, campus navigation, course registration support) is not detailed in the source material and should be defined explicitly in the final documentation.

2. System Requirements

The following components and credentials are required to run and develop the system:

Category	Requirement
Core Infrastructure	A LiveKit Cloud account with a unique **Project URL**, **API Ke** and **API Secret**
Software Environment	A Python environment with the livekit-agents framework inst
Developer Tooling	**LiveKit CLI** (v2.15.0 or higher) — used for documentation access, credential management, and production deploymen
Local Assets	Locally downloaded models for **Silero VAD** (Voice Activity Detection) and the **LiveKit turn detector**

3. System Architecture

The system follows a **Voice AI pipeline** architecture, in which multiple client-side frontends connect to a Python-based agent backend through **LiveKit Cloud**, which handles orchestration of real-time data transmission.

Architecturally, the design favors **modularity**: rather than a single large instruction set driving the agent, complex behavior is broken down into smaller, focused components. This is intended to keep the system maintainable and performant as capabilities grow.

4. Technology Stack

Layer	Technology
Orchestration & Hosting	LiveKit Cloud
Agent Logic	LiveKit Agents (Python framework)
Inference Engine	LiveKit Inference — supports 50+ model providers and "Realtime models"
Conversation Management	Silero VAD + LiveKit turn detector
Monitoring	LiveKit Agent Observability
AI Developer Support	MCP server and AGENTS.md context file, for use with AI coding assistants (e.g., Claude Code, Cursor)

5. Role of Each Major Component

- **LiveKit Cloud:** Acts as the real-time transport layer, routing audio and data streams between the end user and the backend agent.
- **Silero VAD & Turn Detector:** Local models responsible for detecting when a user is speaking and when they have finished a conversational "turn." This prevents the agent from interrupting the user and manages conversational turn-taking.

- **LiveKit Inference:** Executes the AI model workloads — this is the layer that lets the agent call out to various LLM and text-to-speech (TTS) providers without the developer needing to manage that infrastructure directly.
- **Python Agent Code:** Contains the project-specific logic — instructions, tasks, and behavioral definitions that determine how the agent responds and acts.
- **Agent Observability:** Provides monitoring of session quality and latency metrics, intended to support production-level debugging and quality assurance.

6. End-to-End System Workflow

1. **Input:** The user provides voice or text input through a frontend (web, mobile, or telephony channel).
2. **Detection:** Silero VAD and the turn detector analyze the incoming stream to determine when the user's conversational turn has occurred.
3. **Processing:** The detected turn is passed through the voice AI pipeline, where LiveKit Inference generates the agent's response.
4. **Response:** The generated response is transmitted back to the user in real time via LiveKit Cloud.
5. **Monitoring:** The full session is logged and analyzed by Agent Observability for quality assurance purposes.

7. Key Technical Decisions and Reasoning

- **Builder-to-code conversion:** The move from the managed builder to a self-hosted codebase was made explicitly to let developers "make it your own," removing the constraints of the production builder environment.
- **Modular "tasks and handoffs" design:** Rather than relying on one long, monolithic instruction prompt, the system is deliberately structured into smaller, focused components. This decision is aimed at **minimizing latency and improving reliability**.
- **Development/production segregation:** The documentation recommends using a **separate LiveKit Cloud project for development**, so that testing and iteration do not impact live production traffic.

8. Challenges and How They Were Addressed

Challenge	Solution
Latency in complex agent behavior	Restructured logic into reusable, modular compone rather than a single large prompt

Challenge	Solution
Documentation accessibility for AI coding tools	Added an MCP server and an AGENTS.md file to g coding assistants the context needed to work effect on the codebase
Error-prone manual environment configuration	Adopted the LiveKit CLI's "Automatic setup" feature streamline credential and environment configuration

9. Current Implementation Status

The project currently exists as a **functional Python codebase**, equivalent to what would be produced by the LiveKit Agent Builder in a production deployment. At this stage, it is:

- Ready for **local testing via the terminal**
- Prepared for **deployment to LiveKit Cloud**

No further implementation details beyond this were specified in the source material, so any additional claims about current functionality (e.g., specific conversational capabilities already built) should be verified before inclusion.

10. Limitations and Future Work

- **Model selection:** The pipeline infrastructure is in place, but the specific AI models (from the 50+ supported providers) still need to be selected and tuned for the university use case. *(Requires verification/decision.)*
- **Frontend integration:** A frontend still needs to be built or integrated, using one of the available templates (e.g., React, Flutter). *(Requires verification of which template will be used.)*
- **Retrieval-Augmented Generation (RAG):** The provided source material does **not** describe any implementation of RAG or indexing of university-specific website content. This is an open area for future work and **should not be presented as implemented** until confirmed. *(Requires verification/further design.)*

This draft is derived strictly from the provided technical synthesis. Sections flagged "requires verification" indicate information not present in the source material and should be confirmed or completed before this document is finalized for submission.

- *Critique (Claude):*

- I then gave this to Claude and asked it to critique the first draft using the following prompt: Critique the following technical documentation draft for my University Voice Assistant project. Review it specifically for:

    1. Technical inaccuracies or claims that may not be supported by the source material
    2. Missing information that is necessary for understanding the system
    3. Unclear or overly complicated explanations
    4. Claims that need verification before I can include them
    5. Whether the architecture and component responsibilities are explained accurately
    6. Whether the workflow accurately represents how the system operates
    7. Any unnecessary information that should be removed
    8. Any places where the wording sounds generic or AI-generated rather than like a technical student documenting their own project

    Do not rewrite the document yet.

    Instead, provide:

    - What is correct
    - What needs verification
    - What is missing
    - What should be removed or simplified
    - Specific recommended changes

    Keep the critique concise and practical. Do not invent information that is not present in the source material.

- I got this output:

    Critique of Draft v0.1

    What's Correct

    - Core facts align with the synthesis: LiveKit Cloud/CLI requirements, VAD + turn detector role, LiveKit Inference (50+ providers), Agent Observability, MCP server/AGENTS.md, builder-to-code rationale, modular "tasks and handoffs" reasoning, dev/prod project segregation, and current status (functional codebase, ready for local testing, prepared for deployment).
    - The three "requires verification" flags (model selection, frontend, RAG) are correctly identified as unsupported by the source and correctly *not* presented as implemented.

What Needs Verification (beyond what's already flagged)

- **"real-time, conversational interaction"** in the objective — fine as a general description, but nothing in the source confirms latency targets, concurrency, or actual real-time performance has been tested. Word it as a design goal, not a demonstrated capability.
- **CLI version "v2.15.0 or higher"** — this is a specific, falsifiable claim. Confirm this is genuinely the version you used/were told to use, not a plausible-sounding default.
- **"equivalent to what would be produced by the LiveKit Agent Builder in a production deployment"** (Section 9) — this is a strong equivalence claim. The source says the codebase is "identical to production deployments from the LiveKit Agent Builder," which is close, but double check whether "equivalent" vs "identical" matters for your grading criteria — identical is a stronger, more specific claim than the paraphrase suggests.
- **Telephony as a frontend channel** (Section 6, step 1) — telephony is listed as a possible frontend type but there's no confirmation in the source that a telephony integration path was actually part of this project's scope, or just a generic example of a supported channel type.

What's Missing

- **No mention of what the university-specific customization actually consists of.** The doc explains the generic LiveKit pipeline well but says almost nothing about what *you* built versus what LiveKit provides out of the box. A reader can't tell where the "template" ends and your project begins. This is the single biggest gap — grading a capstone likely depends on demonstrating your contribution.
- **No environment/config details** — e.g., how credentials are actually loaded (env vars, .env file, LiveKit CLI setup command). Section 2 lists what's required but not how it's configured, which is a natural thing to include if you know it.
- **No mention of testing performed**, if any (even informal). Section 9 says "ready for local testing" but doesn't say whether you've run it.
- **No versioning/scope statement for the document itself** — fine for a draft, but before finalizing, a short "Project Status" line (e.g., what sprint/phase this represents) would help orient the reader.

What Should Be Removed or Simplified

- **Redundant restating of the objective paragraph verbatim in the pasted content** — the whole document appears twice in your submission; this is likely a copy-paste artifact, not a document issue, but worth double-checking your final file doesn't have this duplication.
- **The closing italicized disclaimer** ("This draft is derived strictly from the provided technical synthesis...") reads as boilerplate you'd remove once these items are verified. It's useful now, not useful in a final capstone submission — consider replacing with an actual project status note once finalized.
- **Section 8's table wording is slightly repetitive with Section 7** (modular design is explained as both a "key decision" and a "solution to a challenge"). Not wrong, but consider merging or cross-referencing rather than explaining the same idea twice from two angles.

Architecture & Workflow Accuracy

- Architecture (Section 3) and workflow (Section 6) are internally consistent with each other and with the component table — no contradictions.
- One imprecision: Section 6 implies a fairly clean linear pipeline (detect → process → respond → monitor), but real voice AI systems typically have VAD/turn-detection running *continuously and concurrently* with generation, not as a strict step-by-step handoff. This is a simplification that's reasonable for an overview but could read as technically imprecise to someone familiar with streaming audio pipelines. Consider a caveat like "conceptually sequential, though components operate concurrently in practice" — but *only if you can verify that's true for your implementation*; otherwise leave as is and flag it.

Wording That Sounds Generic/AI-Generated

This is the area most worth revising before submission as *your own* documentation:

- Phrases like **"trades the convenience of a managed builder for full control over agent logic"**, **"acts as the real-time transport layer"**, and **"is intended to keep the system maintainable and performant as capabilities grow"** are polished but impersonal — they read like generic SaaS documentation rather than a student describing their own build.
- The em-dash-heavy, tightly parallel bullet structure throughout (especially Sections 5, 7, 8) is a stylistic tell of AI generation. A student author would likely vary

sentence length more and occasionally editorialize (e.g., "this took a few iterations to get right" or "I chose X because Y was too slow in testing").
- **Recommendation:** Before final submission, rewrite key explanatory sentences in your own voice, add at least one first-person reflection per major section (why *you* made a choice, what surprised you), and vary sentence rhythm — especially in Sections 3, 5, and 7, which currently read the most templated.

Specific Recommended Changes

1. Add a short "What I built / customized" subsection distinguishing your work from the LiveKit default pipeline.
2. Soften "real-time" and "identical to production" claims to match exact source wording, or verify against your own testing.
3. Confirm the CLI version number is accurate.
4. Add a line on environment/credential configuration if you know it.
5. Rewrite 3–5 key sentences per section in first-person, less uniform prose to reduce the AI-generated feel.
6. Replace the closing disclaimer with a genuine project status statement once verification items are resolved.
7. Consider a caveat on the workflow diagram noting VAD/turn-detection is continuous rather than strictly sequential — only if accurate to your setup.

- *Revise (Claude):*
  - I used this prompt: Revise Draft v0.1 using the critique above. Keep the technical information accurate and do not invent anything that was not supported by the original research or my project information. Address the specific issues identified in the critique, especially:

    - distinguish clearly between what LiveKit provides and what I have actually built/customised;
    - remove or soften unsupported claims;
    - improve the explanation of my technical decisions;
    - reduce generic/AI-sounding language;
    - make the writing sound like me: professional, clear, technically accurate, confident, open to discussion where appropriate, and polite;
    - avoid unnecessary buzzwords and exaggerated claims;
    - keep the structure practical and suitable for technical project documentation.

    Where the critique says something needs verification but I have not provided enough information, **do not guess**. Mark it clearly as [VERIFY] instead. Keep the document focused on the actual

project and my contribution rather than turning it into generic LiveKit documentation. Produce Draft v0.2 as the revised version only.

- The output I got is:

University Voice Assistant — Technical Documentation (Draft v0.2)

1. Project Objective

The goal of this project is to build a customizable Voice AI agent for a university use case, moving it from a managed, low-code "builder" environment into a self-hosted Python codebase that I control directly.

The reason for making this move is control: the builder environment is convenient to get started with, but it limits how far you can customize agent behavior. Working from the underlying Python codebase gives me direct access to the agent logic and lets me integrate AI-assisted development tools (e.g., Claude Code, Cursor) directly against the code.

[VERIFY] The specific university-facing use case this assistant is meant to support (e.g., admissions questions, campus navigation, course registration) is not yet defined in this document and should be added once finalized.

[VERIFY] What I have specifically built or customized beyond the default LiveKit pipeline is not yet documented here — see Section 5 for where this needs to be filled in.

2. System Requirements

To run and develop the system, the following are required:

Category	Requirement
Core Infrastructure	A LiveKit Cloud account with a Project URL, API Key, and API Secret
Software Environment	Python, with the livekit-agents framework installed
Developer Tooling	LiveKit CLI **[VERIFY exact version used]** — used for documentation access, credential management, and deployment
Local Assets	Locally downloaded models for Silero VAD (Voice Activity Detection) and the LiveKit turn detector

[VERIFY] How credentials are configured in my setup (e.g., .env file, CLI automatic setup, manual environment variables) is not yet documented — worth adding once confirmed.

3. System Architecture

The system is built around a voice AI pipeline: client-side frontends (web, mobile, or telephony) connect to a Python-based agent backend, with LiveKit Cloud handling the real-time transport of audio and data between the two.

The agent logic itself is structured modularly — rather than one long instruction set driving all agent behavior, the logic is broken into smaller, focused components. I've kept this structure because it's easier to reason about and modify individual pieces of behavior without affecting the whole system, and it keeps response latency down as the agent's responsibilities grow.

It's worth noting that in practice, voice-turn detection and response generation don't happen as a strictly sequential handoff — VAD and turn detection run continuously alongside the rest of the pipeline. The workflow in Section 6 describes the logical order of operations, not necessarily the exact runtime concurrency. **[VERIFY]** if a more precise description of the concurrency model is needed for this documentation.

4. Technology Stack

Layer	Technology
Orchestration & Hosting	LiveKit Cloud
Agent Logic	LiveKit Agents (Python framework)
Inference	LiveKit Inference — supports 50+ model providers, including realtime models
Conversation Management	Silero VAD + LiveKit turn detector
Monitoring	LiveKit Agent Observability
AI-Assisted Development	MCP server + AGENTS.md context file, for use with AI coding assistants

5. Component Responsibilities — Platform vs. My Work

This section separates what LiveKit provides out of the box from what is specific to this project. As of this draft, most of what's documented below reflects the LiveKit platform itself; the university-specific customization still needs to be filled in.

Component	Provided by LiveKit	My Customization
LiveKit Cloud	Real-time transport layer routing audio/data between user and agent	—
Silero VAD & Turn Detector	Detects when a user is speaking and when a turn ends, to avoid interrupting the user	**[VERIFY]** any tuning of sensitivity/thresholds for this use case
LiveKit Inference	Executes LLM/TTS calls across supported providers	**[VERIFY]** which specific model(s) selected for the university agent, and why

Component	Provided by LiveKit	My Customization
Python Agent Code	Framework for defining agent behavior	**[VERIFY]** the specific instructions, tasks, and conversational logic I've written for the university use case — this is likely the most important gap to fill in for a capstone submission
Agent Observability	Session/latency monitoring	**[VERIFY]** whether this has been configured or reviewed yet

I'd recommend prioritizing the "Python Agent Code" row above before finalizing this document — it's the part of the system that actually represents my contribution to the project, and right now it's the least documented.

6. End-to-End Workflow

1. **Input:** The user provides voice or text input via a frontend (web, mobile, or telephony).
2. **Turn detection:** Silero VAD and the turn detector monitor the stream and identify when the user has finished a conversational turn.
3. **Processing:** The agent processes that turn through the pipeline, using LiveKit Inference to generate a response.
4. **Response:** The response is sent back to the user in real time via LiveKit Cloud.
5. **Monitoring:** The session is logged by Agent Observability for later review.

This is the logical flow of the pipeline as designed. **[VERIFY]** I haven't yet confirmed end-to-end behavior through my own testing — once I have, this section should note what was actually observed (e.g., latency, turn-taking accuracy) rather than just the intended design.

7. Key Technical Decisions

Moving from builder to code. I made this switch because the builder environment doesn't give enough control to customize agent behavior for a specific use case. Working in code means more setup effort, but it's necessary for the level of customization this project needs.

Modular agent logic ("tasks and handoffs") instead of one long prompt. A single large instruction prompt tends to slow the agent down and makes behavior harder to predict as more requirements get added. Splitting logic into smaller, focused components was a deliberate choice to keep latency lower and make the system easier to debug and extend. **[VERIFY]** I haven't yet measured the latency difference directly — if I have data on this, it would strengthen this section.

Separate LiveKit Cloud projects for development and production. I'm keeping development work in a separate project so that testing doesn't affect live traffic once the assistant is deployed for actual use.

8. Challenges and How I Addressed Them

Challenge	How I Addressed It
Agent latency growing as logic gets more complex	Split logic into smaller, modular components rather than one large prompt
AI coding assistants lacking context on the project structure	Added an MCP server and an AGENTS.md file so tools like Claude Code or Cursor have the context needed to work on the codebase effectively
Manual credential/environment setup being error-prone	Used the LiveKit CLI's automatic setup rather than configuring things by hand

[VERIFY] If I've run into any university-specific challenges (e.g., handling ambiguous student questions, integrating with university data sources), those should be added here — this table currently only reflects platform-level challenges, not use-case-specific ones.

9. Current Implementation Status

At this stage, the project is a working Python codebase, structurally the same as what the LiveKit Agent Builder would produce for a production deployment. It is:

- Ready for local testing via the terminal
- Set up to be deployed to LiveKit Cloud

[VERIFY] Whether I've actually run local tests yet, and what the results were, isn't documented here — I should add this once I've done a testing pass.

[VERIFY] No university-specific functionality (e.g., specific conversation flows, integrations, or data sources) is confirmed as implemented yet. Until that's built and confirmed, this section should not claim more than "the underlying pipeline works" — the university-specific agent behavior is still to come.

10. Limitations and Future Work

- **Model selection:** I still need to choose and configure the specific model(s) from LiveKit Inference's supported providers for this use case. **[VERIFY / decision needed]**
- **Frontend:** No frontend has been built or integrated yet. LiveKit provides template options (e.g., React, Flutter) as starting points. **[VERIFY which template, if any, I plan to use]**
- **RAG (Retrieval-Augmented Generation):** Not implemented. If the assistant needs to answer questions using university-specific content (e.g., indexed pages from the university website), this will need to be designed and built separately — nothing in the current pipeline provides this. This is likely the largest remaining piece of work.
- **University-specific agent logic:** As noted in Section 5, this is the part of the project that most needs to be filled in and documented — everything else so far describes the underlying platform rather than my own contribution.

This is a working draft. Items marked [VERIFY] indicate information I need to confirm, decide, or document before this is submission-ready — they are not gaps in the underlying LiveKit platform, but gaps in describing my own work on top of it.

- *Human Review:* I verified if all the claims are true and matches my working.
- *Failure Points:*
  - The AI failed to document my work vs the LiveKit architecture.
  - AI did not cover all the points of my project and simplified the voice pipeline.
  - The draft was initially too generic despite the material provided.
- Time Comparison:
  - Manual: 3 hours
  - AI: 30 minutes
  - Improvement: AI did the work 6 times faster

Run 3:

- *Topic:* Agents VS Workflows & MCP
- *Gather (NotebookLM):*
- I gave AI the following prompt: The topic is Agents VS workflows and MCP. Gather all credible sources covering, AI agents (what they are, architecture, planning, tool use, memory, and examples), AI workflows (what they are, sequential workflows, difference between workflows and agents), MCP (what it is, why it was made, client and server architecture, tools, resources, prompts, how an LLM interacts with it, examples), agents vs workflows (when to use each of them, key differences, when we can combine them for optimized results).
- *Synthesis (NotebookLM):*
- I then asked it to: I need a research synthesis based only on the sources provided. Topic: AI agents, AI workflows, and Model Context Protocol (MCP). Organize the synthesis into:

  - Main findings
  - Important concepts
  - Supporting evidence
  - Contradictions, disagreements, or nuances between sources
  - Research/knowledge gaps
  - Sources associated with each major claim

  Specifically explain:

  1. What an AI agent is and how it works.
  2. What an AI workflow is and how it works.
  3. The key differences between agents and workflows.
  4. What MCP is and what problem it solves.
  5. How MCP works technically, including the relationship between the model, MCP client, MCP server, tools, resources, and prompts.
  6. How agents, workflows, and MCP can work together.

  Do not introduce information that cannot be supported by the provided sources. Clearly identify areas where the sources disagree or where evidence is limited. Include source attribution for claims.

- The output I got is:

  Main Findings

  The research across the provided sources indicates a fundamental paradigm shift in AI development: moving from "how to prompt a model" to "how to design a complete system around the model". The core finding is that the architecture of an AI system matters as much as, or more than, the model itself; specifically, older models running within agentic workflows can outperform more advanced models using a simple zero-shot approach. AI agents are defined by their autonomy

and dynamic decision-making, while AI workflows are characterized by predefined, predictable code paths. To solve the problem of non-scalable, custom integrations for these systems, the Model Context Protocol (MCP) provides an open, standardized bridge between AI models and external data/tools.

Important Concepts

1. AI Agents: What They Are and How They Work

An AI agent is a system that understands high-level goals, reasons about complex problems, and uses tools to complete multi-step tasks.

- Architecture: It consists of a reasoning model (the "brain") and an execution harness (the "runtime environment"). The harness manages tool access, context assembly, and memory retrieval.
- The Agentic Loop: Agents operate in a continuous cycle: the model decides the next action, the harness executes it, and the result is fed back to the model to inform the next step.
- Key Patterns: Core behaviors include Reflection (self-correction), Tool Use (API/database interaction), Planning(decomposing goals), and Multi-Agent Collaboration.
- Memory: Agents require multiple memory layers: short-term (current context), working (execution data), and long-term (persistent facts/preferences).

2. AI Workflows: What They Are and How They Work

AI workflows are systems where Large Language Models (LLMs) and tools are orchestrated through predefined code paths with explicit control flow.

- Execution: They follow a structured sequence of operations where the developer hard-codes the logic.
- Common Patterns:
    - Prompt Chaining: Sequential LLM calls where one output feeds the next.
    - Routing: Classifying an input to direct it to a specialized handler or model.
    - Parallelization: Running independent LLM tasks simultaneously to reduce latency.

3. Model Context Protocol (MCP)

MCP is an open standard designed to replace fragmented, custom integrations with a universal interface for AI context exchange.

- Problem Solved: Managing separate integrations for every tool or data source does not scale; MCP standardizes these connections so one server can work with many AI hosts.

Key Differences: Agents vs. Workflows

Aspect	AI Workflows	AI Agents
Control Flow	Predefined and explicit	Dynamic and autonomous
Decision Making	Hard-coded logic	LLM-driven reasoning
Predictability	High; consistent results	Lower; flexible but variable
Adaptability	Fixed paths	Adaptive to changing conditions
Best Use Case	Well-defined tasks (e.g., content pipelines)	Open-ended problems (e.g., deep research)

Technical Workings of MCP

MCP utilizes a client-server architecture to maintain a clear separation between reasoning and tool implementation.

- MCP Host: The main AI application (e.g., Claude Desktop, VS Code) that manages multiple clients.
- MCP Client: A component within the host that maintains a dedicated connection to a specific server.
- MCP Server: A lightweight program that exposes local or remote resources and tools.
- Primitives:
    - Tools: Executable functions (e.g., "get_weather") that the LLM decides if and when to call.
    - Resources: Data sources the LLM can read (e.g., file contents, database records).
    - Prompts: Pre-defined templates for model interactions.
- The Interaction Flow:

1. Handshake: The Client and Server negotiate protocol versions and exchange "Capabilities" (declaring support for tools/resources).
2. Discovery: The Client fetches a list of available tools.
3. Execution: When the Model decides to use a tool, the Host/Client intercepts the call, routes it to the Server, executes the function, and returns the result to the Model.
4. Sampling (Optional): A unique feature allowing a Server to request the model to generate content, creating bidirectional communication.

How Agents, Workflows, and MCP Work Together

These components are increasingly combined in hybrid production systems.

- Structure and Flexibility: Developers often use workflows for the overall structure (e.g., routing a query) and deploy agents for exploratory subtasks.
- Interoperability: MCP serves as the universal connector, allowing both workflows and agents to access a unified registry of tools and data regardless of their implementation language.
- System Integration: In a software engineering agent example, the harness loads instructions (AGENTS.md), selects dynamic Skills (workflows), retrieves Memory, and discovers Tools via MCP to execute a task.

Supporting Evidence

- Performance: Andrew Ng cites benchmarks where agentic workflows allow older models to outperform newer ones.
- Efficiency: LATS (Language Agent Tree Search) shows that tree-based search combined with external feedback increases success rates in coding (92.7% on HumanEval) while maintaining sample complexity.
- Interoperability: Spring AI's boot starters demonstrate that MCP allows Java-based clients to communicate with Python or TypeScript servers seamlessly.

Contradictions, Disagreements, or Nuances

- Terminology Confusion: There is noted developer confusion regarding MCP's "Client-Server" labels, with some suggesting "Consumer-Provider" would be clearer for full-stack developers.
- Predictability vs. Success: While agents are more "capable," simple Random Drop memory policies surprisingly outperformed sophisticated hybrid policies in narrative coherence because they prevented the reinforcement of stale, self-contradictory context.
- "Agentic Workflow" Nuance: Some sources use "Agentic Workflow" as a synonym for an iterative AI system, while others use "Workflow" specifically to describe the *rigid* counterpart to an autonomous agent.

Research and Knowledge Gaps

- Non-Textual Memory: Current implementations (like MaRS) focus heavily on textual memory; extending these cognitive architectures to multi-modal data (images, audio, GUI traces) remains an area for future work.
- Environment Reversion: Many agent frameworks (like LATS) assume the environment can be "reset" to an earlier state, which is not universally applicable to all real-world scenarios.
- Human Alignment: While "human-in-the-loop" is a recognized pattern, systematic studies on how humans can edit an agent's "inner thoughts" on-the-go to correct behavior are still early.

Sources Associated with Major Claims

- AI Agent/Workflow definitions:
- Agentic design patterns (Reflection, Planning, etc.):
- MCP Architecture (Host, Client, Server):
- MCP Primitives (Tools, Resources):
- Agent memory architectures:
- Hybrid system strategies:

- *Draft (Claude):*
  - I provided Claude the synthesis along with the following prompt: Using only the research synthesis provided below, write a first draft explaining AI agents, AI workflows, and Model Context Protocol (MCP). The intended reader is a technically interested student or early-career AI practitioner who understands basic AI concepts but may not understand the distinction between agents, workflows, and MCP. Explain the concepts clearly and technically without unnecessary jargon. Structure the draft logically:

    1. Introduction
    2. What is an AI workflow?
    3. What is an AI agent?
    4. Agents vs workflows
    5. What is MCP?
    6. How MCP works
    7. How agents, workflows, and MCP relate to each other
    8. Practical examples
    9. Limitations and considerations
    10. Conclusion

    Maintain my voice: professional, clear, engaging, technically accurate, confident, open to discussion where appropriate, and polite. Avoid unnecessary buzzwords or exaggerated claims.

    Do not invent facts, sources, examples, or citations. Where the synthesis does not provide enough evidence, flag the statement for verification rather than filling the gap yourself. Here is the research synthesis: (attached).

- The output:

AI Agents, AI Workflows, and MCP: A Practical Explainer

1. Introduction

Over the past couple of years, the conversation around AI development has shifted. Early discussions focused almost entirely on how to prompt a model well. Increasingly, the more important question is how to design the *system* around the model — what it can access, how it makes decisions, and how it connects to the outside world.

This shift matters because system architecture can outweigh raw model capability. There is evidence that older models running inside well-designed agentic workflows can outperform newer, more advanced models used with a simple zero-shot prompt. In other words, how you structure the system around a model can matter as much as, or more than, the model itself.

This piece walks through three concepts that are often used loosely or interchangeably, but which mean distinct things: **AI workflows**, **AI agents**, and the **Model Context Protocol (MCP)**. By the end, the goal is for you to be able to tell them apart, understand how they relate, and recognize which one is the right tool for a given problem.

2. What is an AI workflow?

An AI workflow is a system in which large language models (LLMs) and tools are orchestrated through **predefined code paths**. The developer hard-codes the logic in advance: what happens first, what happens next, and under what conditions.

The defining trait of a workflow is that its control flow is explicit and fixed. The LLM might be doing sophisticated reasoning at each step, but the *sequence* of steps — and the decision of what happens after each one — is determined by code, not by the model.

Common workflow patterns include:

- **Prompt chaining** — a sequence of LLM calls where the output of one call becomes the input to the next.
- **Routing** — classifying an incoming input and directing it to a specialized handler or model based on that classification.
- **Parallelization** — running multiple independent LLM tasks at the same time to reduce overall latency.

Workflows are well suited to tasks that are well-defined and repeatable, such as content pipelines, where the steps involved don't change much from one run to the next.

3. What is an AI agent?

An AI agent is a system that understands a high-level goal, reasons about how to achieve it, and uses tools to complete a multi-step task — without every step being predetermined by the developer.

Structurally, an agent has two main parts:

- A **reasoning model** (the "brain"), which decides what to do next.
- An **execution harness** (the "runtime environment"), which manages tool access, assembles context, and retrieves memory on the model's behalf.

Agents operate through what's often called the **agentic loop**: the model decides on the next action, the harness executes that action, and the result is fed back into the model to inform its next decision. This loop repeats until the task is complete.

Several recurring behaviors, sometimes called agentic design patterns, show up across agent implementations:

- **Reflection** — the agent reviews and self-corrects its own output.
- **Tool use** — the agent calls APIs, queries databases, or otherwise interacts with external systems.
- **Planning** — the agent decomposes a large goal into smaller sub-tasks.
- **Multi-agent collaboration** — multiple agents work together, each handling part of a problem.

Agents also typically need multiple layers of memory:

- **Short-term memory** — the current context window.
- **Working memory** — data generated during execution.
- **Long-term memory** — persistent facts or preferences that carry over between sessions.

Because agents make their own decisions about what to do next, they are well suited to open-ended problems, such as deep research tasks, where the right sequence of steps can't be known in advance.

4. Agents vs. workflows

The core distinction comes down to who — or what — controls the flow of execution.

Aspect	AI Workflows	AI Agents
Control flow	Predefined and explicit	Dynamic and autonomous
Decision making	Hard-coded logic	LLM-driven reasoning
Predictability	High; consistent results	Lower; flexible but variable
Adaptability	Fixed paths	Adaptive to changing conditions
Best use case	Well-defined tasks (e.g., content pipelines)	Open-ended problems (e.g., deep research)

It's worth flagging a terminology nuance here: some sources use "agentic workflow" loosely, as a general synonym for any iterative AI system, while others use "workflow" more precisely to mean the *rigid* counterpart to an autonomous agent. Both usages exist in the wild, so it's worth checking context when you encounter the phrase.

It's also worth noting that more autonomy isn't automatically better. One finding worth sitting with: in at least one study, a simple "random drop" memory policy outperformed more sophisticated hybrid memory policies on narrative coherence, apparently because it prevented the reinforcement of stale, self-contradictory context. This is a useful reminder that agent sophistication and agent performance don't always move together — worth further investigation before generalizing.

5. What is MCP?

As agents and workflows started needing to connect to more external tools and data sources, a practical problem emerged: building a custom integration for every tool and every data source doesn't scale. Each new tool meant new integration code, repeated across every application that wanted to use it.

The **Model Context Protocol (MCP)** is an open standard designed to solve this. Rather than fragmented, one-off integrations, MCP provides a universal interface for exchanging context between AI models and the tools or data they need. A single MCP server can be built once and then used by many different AI applications ("hosts"), rather than being reimplemented for each one.

6. How MCP works

MCP uses a **client-server architecture**, which keeps a clean separation between the reasoning happening in the model and the actual implementation of tools and data access.

The main components are:

- **MCP Host** — the main AI application (for example, Claude Desktop or a code editor like VS Code) that manages one or more clients.
- **MCP Client** — a component inside the host that maintains a dedicated connection to a specific server.
- **MCP Server** — a lightweight program that exposes local or remote resources and tools to the client.

MCP defines three core "primitives":

- **Tools** — executable functions (e.g., get_weather) that the *LLM itself decides* whether and when to call.
- **Resources** — data sources the LLM can read, such as file contents or database records.
- **Prompts** — pre-defined templates for structuring model interactions.

The interaction between these pieces generally follows four stages:

1. **Handshake** — the client and server negotiate protocol versions and exchange "capabilities," declaring what tools or resources each side supports.
2. **Discovery** — the client fetches a list of the tools available on the server.
3. **Execution** — when the model decides to use a tool, the host/client intercepts that decision, routes the call to the appropriate server, executes the function, and returns the result back to the model.
4. **Sampling (optional)** — a less common but notable feature in which a *server* can request the model to generate content, creating a bidirectional flow of communication rather than a strictly one-directional one.

One point worth flagging: there's some documented developer confusion around the "client-server" terminology itself. Some practitioners have suggested "consumer-provider" would be a clearer framing, particularly for developers coming from full-stack backgrounds where "client-server" already carries different connotations. This is a naming critique rather than a technical one, but it's a reasonable thing to be aware of if you find the terminology confusing at first.

7. How agents, workflows, and MCP relate to each other

It helps to think of these three concepts as operating at different layers of a system rather than as competing alternatives:

- **Workflows** and **agents** are both *patterns for structuring how an LLM-based system behaves* — one rigid and predictable, the other dynamic and autonomous.
- **MCP** is a *connectivity layer* — it's how either a workflow or an agent gets standardized access to external tools and data, regardless of what language or platform those tools were built in.

In practice, production systems increasingly combine all three. A common pattern is to use a **workflow for overall structure** (for example, routing an incoming query to the right place) while **deploying an agent for the exploratory subtasks** within that structure that don't have a predictable, fixed path.

MCP then serves as the shared connector underneath both: rather than each workflow or agent needing its own custom tool integrations, they can draw from a unified registry of tools and data sources exposed via MCP servers, no matter how those servers happen to be implemented.

One illustrative example from the software engineering domain: an agent harness might load a set of standing instructions (referred to in some frameworks as an AGENTS.md file), select from a set of predefined "skills" (which are themselves essentially small workflows), retrieve relevant long-term memory, and then discover and call tools via MCP — all as part of executing a single task. This shows the three concepts working together in one integrated system rather than as separate, isolated choices.

8. Practical examples

A few concrete data points from the research, offered with the caveat that some of these would benefit from independent verification before being cited further:

- **Performance:** Andrew Ng has cited benchmark comparisons where agentic workflows allow older models to outperform newer ones used without that structure. *(The specific benchmark and figures should be verified before quoting this claim precisely.)*
- **Efficiency:** LATS (Language Agent Tree Search), which combines tree-based search with external feedback, is reported to reach a 92.7% success rate on HumanEval (a coding benchmark) while keeping sample complexity manageable. *(Worth checking the original paper for the exact experimental conditions.)*
- **Interoperability:** Spring AI's boot starters are cited as a demonstration that MCP allows a Java-based client to communicate with servers written in Python or TypeScript — illustrating MCP's core promise of standardizing connections across implementation languages.

9. Limitations and considerations

A few open questions and caveats are worth keeping in mind, both because they're intellectually interesting and because they affect how far you should trust any given implementation:

- **Non-textual memory:** Current agent memory architectures (such as MaRS, referenced in the research) focus heavily on textual memory. Extending these architectures to multi-modal data — images, audio, GUI interaction traces — is still an open area of work.
- **Environment reversion:** Frameworks like LATS often assume that the environment an agent operates in can be reset to an earlier state if something goes wrong. This assumption doesn't hold for many real-world scenarios, where actions (sending an email, modifying a database) can't simply be undone.
- **Human alignment:** "Human-in-the-loop" is a recognized and useful pattern for supervising agents, but systematic study of how humans can directly edit an agent's intermediate reasoning ("inner thoughts") in real time, to correct its behavior mid-task, is still an early and underexplored area.
- **Predictability trade-off:** As noted in Section 4, agents trade predictability for flexibility. That's a reasonable trade for open-ended problems, but it's worth being honest that "more autonomous" doesn't mean "more reliable" — the memory-policy finding mentioned earlier is a good illustration of that.

These are gaps flagged in the research rather than settled conclusions, and I'd treat them as open questions rather than established limitations with a clear resolution.

10. Conclusion

Workflows and agents represent two different philosophies for structuring LLM-based systems: one favors predictability through fixed, developer-defined logic; the other favors flexibility through model-driven, dynamic decision-making. Neither is strictly better — the right choice depends on whether the task at hand is well-defined or

open-ended, and in practice, many real systems use both, layering agents into workflows where autonomy is needed and workflows where predictability is needed.

MCP sits underneath both, solving a different problem entirely: how systems connect to the tools and data they need without every integration being built from scratch. As a standardized, open protocol, it lets a single tool implementation serve many different AI applications, in whatever language they happen to be built.

Taken together, these three concepts represent less a single technology stack and more a shared vocabulary for talking about how modern AI systems are architected. Understanding the distinctions — rigid vs. dynamic control flow, and structure vs. connectivity — should make it easier to reason about why a given system is built the way it is, and to make more deliberate choices in your own designs.

A note on sourcing: this draft relies solely on the provided research synthesis. Several specific claims (the Andrew Ng benchmark, the LATS HumanEval figure, and the Spring AI interoperability example) are attributed to named sources in the synthesis but lack full citations here — these should be verified against primary sources before this draft is finalized or published.

- *Critique (Claude):*
  - I gave Claude this prompt: Critique this draft as a technical reviewer. Check specifically for:

    - factual inaccuracies
    - unsupported claims
    - incorrect distinctions between agents and workflows
    - incorrect explanations of MCP
    - oversimplification of MCP's architecture
    - missing important concepts
    - claims that need source verification
    - repetition
    - generic or AI-generated wording
    - places where the explanation is technically correct but unclear

      Separate your critique into:

      1. What's correct
      2. What needs verification
      3. What's missing
      4. What should be removed or simplified
      5. Technical accuracy issues
      6. Generic/AI-generated wording
      7. Specific recommended changes

    Do not rewrite the article yet. I want the critique first.

- The output I got is:

Technical Review: "AI Agents, AI Workflows, and MCP"

1. What's Correct

- The core workflow/agent distinction (predefined control flow vs. model-driven control flow) is accurately represented and matches the synthesis.
- The workflow patterns (prompt chaining, routing, parallelization) are correctly described and match how these terms are commonly used in the field.
- The agentic loop description (decide → execute → feed back) is accurate at a high level.
- The four design patterns (Reflection, Tool Use, Planning, Multi-Agent Collaboration) are correctly named and briefly but accurately described.
- The three-layer memory model (short-term, working, long-term) is presented as given in the synthesis without embellishment.
- MCP's stated purpose — replacing fragmented one-off integrations with a standardized interface — is accurate and is the correct "why" for MCP's existence.
- Host/Client/Server roles are correctly assigned, and the handshake → discovery → execution flow is a reasonable simplification of the real MCP lifecycle.
- Sampling is correctly identified as the bidirectional/unusual primitive.

2. What Needs Verification

- **Intro claim vs. Section 8 claim — internal inconsistency.** The introduction states as settled fact that "older models running within agentic workflows can outperform more advanced models using a simple zero-shot approach." Section 8 then attributes essentially the same claim to Andrew Ng and flags it for verification. The draft can't have it both ways — either this is an established finding (and should be sourced) or it's a single attributed claim (and shouldn't be asserted unqualified in the intro). This is the most important verification issue in the piece.
- **92.7% HumanEval figure (LATS).** Specific benchmark numbers are exactly the kind of claim that gets misquoted when copied secondhand. Needs a primary-source check before publishing, not just a footnote caveat.
- **"MaRS" reference** (Section 9) — this acronym is used with no expansion or context. Even within a "flag for verification" framing, the draft should note that the term itself is unclear from the source material, not just that the claim needs checking.
- **Random Drop memory policy finding.** This is presented as an interesting nugget but it's a single-study result being used to make a fairly strong generalizable-sounding point ("more sophisticated ≠ better"). Worth explicitly flagging as *one finding, not a general principle*, since the current phrasing risks being read as a broader conclusion.
- **Spring AI example.** Fine as an illustration, but as written it reads as a settled proof point ("demonstrates") rather than one example. Soften the verb.
- **AGENTS.md reference.** Treated as an established convention. Worth checking whether the synthesis intends this as a specific named project/spec

or a generic illustrative label — the draft currently presents it with more confidence than the source material seems to warrant.

3. What's Missing

- **Transport mechanisms.** MCP's Host/Client/Server model is described entirely in the abstract, with no mention of how a client and server actually communicate (e.g., local process vs. network transport). Without this, a technically-minded reader can't picture how "local or remote resources" actually gets implemented.
- **Who owns/originated MCP.** The draft never states that MCP is Anthropic's protocol. This is a basic factual anchor a technical reader would expect and its absence is conspicuous, especially given the piece will presumably be read by people who already use Claude-related tools.
- **Security/trust considerations.** This is the biggest substantive gap. Any serious discussion of MCP's limitations should mention the trust boundary problem: a server can expose malicious or misleading tool descriptions, and a host is generally trusting the server's declared capabilities. Similarly, tool-use in agents raises prompt-injection risk when tool outputs feed back into the model. The "Limitations" section covers memory, environment reversion, and human alignment, but omits this — arguably the most commonly discussed limitation of both agents and MCP in current practice.
- **Sampling's approval mechanism.** The draft describes sampling as a server asking the model to generate content, but omits that this typically routes through the client/host, which can gate or approve the request. Without this, sampling sounds like it lets a server talk to the model directly and autonomously, which overstates server autonomy and understates the human-in-the-loop design intent.
- **Termination conditions for the agentic loop.** The loop is described as running "until the task is complete," but how an agent recognizes completion (or hits a step/budget limit) is never addressed. For a reader trying to understand how agents actually behave in production, this is a meaningful gap.
- **Cost/latency tradeoffs.** A practical, well-known consideration — agents typically require more LLM calls and more wall-clock time than a workflow doing the same job — is absent from the "Limitations" section, despite being one of the most concrete, practitioner-relevant tradeoffs.
- **The fact that "tool calling" predates and is separate from MCP.** The draft's phrasing could lead a reader to think MCP is what enables an LLM to decide to call a tool. In reality, function/tool-calling is a model capability; MCP standardizes how the *tool itself* is discovered and exposed, not the model's decision mechanism. This distinction is currently blurred.
- **Agents/workflows as a spectrum, not a binary.** Real systems often blend the two (e.g., an agent loop with a hard-coded step limit, or a workflow with one LLM-routed branch point). The draft presents the distinction as a clean dichotomy, which is a reasonable simplification for a first pass but should probably be acknowledged explicitly.

4. What Should Be Removed or Simplified

- **Repetition of the "predefined vs. dynamic" framing.** This point is made in Section 2, restated in the Section 4 table, and then re-explained in prose immediately below the table. One clean statement plus the table should be sufficient.
- **The Random Drop memory finding is mentioned twice** (Section 4 and Section 9) with overlapping framing. Pick one location — Section 9 (Limitations) is the more natural home — and drop it from Section 4.
- **The client-server/consumer-provider terminology aside** (Section 6) is a minor naming complaint that doesn't add much technical value and could be trimmed to a single sentence rather than a full paragraph.
- **Hedging language is repeated at a rate that undercuts confidence.** "It's worth flagging," "it's worth noting," "it's worth sitting with," "it's worth being honest that" — four variations of the same construction across a fairly short piece. Consolidate the epistemic caveats rather than distributing them evenly through nearly every section.

5. Technical Accuracy Issues

- **Predictability is conflated with determinism.** The workflow/agent table lists "Predictability: High" for workflows, but a workflow's *control flow* is fixed — the LLM outputs within each step are still non-deterministic (temperature, sampling variance). The draft doesn't distinguish "predictable structure" from "predictable output," which could mislead a reader into thinking workflows guarantee consistent results end-to-end.
- **Sampling described as letting a server "request the model to generate content"** without noting the client/host mediates and typically requires approval. As written, this slightly overstates server autonomy in the MCP architecture — which cuts against MCP's actual design intent of keeping the host/client in control.
- **"Prompts" primitive is underexplained.** It's listed as "pre-defined templates for model interactions" with no indication of how they differ functionally from Tools or Resources (e.g., that they're typically surfaced to the *user* to invoke, not something the model calls autonomously). As written, a reader could reasonably wonder why this is a separate primitive at all.
- **"One MCP server can work with many AI hosts"** is true but implies more automatic interoperability than exists — a host still needs to implement an MCP client to talk to any server. Worth a qualifying clause.
- **The word "harness"** is used from Section 3 onward without being anchored to a concrete example. It's not wrong, but for the target reader (technically interested but not necessarily familiar with agent infrastructure jargon), this term is introduced and reused without ever being made concrete.

6. Generic/AI-Generated Wording

- Repeated hedge-phrase template: "It's worth flagging...", "It's also worth noting...", "worth sitting with", "worth being honest that" — this is a recognizable AI-writing tic and appears often enough to read as formulaic rather than considered.
- The closing line — *"these three concepts represent less a single technology stack and more a shared vocabulary..."*— is a classic "less X, more Y" AI-

generated cadence. It's not factually wrong, but it reads as a stock rhetorical flourish rather than a specific, earned conclusion.
- "Taken together..." as a paragraph opener in the conclusion is a generic transition with no real content.
- Several transitions ("This shift matters because...", "This shows the three concepts working together...") restate what was just said rather than adding new information — mild filler.
- The bullet-heavy structure throughout risks reading like a slide deck rather than the "engaging, professional" prose voice requested. Several sections (3, 6, 7) could carry more of their content in connected prose with bullets reserved for genuinely list-like content (e.g., the primitives, the four-stage flow).

7. Specific Recommended Changes

1. Resolve the intro/Section 8 inconsistency on the "older models + workflows outperform newer zero-shot models" claim — either cite it properly in the intro or soften the intro's phrasing to match the hedged treatment in Section 8.
2. Add one sentence identifying MCP's origin (Anthropic).
3. Add a short explanation of transport (local process vs. network) to Section 6.
4. Add sampling's approval/mediation step to the sampling description.
5. Clarify that tool-calling as a model capability predates MCP; MCP standardizes discovery/exposure, not the decision mechanism.
6. Move the Random Drop memory finding to Section 9 only, and reframe it explicitly as a single-study result rather than a general principle.
7. Add a security/trust-boundary point to Limitations (malicious tool descriptions, prompt injection via tool outputs).
8. Add a cost/latency tradeoff point to Limitations.
9. Trim the client-server/consumer-provider aside to one sentence.
10. Add a line acknowledging agents/workflows as a spectrum rather than a strict binary.
11. Replace at least two of the four "worth noting/flagging/sitting with" constructions with direct statements.
12. Clarify the "Predictability" row in the table (or accompanying text) to distinguish control-flow predictability from output determinism.
13. Rewrite the closing paragraph to avoid the "less X, more Y" construction and end on a more concrete, specific note rather than a rhetorical flourish.

- *Revise (Claude):*
  - The prompt provided was: Using the critique above, revise the draft. Keep the information grounded in the original research synthesis. Do not introduce unsupported facts. Preserve my writing style: professional, clear, engaging, technically accurate, confident, open to discussion where appropriate, and polite. Make the distinction between agents, workflows, and MCP especially clear. Show me the revised version and briefly list the major changes you made.
  - The output given is:

AI Agents, AI Workflows, and MCP: A Practical Explainer

1. Introduction

Over the past couple of years, the conversation around AI development has shifted. Early discussions focused almost entirely on how to prompt a model well. Increasingly, the more important question is how to design the *system* around the model — what it can access, how it makes decisions, and how it connects to the outside world.

That shift is motivated by a specific claim worth stating carefully rather than as settled fact: Andrew Ng has cited benchmark comparisons where older models, run inside well-designed agentic workflows, outperform newer models used with a simple zero-shot prompt (see Section 8 — the specific benchmark should be checked before this is quoted further). If that holds up, it's a useful reminder that system architecture can matter as much as raw model capability. I'm treating it here as a strong, cited claim rather than an established law, since I haven't verified the underlying benchmark myself.

This piece walks through three concepts that are often used loosely or interchangeably, but which mean distinct things: **AI workflows**, **AI agents**, and the **Model Context Protocol (MCP)**. By the end, the goal is for you to be able to tell them apart, understand how they relate, and recognize which one is the right tool for a given problem.

2. What is an AI workflow?

An AI workflow is a system in which large language models (LLMs) and tools are orchestrated through **predefined code paths**. The developer hard-codes the logic in advance: what happens first, what happens next, and under what conditions.

The defining trait of a workflow is that its control flow is explicit and fixed. The LLM might be doing sophisticated reasoning at each step, but the *sequence* of steps — and the decision of what happens after each one — is determined by code, not by the model.

Common workflow patterns include:

- **Prompt chaining** — a sequence of LLM calls where the output of one call becomes the input to the next.
- **Routing** — classifying an incoming input and directing it to a specialized handler or model based on that classification.
- **Parallelization** — running multiple independent LLM tasks at the same time to reduce overall latency.

Workflows are well suited to tasks that are well-defined and repeatable, such as content pipelines, where the steps involved don't change much from one run to the next.

3. What is an AI agent?

An AI agent is a system that understands a high-level goal, reasons about how to achieve it, and uses tools to complete a multi-step task — without every step being predetermined by the developer.

Structurally, an agent has two main parts:

- A **reasoning model** (the "brain"), which decides what to do next.
- An **execution harness** (the "runtime environment"), which manages tool access, assembles context, and retrieves memory on the model's behalf. Later on, when we get to MCP, this is the same harness that discovers and calls tools through it — so it's worth keeping this piece in mind as the connective tissue between the two topics.

Agents operate through what's often called the **agentic loop**: the model decides on the next action, the harness executes that action, and the result is fed back into the model to inform its next decision. This loop repeats until the task is judged complete — though the synthesis doesn't specify how that completion condition is typically determined or bounded (e.g., a step limit), which is worth keeping in mind as an open question rather than a settled mechanism.

Several recurring behaviors, sometimes called agentic design patterns, show up across agent implementations:

- **Reflection** — the agent reviews and self-corrects its own output.
- **Tool use** — the agent calls APIs, queries databases, or otherwise interacts with external systems.
- **Planning** — the agent decomposes a large goal into smaller sub-tasks.
- **Multi-agent collaboration** — multiple agents work together, each handling part of a problem.

Agents also typically need multiple layers of memory:

- **Short-term memory** — the current context window.
- **Working memory** — data generated during execution.
- **Long-term memory** — persistent facts or preferences that carry over between sessions.

Because agents make their own decisions about what to do next, they are well suited to open-ended problems, such as deep research tasks, where the right sequence of steps can't be known in advance.

4. Agents vs. workflows

The core distinction comes down to who — or what — controls the flow of execution.

Aspect	AI Workflows	AI Agents
Control flow	Predefined and explicit	Dynamic and autonomous
Decision making	Hard-coded logic	LLM-driven reasoning

Aspect	AI Workflows	AI Agents
Predictability of structure	High; the sequence of steps is fixed	Lower; the sequence itself can vary
Adaptability	Fixed paths	Adaptive to changing conditions
Best use case	Well-defined tasks (e.g., content pipelines)	Open-ended problems (e.g., deep research)

One clarification worth making explicit: "predictability" here refers to the *structure* of execution — which steps run, and in what order — not to the output of any individual LLM call within that structure. A workflow guarantees you'll hit the same steps in the same sequence every time; it doesn't guarantee the model's response at each step will be identical, since that still depends on the model's own generation behavior. Predictable control flow and deterministic output are two different things, and it's easy to conflate them.

A terminology note: some sources use "agentic workflow" loosely, as a general synonym for any iterative AI system, while others use "workflow" more precisely to mean the *rigid* counterpart to an autonomous agent. Both usages exist, so it's worth checking context when you encounter the phrase.

Finally, this table presents a clean dichotomy, but real systems rarely sit purely at one end. As Section 7 covers, production systems commonly blend the two — using a workflow for overall structure and an agent for the open-ended parts within it. Treat the table as describing two poles on a spectrum, not two mutually exclusive categories.

5. What is MCP?

As agents and workflows started needing to connect to more external tools and data sources, a practical problem emerged: building a custom integration for every tool and every data source doesn't scale. Each new tool meant new integration code, repeated across every application that wanted to use it.

The **Model Context Protocol (MCP)** is an open standard designed to solve this. Rather than fragmented, one-off integrations, MCP provides a universal interface for exchanging context between AI models and the tools or data they need. A single MCP server can be built once and then used by many different AI applications ("hosts"), rather than being reimplemented for each one.

6. How MCP works

MCP uses a **client-server architecture**, which keeps a clean separation between the reasoning happening in the model and the actual implementation of tools and data access.

The main components are:

- **MCP Host** — the main AI application (for example, Claude Desktop or a code editor like VS Code) that manages one or more clients.

- **MCP Client** — a component inside the host that maintains a dedicated connection to a specific server. This is worth noting explicitly: a host needs an MCP client implementation to talk to *any* server, so "one server can work with many hosts" holds only for hosts that have implemented the client side of the protocol — it isn't automatic just because a server exists.
- **MCP Server** — a lightweight program that exposes local or remote resources and tools to the client.

An important distinction that's easy to blur: the *decision* to call a tool is made by the model itself, as part of its normal reasoning — that's a model capability, not something MCP introduces. What MCP standardizes is the other half of the problem: how a tool is described, discovered, and exposed to the client in the first place, so that the model has something consistent to decide *about*. MCP doesn't teach a model to use tools; it standardizes how tools get in front of the model.

MCP defines three core "primitives":

- **Tools** — executable functions (e.g., get_weather) that the LLM decides whether and when to call.
- **Resources** — data sources the LLM can read, such as file contents or database records.
- **Prompts** — pre-defined templates for structuring model interactions. The synthesis doesn't detail how these operate differently from tools or resources in practice, so treat this primitive as noted but not fully explained here — worth checking primary MCP documentation if you need the operational distinction.

The interaction between these pieces generally follows four stages:

1. **Handshake** — the client and server negotiate protocol versions and exchange "capabilities," declaring what tools or resources each side supports.
2. **Discovery** — the client fetches a list of the tools available on the server.
3. **Execution** — when the model decides to use a tool, the host/client intercepts that decision, routes the call to the appropriate server, executes the function, and returns the result back to the model.
4. **Sampling (optional)** — a less common but notable feature in which a *server* can request the model to generate content, creating a bidirectional flow of communication rather than a strictly one-directional one. Given that the host/client mediates tool execution in stage 3, it's reasonable to expect similar mediation applies here too — but the synthesis doesn't spell out whether or how a host approves or gates a server's sampling request, so I'd flag that as a detail to confirm rather than assume.

On terminology: there's some documented developer confusion around the "client-server" labels themselves, with some practitioners suggesting "consumer-provider" as a clearer alternative for those coming from full-stack backgrounds. It's a naming critique rather than a technical one, but worth knowing if the terms feel unintuitive at first.

7. How agents, workflows, and MCP relate to each other

It helps to think of these three concepts as operating at different layers of a system rather than as competing alternatives:

- **Workflows** and **agents** are both *patterns for structuring how an LLM-based system behaves* — one rigid and predictable, the other dynamic and autonomous.
- **MCP** is a *connectivity layer* — it's how either a workflow or an agent gets standardized access to external tools and data, regardless of what language or platform those tools were built in.

In practice, production systems increasingly combine all three. A common pattern is to use a **workflow for overall structure** (for example, routing an incoming query to the right place) while **deploying an agent for the exploratory subtasks** within that structure that don't have a predictable, fixed path.

MCP then serves as the shared connector underneath both: rather than each workflow or agent needing its own custom tool integrations, they can draw from a unified registry of tools and data sources exposed via MCP servers, no matter how those servers happen to be implemented.

One illustrative example from the software engineering domain: an agent harness might load a set of standing instructions (referred to in some frameworks as an AGENTS.md file), select from a set of predefined "skills" (which are themselves essentially small workflows), retrieve relevant long-term memory, and then discover and call tools via MCP — all as part of executing a single task. This shows the three concepts working together in one integrated system rather than as separate, isolated choices.

8. Practical examples

A few concrete data points from the research, offered with the caveat that some of these would benefit from independent verification before being cited further:

- **Performance:** Andrew Ng has cited benchmark comparisons where agentic workflows allow older models to outperform newer ones used without that structure. *(The specific benchmark and figures should be verified before quoting this claim precisely.)*
- **Efficiency:** LATS (Language Agent Tree Search), which combines tree-based search with external feedback, is reported to reach a 92.7% success rate on HumanEval (a coding benchmark) while keeping sample complexity manageable. *(Worth checking the original paper for the exact experimental conditions.)*
- **Interoperability:** Spring AI's boot starters are cited as a demonstration that MCP allows a Java-based client to communicate with servers written in Python or TypeScript — illustrating MCP's core promise of standardizing connections across implementation languages.

9. Limitations and considerations

A few open questions and caveats are worth keeping in mind, both because they're intellectually interesting and because they affect how far you should trust any given implementation:

- **More sophisticated isn't always better.** In at least one study referenced in the research, a simple "random drop" memory policy outperformed more sophisticated hybrid memory policies on narrative coherence — apparently because it avoided reinforcing stale, self-contradictory context. This is a single result, not a general rule, but it's a useful check against assuming that a more elaborate memory system automatically produces better agent behavior.
- **Non-textual memory:** Current agent memory architectures (such as MaRS, referenced in the research, though the synthesis doesn't expand on what it stands for) focus heavily on textual memory. Extending these architectures to multi-modal data — images, audio, GUI interaction traces — is still an open area of work.
- **Environment reversion:** Frameworks like LATS often assume that the environment an agent operates in can be reset to an earlier state if something goes wrong. This assumption doesn't hold for many real-world scenarios, where actions (sending an email, modifying a database) can't simply be undone.
- **Human alignment:** "Human-in-the-loop" is a recognized and useful pattern for supervising agents, but systematic study of how humans can directly edit an agent's intermediate reasoning ("inner thoughts") in real time, to correct its behavior mid-task, is still an early and underexplored area.
- **Open questions the synthesis doesn't cover.** Two things a technical reader will likely want to know that fall outside the scope of the source material here: how much more expensive or slower an agent typically is compared to an equivalent workflow (the LATS efficiency result touches on sample complexity in tree search specifically, but doesn't generalize this to agents broadly), and what trust boundary exists between a host and a server it connects to — for instance, whether a server's declared capabilities are otherwise verified. Both are reasonable questions to raise, but I don't have grounding in the research to answer them here, so I'm flagging them rather than guessing.

These are gaps flagged in the research rather than settled conclusions, and I'd treat them as open questions rather than established limitations with a clear resolution.

10. Conclusion

To bring the three ideas back together with the distinction stated plainly: **workflows** and **agents** are two different answers to the question "who controls the sequence of steps?" — a developer, or the model itself. **MCP** answers a separate question entirely — "how does a tool or data source get connected to a model, regardless of which of the two answers above applies?" A workflow can use MCP to reach its tools. So can an agent. MCP doesn't care which one is asking.

Neither workflows nor agents are strictly better — the right choice depends on whether the task at hand is well-defined or open-ended, and, as Section 7 covered, many production systems use both at once: a workflow for overall structure, with an agent handling the exploratory parts inside it. MCP's job, in either case, is the same:

standardizing how that structure or that agent reaches the tools and data it needs, without every integration being built from scratch.

If you're deciding how to structure your own system, the practical question to ask first isn't "should I build an agent," but "does this task have a fixed, known sequence of steps, or not?" That answer points you toward a workflow or an agent respectively — and MCP is very likely relevant either way, as soon as you need to connect to more than one external tool or data source.

A note on sourcing: this draft relies solely on the provided research synthesis. A few things should be verified against primary sources before this draft is finalized or published: the Andrew Ng benchmark claim (Sections 1 and 8), the LATS HumanEval figure, the Spring AI interoperability example, what "MaRS" refers to, and whether MCP's sampling requests are host/client-mediated in the way this draft infers. None of these were independently confirmed here — they're carried through from the synthesis as-is or flagged where the synthesis was silent.

- *Human Review:*
  - Review the information, citations and facts.
- *Failure Points:*
  - The sources cited are incomplete.
  - MCP explanation did not contain the necessary technical details.
  - Some information was oversimplified and every critique point was not resolved.
  - The wording is too generic at some points.
- Time Comparison:
  - Manual time: 2 hours
  - AI: 45 minutes.
  - Improvement: 62.5% reduction in time.

Run 4:

- *Topic:* Steganography research paper
- *Gather (NotebookLM):*
    - The prompt provided is: The topic is steganography. Gather credible and academically reliable sources that can support a research paper on steganography. Focus on the fundamental concept and purpose of steganography, its history and development, different types of steganography, common techniques and algorithms, image/audio/video/text steganography, spatial-domain and transform-domain methods, steganalysis and detection, security and robustness, capacity and imperceptibility, common evaluation metrics, applications, limitations, current challenges, and recent developments. Prioritize peer-reviewed research papers, academic publications, books, conference papers, university resources, and authoritative technical sources. Where possible, include foundational papers as well as recent research. The sources should help answer:

        1. What is steganography and how does it work?
        2. How is steganography different from cryptography?
        3. What are the major types of steganography?
        4. What are the main techniques used for hiding information?
        5. How is hidden information detected?
        6. What makes a steganographic method effective?
        7. What are the main applications and limitations?
        8. What are the current research challenges and potential future directions?

        Do not write the research paper yet. Only gather credible sources and information that can be used later for synthesis. Prioritize source quality and relevance over quantity.

    - The output given is:

        That sounds like a fascinating and rigorous research project! Steganography is such a deep field, spanning from ancient history to cutting-edge AI detection methods.

        I've just initiated a Deep Research search to gather the high-quality academic sources you need, including foundational papers and recent developments in areas like transform-domain methods and steganalysis.

        Because this is a comprehensive search for peer-reviewed and authoritative technical material, it will take several minutes to complete. Once finished, you'll find a list of suggested sources in the Source Panel on the left.

While we wait for those results to populate, are there any specific historical periods or modern algorithms (like LSB or DCT) you're most interested in focusing on for your paper?

- *Synthesise (NotebookLM):*
  - The prompt given is: I need a research synthesis based only on the sources gathered for this project. Do not introduce information that cannot be supported by the provided sources. Topic: Steganography. Organize the synthesis into:

    1. Main findings
    2. Important concepts
    3. Supporting evidence
    4. Contradictions, disagreements, or nuances between sources
    5. Research/knowledge gaps
    6. Sources associated with each major claim

    Specifically explain:

    1. What steganography is and what problem it is designed to solve.
    2. The history and development of steganography, including important developments supported by the sources.
    3. How steganography differs from cryptography and how the two can be used together.
    4. The major types of steganography, including image, audio, video, and text-based approaches.
    5. The main technical approaches used for hiding information, including spatial-domain and transform-domain methods where supported by the sources.
    6. How techniques such as LSB and DCT-based steganography work, if supported by the gathered sources.
    7. The key properties used to evaluate steganographic techniques, including embedding capacity, imperceptibility, robustness, and security, and how these properties can involve trade-offs.
    8. What steganalysis is, how hidden information can be detected, and the main approaches used for detection.
    9. The role of machine learning and deep learning in modern steganography and steganalysis, where supported by the sources.
    10. Practical applications of steganography.
    11. Major limitations, security concerns, and challenges.
    12. Current research directions and knowledge gaps.

    For each major technical claim, identify the specific source(s) that support it. Distinguish clearly between established findings, individual research results, and claims that require further verification.

    Pay particular attention to disagreements between sources and differences in the assumptions, datasets, methods, or evaluation metrics used in studies. Do not treat the results of one study as a general conclusion unless the sources support that generalisation.

Do not write the research paper yet. This is only the research synthesis stage. Keep the synthesis detailed enough to support a later academic draft but remain faithful to the gathered sources.

- The output is:

This research synthesis on steganography is based on the provided technical sources, covering fundamental principles, historical evolution, technical methodologies, and contemporary frontiers in deep learning and generative models.

Main Findings

- **Steganography serves as a vital complement to cryptography.** While cryptography secures a message's content by producing unreadable ciphertext, steganography secures the communication channel by **concealing the very existence of the message**.
- **The "Steganographic Triangle" represents a fundamental trade-off.** Implementations are constrained by the competing goals of **embedding capacity** (payload volume), **imperceptibility** (statistical/visual indistinguishability), and **robustness** (survival under distortion).
- **Generative models have revolutionized the field.** Modern steganography is shifting from modification-based approaches (altering existing covers) to **generation-based approaches** (synthesizing new, natural-looking media from secret bitstreams), which complicates detection by evading traditional pixel-level artifacts.
- **Deep Learning (DL) has unified steganographic detection.** Contemporary steganalysis has moved from handcrafted statistical models to end-to-end **neural network architectures** (e.g., CNNs, RNNs, and GANs) that can automatically extract subtle embedding traces.

Important Concepts

- **The Prisoners' Problem:** A classic formalization where two parties (Alice and Bob) must coordinate an escape through messages monitored by a warden (Eve); any detected anomaly leads to communication termination.
- **Cachin's Security Model:** A theoretical framework where steganographic security is measured by the **Kullback-Leibler (KL) divergence** between the distribution of cover objects and stego-objects. **Perfect security** is achieved when the divergence is zero
- **Coverless Steganography:** A paradigm where information is hidden without any modification to the carrier by establishing mapping relationships between secret data and existing semantic features (Deep Semantic Mapping).
- **Distortion Minimization Framework:** An optimization approach where embedding changes are guided by a localized **cost map** to cluster modifications in textured or noisy regions that are difficult for steganalysts to model.

Supporting Evidence

Major Types of Steganography

- **Image Steganography:** The most popular form due to high digital redundancy. It utilizes **pixel intensities** to hide information.
- **Audio Steganography:** Exploits the limitations of the **Human Auditory System (HAS)**. Techniques include **phase coding**, which hides data in inaudible phase shifts, and **echo data hiding**, which embeds bits through small, controlled delays.
- **Video Steganography:** Offers immense capacity and masks data using **temporal dependencies**. Key domains include modifying **motion vectors (MVs)** or intra-frame prediction modes.
- **Text (Linguistic) Steganography:** Traditionally involved format changes (spaces, tabs). Modern **generative linguistic steganography** uses Large Language Models (LLMs) to synthesize fluent text directly from message bits.

Technical Approaches for Hiding Information

- **Spatial-Domain Methods:** These involve direct modification of the carrier's raw values. **Least Significant Bit (LSB) substitution** is the most common, replacing the lowest bits of cover elements with message bits. While simple and providing high capacity, these methods are highly vulnerable to statistical analysis.
- **Transform-Domain Methods:** Information is embedded in **frequency coefficients** (e.g., DCT, DWT, or FFT).
    - **Discrete Cosine Transform (DCT):** Used in JPEG steganography, it breaks images into 8x8 blocks and hides data in quantized AC coefficients. It is more robust than spatial methods against compression.
    - **Discrete Wavelet Transform (DWT):** Provides a multi-resolution description, matching human visual perception more closely and offering high robustness.

Steganalysis and Detection

- **Targeted vs. Universal:** Targeted steganalysis exploits specific vulnerabilities of known algorithms, whereas **universal (blind) steganalysis** uses machine learning to detect anomalies independently of the embedding method.
- **Rich Models:** Statistical detection tools like the **Spatial Rich Model (SRM)** use a union of diverse high-pass filters to capture noise residuals across neighbouring pixel dependencies.
- **Deep Learning Steganalysis:** CNNs have largely replaced handcrafted features. Key architectures like **SRNet** use deep residual networks to learn noise residuals without requiring handcrafted pre-filters.

Contradictions, Disagreements, or Nuances

- **Security vs. Secrecy:** Some sources distinguish between **security** (preventing detection) and **secrecy** (preventing extraction/decoding even if the channel is detected).
- **The Generative Steganography Trilemma:** A disagreement exists in how to prioritize the three goals for generative models: **Optimal Capacity**, **Optimal**

Efficiency (O(1) complexity), and **Optimal Security**. While methods like **Arithmetic Coding (AC)** offer optimal capacity, they often compromise on efficiency and statistical security.
- **Robustness Priorities:** Sources differentiate between **steganography** (prioritizing secrecy) and **watermarking** (prioritizing robustness against active distortion/attacks).

Research and Knowledge Gaps

- **Cover Source Mismatch (CSM):** A major challenge where detection accuracy degrades significantly when real-world data distributions differ from the training datasets.
- **Real-Time Detection:** Detecting hidden information in high-bandwidth, low-latency communication like **VoIP streams** or live video remains difficult.
- **Standardization:** There is a lack of unified taxonomy and **standardized evaluation frameworks** for comparing diverse deep semantic mapping and generative approaches.
- **Active Adversaries:** Most information-theoretic models assume a **passive warden** (observer); resilience against active attackers who intentionally perturb traffic is less explored in high-capacity contexts.

Sources Associated with Major Claims

- **Fundamental Definitions:**.
- **History and Ancient Techniques:**.
- **Simmons' Prisoners' Problem:**.
- **KL Divergence and Cachin's Model:**.
- **LSB and Spatial Techniques:**.
- **DCT and Transform Domain:**.
- **Audio Hiding Paradigms:**.
- **Video Hiding and HEVC:**.
- **Syndrome-Trellis Codes (STC):**.
- **UNIWARD Distortion Function:**.
- **SRM and Rich Models:**.
- **Deep Learning for Steganalysis:**.
- **Generative Linguistic Steganography:**.
- **Coverless DSM and GANs:**.
- **Minimum Entropy Coupling (iMEC):**.

- *Draft (Claude):*
    - The prompt given is: Using only the research synthesis provided below, write a first draft of an academic research paper on Steganography. The intended reader is a technically interested student or early-career researcher who understands basic computing and AI concepts but may not have prior knowledge of steganography. Explain the subject clearly and technically without unnecessary jargon. Structure the draft logically:

        1. Introduction
        2. What is Steganography?

3. Steganography and Cryptography
4. History and Fundamental Concepts
5. Types of Steganography
6. Technical Methods
    - Spatial-domain techniques
    - LSB substitution
    - Transform-domain techniques
    - DCT and DWT
7. Steganalysis and Detection
    - Targeted vs. universal steganalysis
    - Rich models
    - Deep-learning-based detection
8. Security, Capacity, Imperceptibility and Robustness
9. Modern Developments
    - Deep learning
    - Generative steganography
    - Generative linguistic steganography
    - Coverless steganography
10. Applications
11. Limitations and Research Challenges
12. Future Research Directions
13. Conclusion

Explain important concepts such as the Prisoners' Problem, Cachin's security model, the steganographic triangle, distortion minimization, and coverless steganography where they are supported by the synthesis. When discussing technical methods, explain how they work, not just what they are. For example, explain what happens when information is embedded using LSB and why transform-domain approaches such as DCT can provide different robustness characteristics. Clearly distinguish steganography from digital watermarking and cryptography, particularly in terms of their goals and threat models. When discussing modern research, explain how machine learning and deep learning have changed both steganographic embedding and steganalysis, while avoiding the implication that newer methods are automatically better. Important source-handling requirements:

- Use only information supported by the supplied research synthesis.
- Do not invent sources, citations, authors, dates, algorithms, performance figures, datasets, or experimental results.
- Associate claims with their supporting sources where the synthesis provides source information.
- The current synthesis contains headings for source associations but does not actually list the source names under them. Do not fabricate these citations. Instead, mark claims requiring source attribution as [SOURCE TO VERIFY] where appropriate.
- Clearly distinguish established concepts from individual research findings and open research questions.
- Do not generalise the findings of individual studies beyond what the synthesis supports.

Maintain my writing style: professional, clear, engaging, technically accurate, confident, open to discussion where appropriate, and polite. Avoid unnecessary buzzwords, exaggerated claims, repetitive explanations, and generic AI-generated phrasing. Prefer clear connected prose over excessive bullet points. Do not critique or revise the paper yet. This is the first-draft stage only. Here is the research synthesis:

- The output is:

Steganography: Principles, Methods, and Contemporary Developments — A Research Synthesis

Draft for review

1. Introduction

Steganography is the practice of hiding information within other, seemingly innocuous data so that the presence of the hidden message itself remains undetected. As digital communication has grown in volume and complexity, the techniques used to conceal information — and the techniques used to detect that concealment — have evolved considerably, moving from simple manipulations of file formats to sophisticated applications of deep learning and generative modelling. This paper synthesises the fundamental principles of steganography, its historical and theoretical foundations, the major technical approaches used to embed and detect hidden data, and the developments that are currently reshaping the field. The aim is to give a reader with a general computing and AI background a clear, technically grounded understanding of the subject, without assuming prior exposure to steganography specifically.

2. What is Steganography?

At its core, steganography addresses a single question: how can a message be hidden inside another piece of data — an image, an audio file, a video, or even ordinary text — such that an observer sees nothing unusual? Unlike secrecy achieved by scrambling a message's content, steganography aims to make the very existence of a hidden message invisible. A successful steganographic system produces a "stego-object" (the carrier plus hidden message) that is statistically and perceptually indistinguishable from an ordinary, unmodified "cover object" (the carrier alone).

This distinction between hiding content and hiding existence is central to understanding why steganography is treated as a separate discipline from cryptography, even though the two are frequently used together in practice.

3. Steganography and Cryptography

Steganography and cryptography address different threat models and pursue different goals, and it is worth being precise about this distinction. Cryptography assumes that an adversary may see the encrypted message and focuses on making its *content* unreadable without the correct key — the existence of communication is

not hidden, only its meaning. Steganography, by contrast, assumes that an adversary is watching a communication channel for anomalies, and its goal is to conceal the *existence* of the message so that no communication appears to be occurring at all. As the synthesis puts it, cryptography secures a message's content by producing unreadable ciphertext, while <cite index="1-1">steganography secures the communication channel by concealing the very existence of the message</cite>.

It is also useful to distinguish steganography from digital watermarking, a related but distinct discipline. Both involve embedding information into a carrier, but their priorities differ: <cite index="1-24">steganography prioritizes secrecy, while watermarking prioritizes robustness against active distortion or attacks</cite>. A watermark is often expected to survive deliberate attempts to remove or damage it (for example, resizing or re-compressing an image), and its presence may even be publicly known — the challenge is protecting its integrity, not its secrecy. A steganographic payload, by contrast, is only useful for as long as its presence remains unsuspected; robustness against manipulation is often a secondary concern compared to staying undetected. This trade-off matters when choosing or evaluating a technique for a given task and reappears later in the discussion of the "steganographic triangle."

4. History and Fundamental Concepts

Steganography is not a purely digital invention — the underlying idea of concealing communication predates modern computing — but the theoretical models used to formalise and analyse it are more recent, and it is these formal concepts that provide the conceptual backbone for the rest of this paper.

The Prisoners' Problem

The classical formalisation of the problem is Simmons' **Prisoners' Problem**. In this setup, two prisoners, conventionally named Alice and Bob, wish to coordinate an escape plan. <cite index="1-8">They must coordinate an escape through messages monitored by a warden, Eve; any detected anomaly leads to communication termination.</cite> This scenario captures the essential adversarial structure of steganography: the hidden communicators are not simply trying to keep a secret from being read, but trying to avoid triggering suspicion in the first place. If the warden (Eve) detects anything statistically unusual about the messages passing between Alice and Bob, the channel is shut down regardless of whether she can actually decode the hidden content. This is what distinguishes steganographic security from cryptographic security: detection alone is a failure, even without decryption.

Cachin's Security Model

Building on this adversarial framing, **Cachin's security model** offers a way to formally measure how "secure" a steganographic scheme is. In this framework, security is quantified by the statistical similarity between the distribution of cover objects and the distribution of stego-objects (objects containing a hidden message). Specifically, security is measured by the <cite index="1-9">Kullback-Leibler (KL) divergence between the distribution of cover objects and stego-objects</cite>, with

<cite index="1-9">perfect security achieved when the divergence is zero</cite> — that is, when a stego-object is drawn from exactly the same distribution as an ordinary cover object, making it theoretically impossible for even an optimal statistical test to distinguish the two. This model gives researchers a rigorous, information-theoretic target to aim for, even though achieving zero divergence in practice, for arbitrary embedding rates, is an extremely demanding goal.

The Steganographic Triangle

Practical steganographic systems must navigate a persistent trade-off often described as the **steganographic triangle**: the competing goals of embedding capacity, imperceptibility, and robustness. Implementations are constrained by <cite index="1-2">the competing goals of embedding capacity (payload volume), imperceptibility (statistical/visual indistinguishability), and robustness (survival under distortion)</cite>. Increasing how much data can be hidden (capacity) tends to make the changes to the cover object more noticeable, whether to the human eye/ear or to a statistical detector (imperceptibility), and neither of these can typically be maximised without affecting how well the hidden message survives further processing or transmission (robustness). Different applications will sit at different points on this triangle: a covert communication channel between two parties might prioritise imperceptibility and accept lower capacity, whereas a use case tolerant of detection risk might push for higher capacity.

5. Types of Steganography

Steganography can be applied to essentially any digital medium with sufficient redundancy — that is, more information capacity than is strictly needed to represent the content meaningfully. The major categories are as follows.

Image steganography is <cite index="1-11">the most popular form due to high digital redundancy, and it utilizes pixel intensities to hide information</cite>. Images are a natural target because small changes to individual pixel values are often imperceptible to the human eye, and image files are large enough to accommodate meaningful hidden payloads.

Audio steganography works by <cite index="1-12">exploiting the limitations of the Human Auditory System</cite>. Two notable techniques are <cite index="1-12">phase coding, which hides data in inaudible phase shifts, and echo data hiding, which embeds bits through small, controlled delays</cite>. Both exploit aspects of sound that human hearing is relatively insensitive to, allowing data to be embedded without perceptible distortion.

Video steganography benefits from the fact that video <cite index="1-13">offers immense capacity and masks data using temporal dependencies</cite>, meaning that changes can be spread across many frames over time rather than concentrated in a single image. Key approaches include <cite index="1-13">modifying motion vectors or intra-frame prediction modes</cite>, both of which are internal components of how video compression represents movement and structure between frames.

Text (linguistic) steganography has a longer and more varied history than the other categories. <cite index="1-14">Traditionally it involved format changes such as spaces and tabs</cite>, exploiting formatting choices that are largely invisible to a casual reader. More recently, <cite index="1-14">modern generative linguistic steganography uses Large Language Models to synthesize fluent text directly from message bits</cite> — rather than modifying existing text, the hidden message effectively determines which words or phrases a language model generates, producing text that reads naturally while encoding the secret payload in its word choices. This generative approach to text steganography is discussed further in Section 9.

6. Technical Methods

Spatial-Domain Techniques

Spatial-domain methods work directly on the carrier's raw values — for an image, this means the pixel values themselves, rather than any transformed or compressed representation of them. These methods are conceptually the simplest form of steganography and are typically the first techniques introduced to newcomers to the field.

LSB substitution is the most widely used spatial-domain method. It works by replacing the **least significant bit(s)** of each cover element (for example, each colour channel of each pixel) with bits from the secret message. A pixel's brightness or colour value is represented as a binary number; because the least significant bit contributes the smallest possible amount to that value, changing it produces a change in the pixel's actual value of at most one unit out of, typically, 256 possible levels. This change is generally imperceptible to the human eye, which is why LSB substitution can hide substantial amounts of data with little visible distortion. The synthesis describes this method as <cite index="1-16">replacing the lowest bits of cover elements with message bits, and notes that while simple and providing high capacity, these methods are highly vulnerable to statistical analysis</cite>. The vulnerability arises precisely because LSB substitution alters the statistical relationships between neighbouring pixel values in ways that, while invisible to a human observer, are detectable by tools designed to look for exactly this kind of anomaly (see Section 7).

Transform-Domain Techniques

Transform-domain methods take a different approach: instead of modifying raw pixel or sample values directly, they first convert the carrier into a different mathematical representation — a set of **frequency coefficients** — and embed the message by altering those coefficients. Information is embedded in <cite index="1-17">frequency coefficients, for example those produced by the DCT, DWT, or FFT</cite>. The key idea is that certain frequency components of an image or audio signal are less perceptually important than others (for example, very fine, rapidly-changing detail is often less noticeable than broad, smooth variation), so embedding data in the "right" coefficients allows changes to survive processes that would otherwise disrupt a naive spatial-domain embedding, such as compression.

DCT and DWT

The **Discrete Cosine Transform (DCT)** is widely used in JPEG steganography, largely because DCT is also the transform used internally by the JPEG compression standard itself. It works by <cite index="1-18">breaking images into 8x8 blocks and hiding data in quantized AC coefficients</cite>. Because the message is embedded in coefficients that are already part of the image's compressed representation, DCT-based steganography tends to be <cite index="1-18">more robust than spatial methods against compression</cite> — the hidden data is, in a sense, embedded in a form that is already compatible with how the carrier will typically be stored or transmitted, rather than being layered on top of the raw pixel data where it would be more easily disrupted by re-compression.

The **Discrete Wavelet Transform (DWT)** offers a different set of properties. It <cite index="1-19">provides a multi-resolution description, matching human visual perception more closely and offering high robustness</cite>. Rather than representing the image at a single fixed scale (as DCT's fixed 8x8 blocks do), DWT decomposes the image into multiple layers of detail at different resolutions, which aligns more naturally with how the human visual system processes both coarse structure and fine detail simultaneously. This multi-resolution property is part of why DWT-based methods are noted for high robustness.

It is worth being careful here not to treat "more robust" as synonymous with "better" in an unqualified sense: robustness is only one vertex of the steganographic triangle described in Section 4, and a method optimised for robustness may trade away capacity or imperceptibility. The choice between spatial- and transform-domain methods, or between DCT and DWT specifically, depends on which properties a given application actually requires.

7. Steganalysis and Detection

Steganalysis is the counterpart discipline to steganography: the study of detecting, and where possible extracting, hidden messages. Just as steganographic embedding techniques have diversified, so too have the methods used to detect them.

Targeted vs. Universal Steganalysis

A basic distinction in steganalysis is between targeted and universal approaches. <cite index="1-20">Targeted steganalysis exploits specific vulnerabilities of known algorithms, whereas universal (blind) steganalysis uses machine learning to detect anomalies independently of the embedding method</cite>. Targeted approaches can be highly effective against a specific, known embedding technique but generalise poorly to techniques they were not designed for. Universal approaches sacrifice some of this specificity in exchange for broader applicability, since they attempt to learn general statistical signatures of "anomalous" data rather than signatures tied to one particular algorithm.

Rich Models

One influential family of universal, statistically-driven detection tools is known as **rich models**. The <cite index="1-21">Spatial Rich Model (SRM) uses a union of diverse high-pass filters to capture noise residuals across neighbouring pixel dependencies</cite>. In essence, this approach applies many different filters designed to highlight subtle, high-frequency noise patterns in an image, on the reasoning that steganographic embedding — even when imperceptible to the eye — tends to disturb the natural statistical relationships between neighbouring pixels in ways these filters can capture. By combining many such filters ("rich" in the sense of covering a wide variety of statistical features), SRM and similar models aim to catch a broad range of embedding techniques rather than being tuned to just one.

Deep-Learning-Based Detection

More recently, steganalysis — like embedding — has been transformed by deep learning. <cite index="1-22">CNNs have largely replaced handcrafted features</cite> such as those used in rich models. Rather than a human researcher designing specific filters to capture suspected statistical anomalies, a convolutional neural network can be trained directly on examples of cover and stego objects, learning for itself which patterns are indicative of hidden data. A notable architecture in this space is **SRNet**, which <cite index="1-22">uses deep residual networks to learn noise residuals without requiring handcrafted pre-filters</cite>. This represents a shift from steganalysis as a manually engineered statistical exercise to one where much of the feature-discovery process is delegated to the learning algorithm itself — though, as discussed in Section 11, this shift brings its own limitations, particularly around how well models trained on one dataset generalise to new, real-world data.

8. Security, Capacity, Imperceptibility and Robustness

It is useful to return to and slightly expand on the core evaluation criteria for steganographic systems, since they recur throughout the literature and inform how different techniques are compared.

Capacity refers to how much secret data can be embedded in a given cover object. **Imperceptibility** refers to how statistically and visually indistinguishable a stego-object is from an unmodified cover — this includes both what a human observer would notice and what a statistical test might detect, and these are not always the same thing (a change can be invisible to the eye yet detectable by a rich model or CNN). **Robustness** refers to whether the hidden message survives further processing of the carrier, such as compression, resizing, or transmission.

The synthesis also draws a further, related distinction between **security** and **secrecy**: <cite index="1-23">some sources distinguish between security (preventing detection) and secrecy (preventing extraction or decoding even if the channel is detected)</cite>. This is a meaningful nuance — a scheme could in principle be detected as containing hidden data (a failure of security) while still keeping the actual message content protected from extraction (a success of secrecy), and the two properties are not automatically linked. This distinction is [SOURCE TO VERIFY] but is a useful conceptual separation to keep in mind when evaluating what a given steganographic system is actually claiming to protect against.

The **distortion minimization framework** is a practical optimisation approach used to balance these competing goals during embedding. Rather than embedding a message at fixed, predictable locations throughout a cover object, this framework guides embedding changes using a <cite index="1-6">localized cost map to cluster modifications in textured or noisy regions that are difficult for steganalysts to model</cite>. The underlying logic is that regions of an image that are already visually complex or "noisy" — for example, busy textures rather than smooth, uniform areas — can better hide small alterations, both because human perception is less sensitive to change there and because statistical models (including rich models) find such regions harder to characterise precisely. By concentrating embedding changes in these regions, a distortion-minimising scheme can improve imperceptibility and resistance to detection without necessarily sacrificing capacity.

9. Modern Developments

Deep Learning

As already discussed in the context of steganalysis, deep learning has reshaped much of the field, moving analysis away from handcrafted statistical features and toward learned representations. The synthesis frames this as a broader trend: <cite index="1-4">contemporary steganalysis has moved from handcrafted statistical models to end-to-end neural network architectures such as CNNs, RNNs, and GANs that can automatically extract subtle embedding traces</cite>. It is worth noting that this same family of architectures — CNNs, RNNs, and GANs — has also influenced steganographic *embedding*, not just detection, which sets up something of an ongoing contest between learned embedding techniques and learned detection techniques.

Generative Steganography

One of the more significant shifts described in the synthesis is the move from modification-based to generation-based steganography. <cite index="1-3">Modern steganography is shifting from modification-based approaches, which alter existing covers, to generation-based approaches, which synthesize new, natural-looking media directly from secret bitstreams</cite>. This is a meaningfully different paradigm: rather than starting with an existing image or audio file and subtly altering it (which necessarily introduces some statistical trace, however small, of the original unmodified content), a generative approach produces an entirely new piece of media whose content is itself determined by the secret message. Because there is no "original" cover object being modified, this approach <cite index="1-3">complicates detection by evading traditional pixel-level artifacts</cite> — the sorts of statistical anomalies that rich models and CNN-based steganalysis are designed to catch are premised, at least in part, on comparing a stego-object to what an unmodified cover "should" look like, and this comparison is less straightforward when there was never an unmodified cover to begin with.

This shift also introduces new design challenges, described in the synthesis as the **Generative Steganography Trilemma**: <cite index="1-25">a disagreement exists in how to prioritize the three goals for generative models — Optimal Capacity, Optimal Efficiency (O(1) complexity), and Optimal Security</cite>. As with the

steganographic triangle discussed earlier, these goals are in tension. The synthesis notes that <cite index="1-25">methods like Arithmetic Coding offer optimal capacity but often compromise on efficiency and statistical security</cite>, illustrating that generative approaches, while promising, have not resolved the underlying trade-offs of steganography so much as recast them in a new form. It should not be assumed that generative methods are simply "better" than modification-based methods; they represent a different point of balance among the same competing constraints, with their own strengths and open problems ([SOURCE TO VERIFY] for the specific comparative claims regarding Arithmetic Coding's efficiency trade-offs).

Generative Linguistic Steganography

Within text steganography specifically, the generative paradigm takes the form described earlier in Section 5: <cite index="1-14">modern generative linguistic steganography uses Large Language Models to synthesize fluent text directly from message bits</cite>. This is a natural extension of the modification-versus-generation shift into the text domain — instead of hiding data in the formatting of existing text, the language model's own generative process is steered by the secret bits, producing text that is fluent and natural while its specific word choices encode the hidden payload.

Coverless Steganography

Coverless steganography represents perhaps the most conceptually distinct of the modern developments discussed in the synthesis. Rather than embedding a message into a cover object at all — whether by modification or generation — coverless approaches hide information without any modification to the carrier, by <cite index="1-5">establishing mapping relationships between secret data and existing semantic features, an approach referred to as Deep Semantic Mapping</cite>. In this scheme, the secret message is not embedded into the object in any conventional sense; instead, an existing, entirely unmodified object is selected or matched because its inherent semantic features (for example, particular characteristics recognisable by a deep learning model) correspond to the message being communicated. Because no modification takes place, many of the statistical detection techniques discussed in Section 7 — which rely on identifying traces left by an embedding process — are not applicable in the same way, since there is no embedding process to leave a trace. This makes coverless steganography a genuinely different technical strategy rather than simply a more advanced version of modification- or generation-based embedding.

10. Applications

The synthesis provided does not detail specific application domains for steganography in depth, and this section is accordingly limited. In general terms, based on the threat models and technical properties already discussed, steganographic techniques are relevant wherever covert communication or the discreet embedding of information is desired — this includes contexts requiring covert communication channels, as suggested by the Prisoners' Problem framing itself. Any more specific claims about deployed applications, industries, or use cases

would go beyond what is supported by the current synthesis and are marked here as an area requiring further source material [SOURCE TO VERIFY].

11. Limitations and Research Challenges

Several open challenges emerge from the synthesis, spanning both steganographic embedding and steganalysis.

Cover Source Mismatch (CSM) is described as a major challenge in which <cite index="1-26">detection accuracy degrades significantly when real-world data distributions differ from the training datasets</cite>. This is a particularly important limitation for deep-learning-based steganalysis: a model trained to detect stego-objects derived from one type of cover data (for example, images from a particular camera or processing pipeline) may perform considerably worse when applied to cover data from a different source, since the statistical patterns the model learned to recognise as "normal" may not transfer.

Real-time detection is also identified as an unresolved problem, particularly for <cite index="1-27">high-bandwidth, low-latency communication such as VoIP streams or live video</cite>. The computational demands of thorough statistical or deep-learning-based analysis are in tension with the speed required to analyse a live, continuous stream of data, making real-time steganalysis substantially harder than analysing static files after the fact.

There is also a **lack of standardisation** in the field: the synthesis notes <cite index="1-28">a lack of unified taxonomy and standardized evaluation frameworks for comparing diverse deep semantic mapping and generative approaches</cite>. Without agreed benchmarks or shared terminology, it becomes difficult to make confident comparative claims about which techniques genuinely outperform others, or under what conditions — a caution worth bearing in mind when reading claims about "improved" methods in this space, including in this paper.

Finally, most theoretical treatments of steganographic security assume a **passive warden** — an observer who monitors communication but does not interfere with it. The synthesis notes that <cite index="1-29">resilience against active attackers who intentionally perturb traffic is less explored in high-capacity contexts</cite>. This is a meaningful gap: an adversary who can actively modify or inject noise into a communication channel (rather than merely observing it) represents a different and, in some respects, more demanding threat model than the one most information-theoretic security results are built around.

12. Future Research Directions

Based on the limitations identified above, several directions for future research follow naturally from the synthesis, though it should be emphasised that these are inferred from the identified gaps rather than stated as explicit research programmes in the source material.

Addressing Cover Source Mismatch would likely require methods that are more robust to distributional shift between training and deployment data, an issue also

relevant across many other applications of machine learning beyond steganalysis specifically. Progress on real-time detection would need approaches that meaningfully reduce the computational cost of analysis without sacrificing accuracy, particularly for streaming media. The absence of standardised evaluation frameworks suggests a clear opportunity — and arguably a prerequisite — for the field to develop shared benchmarks, particularly for comparing generative and coverless approaches, which are currently difficult to evaluate against one another on common terms. Finally, extending security models beyond the passive warden assumption to account for active adversaries appears to be a genuine theoretical gap, particularly as steganographic capacity increases and generative methods become more prevalent. These directions are offered as reasonable extrapolations from the stated research gaps rather than as claims about specific ongoing work [SOURCE TO VERIFY].

13. Conclusion

Steganography occupies a distinct position among information-security disciplines: where cryptography protects the content of a message and watermarking protects the integrity of an embedded signal, steganography protects the very fact that a message exists. This paper has traced the field from its foundational theoretical models — the Prisoners' Problem and Cachin's KL-divergence-based security framework — through the practical trade-offs captured by the steganographic triangle, to the concrete technical methods used to embed data in the spatial and transform domains, and the corresponding detection techniques developed in steganalysis. The recent influence of deep learning is evident on both sides of this contest: embedding techniques have moved toward generative and coverless paradigms that sidestep some traditional detection strategies, while steganalysis has moved toward learned, end-to-end architectures that can adapt to a wider range of embedding methods than earlier handcrafted statistical models. Importantly, none of these developments should be read as settling the field's underlying tensions — capacity, imperceptibility, robustness, security, and now efficiency in the generative setting remain in competition, and open challenges around distributional mismatch, real-time analysis, standardisation, and active adversaries suggest that steganography remains an active and unresolved area of research rather than a solved technical problem.

Note on sourcing: This draft synthesises claims from the provided research summary. Where the summary indicated a supporting source category without naming specific sources, claims have been marked [SOURCE TO VERIFY] rather than attributed to invented citations. A revision pass incorporating verified, named sources is recommended before this draft is considered final.

- *Critique (Claude):*
    - The prompt given is: Critically evaluate the following research synthesis and draft paper on “Steganography: Principles, Methods, and Contemporary Developments.” Do not rewrite the paper yet. I want a rigorous academic critique first. Evaluate:

1. Research quality — Is the research sufficiently comprehensive and academically credible?
2. Technical accuracy — Identify any technically inaccurate, misleading, oversimplified, or outdated claims.
3. Depth — Identify areas that need deeper technical explanation or stronger evidence.
4. Structure and coherence — Assess whether the sections flow logically and whether any sections should be merged, removed, or reorganised.
5. Argument quality — Determine whether the paper actually analyses and synthesises the literature or merely describes concepts.
6. Balance — Check whether competing perspectives, limitations, trade-offs, and disagreements are adequately represented.
7. Evidence — Identify claims that require citations, claims that are weakly supported, and claims that need stronger academic sources.
8. Research gaps — Evaluate whether the identified gaps are genuinely supported by the literature or are assumptions/inferences.
9. Contemporary relevance — Assess whether the discussion of deep learning, generative steganography, LLM-based linguistic steganography, coverless steganography, and AI-based steganalysis is sufficiently current.
10. Originality and research value — Assess whether the paper has a meaningful angle or whether it remains too broad/general.
11. Academic writing — Identify problems with terminology, repetition, unsupported statements, informal phrasing, and clarity.

Pay particular attention to:

- the distinction between steganography, cryptography, and watermarking;
- Simmons' Prisoners' Problem;
- Cachin's security model;
- the steganographic triangle;
- LSB, DCT and DWT;
- steganalysis and SRM;
- SRNet and deep-learning-based detection;
- distortion minimisation and UNIWARD;
- generative steganography;
- generative linguistic steganography;
- coverless steganography;
- Cover Source Mismatch;
- real-time steganalysis;
- active adversaries;
- the Generative Steganography Trilemma.

For each issue identified, classify it as: Critical / Major / Minor / Optional Then provide: A. Overall assessment Give the paper a score out of 10 for research quality, technical depth, academic quality, and originality. B. Major weaknesses List the most important problems that should be fixed before submission. C. Section-by-section critique Review Sections 1–13 individually. D. Missing content Identify important concepts, papers, methods,

comparisons, or perspectives that are missing. E. Claims requiring verification List every claim that needs additional evidence or citation. F. Recommended improvements Give specific, actionable recommendations for improving the paper. G. Final verdict State whether the current draft is:

- Not ready
- Needs major revision
- Needs moderate revision
- Nearly ready

Be critical rather than complimentary. Do not assume that statements in the draft are correct simply because they appear in the research synthesis.

- The output is:

Critical Review: "Steganography: Principles, Methods, and Contemporary Developments"

A. Overall Assessment

Dimension	Score (/10)	Rationale
Research quality	4/10	Entirely dependent on a single, uncredited synthesis with no named sources, no dates, no bibliography. Cannot be assessed as "comprehensive" because it is bounded by what one document chose to summarise.
Technical depth	5/10	Correctly explains *what* several methods do; consistently under-explains *how* the more advanced or coding-theoretic mechanisms actually work (STC, ensemble classifiers, UNIWARD's cost function itself).
Academic quality	3/10	No verifiable citations, no reference list, heavy reliance on paraphrased-with-tags synthesis content rather than independent critical synthesis. Reads as a well-organised literature summary, not a research paper.
Originality	2/10	No original argument, no novel comparison, no stated research question. It is a structured explainer, not a contribution.

This draft is a **competent technical explainer**, not yet a **research paper**. That distinction matters and should drive the revision.

B. Major Weaknesses (ranked)

1. **Critical — No real citations exist.** Every substantive claim is tagged [SOURCE TO VERIFY] or attributed vaguely to "the synthesis." An academic paper cannot be evaluated for research quality if none of its claims

are traceable to an actual, checkable source (author, year, venue). This is the single largest issue and undermines every other section.
2. **Critical — No reference list / bibliography.** Related to (1): even in draft form, a research paper should carry placeholder citations (e.g., "Simmons, 1984," "Cachin, 1998," "Holub & Fridrich, 2014" for UNIWARD, "Boroumand et al., 2019" for SRNet) rather than leaving every attribution as an open question. Right now a reader cannot distinguish "well-established textbook fact" from "single paper's finding."
3. **Major — Named techniques promised by the synthesis headings are missing from the body text.** Syndrome-Trellis Codes (STC), UNIWARD (by name), and Minimum Entropy Coupling (iMEC) all appear as *source headings* in the synthesis but are never actually explained in the draft. This is a real content gap, not just a citation gap — STC in particular is the standard mechanism by which "distortion minimisation" is actually implemented, and the draft describes the *goal* (concentrate distortion in noisy regions) without describing the *mechanism* (coding scheme that achieves near-optimal embedding efficiency for a given distortion budget).
4. **Major — Descriptive, not analytical.** Almost every paragraph follows the pattern "X is defined as Y [cited to synthesis]." There is very little genuine synthesis — i.e., comparing claims against each other, identifying where they might be in tension, or evaluating strength of evidence. The paper tells the reader what concepts are; it rarely tells the reader what to *think* about them.
5. **Major — No temporal or contemporary anchoring.** A paper claiming to cover "Contemporary Developments" contains no dates, no named recent papers, no indication of how recent "recent" is. Deep-learning steganalysis and generative steganography have moved quickly; without dates or named studies, "modern" and "contemporary" are unfalsifiable adjectives.
6. **Moderate — Repetition between Sections 5 and 9.** Generative linguistic steganography is described almost identically in Section 5 (Types) and Section 9.3 (Modern Developments), using overlapping phrasing and the same underlying synthesis sentence. This should be consolidated.
7. **Moderate — One-sided framing of newer methods despite disclaimers.** The paper repeatedly inserts a caveat ("this should not be read as newer being better") but the actual prose devotes far more space to explaining the appeal of generative/coverless methods than to their practical costs (e.g., coverless methods requiring large indexed databases and generally very low embedding capacity; generative methods' fluency-vs-payload trade-offs). The disclaimers are present but structurally outweighed.
8. **Minor — Some claims are asserted more confidently than the underlying synthesis supports.** E.g., "DCT is more robust than spatial methods against compression" is stated as a general truth; the synthesis supports this only as a general tendency, not a universal ranking (adaptive spatial methods with distortion minimisation can be quite robust too, and the synthesis doesn't compare them directly).

C. Section-by-Section Critique

1. Introduction — Adequate scene-setting. *Minor*: makes a claim of "growing volume and complexity" of digital communication that is generic scene-setting not supported by any source — flag as unsupported framing device, common in AI-generated introductions, should be trimmed or justified.

2. What is Steganography? — Clear and accurate at a definitional level. *Minor*: "statistically and perceptually indistinguishable" is doing a lot of work here that isn't unpacked until much later (Section 7–8); consider forward-referencing explicitly.

3. Steganography and Cryptography — This section actually does two jobs (crypto distinction *and* watermarking distinction) but only crypto is in the title. *Minor* structural issue: rename to "Steganography, Cryptography, and Watermarking" for honesty, or move watermarking to its own short subsection. Content itself is accurate to the synthesis and is one of the stronger, more analytically useful sections (it foreshadows the triangle).

4. History and Fundamental Concepts — Title promises "history" but delivers almost none — a *Major* mismatch between heading and content. The synthesis itself provides essentially no historical material (no dates, no named ancient techniques beyond a source heading called "History and Ancient Techniques" that is never actually filled in). This is a case where the draft should have flagged the gap explicitly rather than silently omitting the promised content; the paper does this reasonably well elsewhere but not here. Recommend renaming the section to "Fundamental Concepts" and adding an explicit note that historical material was not available in the source synthesis.

5. Types of Steganography — Solid, well-organised, correctly scoped to the synthesis. *Minor*: the generative linguistic steganography content duplicates Section 9.3 almost verbatim (see weakness #6 above).

6. Technical Methods — The strongest technical section in terms of mechanism-level explanation (LSB and the "changes value by at most one unit" explanation is genuinely useful and correct at the level of individual bit-planes). *Major* gap: DCT/DWT discussion never engages with **Syndrome-Trellis Codes**, which is the actual state-of-the-art mechanism connecting a distortion function to a concrete embedding algorithm — without it, "distortion minimisation" in Section 8 reads as a vague heuristic rather than an actual coding-theoretic result. *Minor*: LSB discussion says "least significant bit(s)" (plural) but only explains the single-bit case; should either extend to multi-bit-plane embedding or drop the plural.

7. Steganalysis and Detection — Good conceptual separation of targeted/universal/SRM/CNN-based approaches. *Major*omission: SRM is historically always paired with an **ensemble classifier** (not a single monolithic model) — omitting this makes SRM sound like a self-contained detector rather than a feature-extraction step that still requires a downstream classifier. *Minor*: SRNet is presented as representative of "deep learning steganalysis" broadly, but it is one architecture among several (e.g., earlier CNN steganalysis work) — the synthesis supports only the SRNet claim specifically, and the draft should be more careful not to let one named architecture stand in for an entire subfield.

8. Security, Capacity, Imperceptibility, Robustness — Conceptually useful, but this section overlaps substantially with Section 4 (triangle) and arguably should be merged with it rather than split apart with a 4-section gap in between. *Minor*: the security-vs-secrecy distinction is interesting but is asserted then dropped — never revisited when discussing, e.g., coverless steganography, where the distinction is arguably most relevant (a coverless message may be secure from detection precisely because it is never "extracted" from anything).

9. Modern Developments — Content is reasonable but suffers from three issues: (a) repetition with Section 5 (linguistic stego), (b) the "Generative Steganography Trilemma" is presented as an established field-wide framework/disagreement when the synthesis frames it more narrowly ("a disagreement exists" — it's unclear how many sources actually hold this trilemma framing versus it being one paper's proposed taxonomy), and (c) "O(1) complexity" is used without defining what operation this refers to (per-bit embedding cost? per-symbol generation cost?) — this is jargon presented without the explanation the assignment brief explicitly asked for.

10. Applications — Weakest section, appropriately flagged as such by the draft itself. Given how thin this is, *Major*recommendation: either cut this section down to 2–3 sentences and merge into the Introduction/Conclusion, or clearly mark it as a stub pending further sourcing. As currently written, a full numbered section for essentially one hedged sentence is disproportionate.

11. Limitations and Research Challenges — One of the better sections; correctly separates named challenges (CSM, real-time detection, standardisation, active adversaries) and ties them back to earlier technical discussion. *Minor*: doesn't connect CSM back to the *pre-deep-learning* steganalysis literature, where distributional mismatch between training and test cover sources was also a known issue (i.e., this is not a deep-learning-specific problem) — the synthesis may not support this addition, but it's a place where an author with domain knowledge should flag the omission.

12. Future Research Directions — Honestly labelled as inference rather than sourced claims, which is good practice. *Major* weakness: the actual content is generic ML platitudes ("methods more robust to distributional shift") that could be written about almost any ML application, not steganography specifically. This section currently adds little unique value and reads as filler.

13. Conclusion — Reasonable summary; doesn't introduce new unsupported claims, which is good discipline. *Minor*: slightly repeats phrasing from the Introduction almost verbatim ("occupies a distinct position," "protects the very fact that a message exists" echoes similar phrasing earlier) — tighten for non-repetition.

D. Missing Content

- **Syndrome-Trellis Codes (STC)** — named in the synthesis's source list but never explained. This is arguably the single most important missing technical

mechanism, since it is what actually operationalises "distortion minimisation" into a concrete embedding algorithm.
- **UNIWARD**, named explicitly in the assignment brief and in the synthesis's source headings, is never named in the draft body — only described generically as "a localized cost map." Should be named and briefly explained (it is a specific distortion function family: WOW, S-UNIWARD, etc.).
- **Minimum Entropy Coupling (iMEC)** — appears only as an unexplained heading in the synthesis; the draft correctly avoids inventing content for it, but should explicitly flag this as an unaddressed gap rather than silently dropping it.
- **Ensemble classifiers** as the standard companion to rich-model features (SRM).
- **A references/bibliography section**, even in placeholder form.
- **Any discussion of practical limitations of coverless and generative methods** (capacity ceilings, retrieval/database overhead for coverless approaches, fluency-vs-payload trade-offs for generative linguistic stego) — currently the paper explains their appeal (evading pixel-level detection) far more than their costs.
- **A clear statement of the paper's own scope and limitations as a synthesis-only document** — this should probably appear in the Introduction, not just as a closing note.

E. Claims Requiring Verification

All of the following are currently unsupported by any checkable source and should be either cited properly or explicitly softened to "reportedly" / flagged as unverified in any revision:

- The KL-divergence formulation of Cachin's model and the exact "zero divergence = perfect security" claim (Section 4).
- The specific claim that DCT is "more robust than spatial methods against compression" as a general rule (Section 6).
- The claim that DWT "matches human visual perception more closely" (Section 6) — this is a design rationale, not a measured result.
- The Generative Steganography Trilemma framing itself, and the specific claim that Arithmetic Coding achieves "optimal capacity" but compromises efficiency/security (Section 9).
- The security-vs-secrecy distinction as a field-wide convention rather than one source's framing (Section 8).
- All four items in Section 11 (CSM, real-time detection limits, lack of standardisation, passive-warden assumption) — plausible and consistent with general knowledge of the field, but currently traceable only to an unnamed synthesis document.

F. Recommended Improvements

1. Replace every [SOURCE TO VERIFY] tag with an actual placeholder citation format (even "Author, Year — TO CONFIRM") so the reader can see the *shape* of the intended evidence base, and begin sourcing real citations for foundational claims (Simmons, Cachin, Fridrich/Holub for UNIWARD, Boroumand for SRNet) before this is treated as submission-ready.
2. Add a short explanation of Syndrome-Trellis Codes to Section 6 or 8, and name UNIWARD explicitly rather than only describing it functionally.
3. Merge Sections 4 and 8 (both address the triangle/trade-off concept) to avoid splitting one idea across the paper.
4. Cut the duplicated generative-linguistic-steganography content — keep the full explanation in Section 9 and reduce Section 5 to a one-line forward reference.
5. Shrink Section 10 (Applications) substantially or mark it clearly as underdeveloped pending sources, rather than presenting a near-empty section at full structural weight.
6. Add a paragraph explicitly weighing the practical *costs* of generative and coverless methods (capacity, computational overhead, reliance on large feature databases) to balance the current emphasis on their detection-evasion advantages.
7. Add an explicit "Scope and Limitations" note early in the paper (Introduction or a short new subsection) stating that the paper is derived entirely from a single provided synthesis without independent literature verification — this is more honest than only disclosing it at the very end.
8. Tighten repeated framing phrases ("the synthesis notes/describes/indicates") — used dozens of times — by varying sentence structure and reducing reliance on this scaffolding once real citations replace synthesis-attribution.

G. Final Verdict

Needs major revision.

The draft is well-organised, generally faithful to its source material, and appropriately cautious about not overstating what is supported — those are real strengths. But as an academic research paper it currently fails on the two dimensions that matter most for that classification: it has no verifiable evidence base (no real citations, no bibliography), and it is structurally descriptive rather than analytical (it explains concepts well but rarely compares, weighs, or critiques them against each other). Before this is treated as a submittable draft, it needs (a) real sourcing work, (b) the missing technical mechanisms named above (STC, UNIWARD by name), and (c) a more balanced treatment of the costs — not just the appeal — of the "modern developments" discussed in Section 9.

- *Revise (Claude):*
    - The prompt given is: Revise the steganography research paper using the attached draft and critical review as the primary inputs. Do not simply rewrite the paper for better wording. Implement the critique substantively.

Requirements:

1. Preserve the paper's central topic and technical scope while improving its academic quality.
2. Replace unsupported claims with properly sourced claims where sources are available. Do not invent references, authors, publication dates, venues, or findings.
3. Add and explain Syndrome-Trellis Codes (STC) and explain how they operationalise distortion-minimisation-based embedding.
4. Explicitly introduce and explain UNIWARD/S-UNIWARD and its role as a distortion function/cost model.
5. Explain that Spatial Rich Models (SRM) are feature-extraction/statistical modelling approaches typically paired with an ensemble classifier, rather than presenting SRM as a complete classifier by itself.
6. Reduce repetition between the general "Types of Steganography" discussion and the "Modern Developments" section, particularly generative linguistic steganography.
7. Reorganise the discussion of capacity, imperceptibility, robustness, and security so that overlapping sections are consolidated where appropriate.
8. Present generative and coverless steganography in a balanced way by discussing both their potential advantages and their practical limitations, including capacity, computational requirements, database/retrieval dependence, and payload/fluency trade-offs where supported by the sources.
9. Do not present the "Generative Steganography Trilemma" as an established universal framework unless the sources genuinely support that interpretation. Clearly attribute it to the relevant literature if it is a particular research proposal or perspective.
10. Improve the discussion of future research so that proposed directions are specifically grounded in identified steganography/steganalysis problems rather than generic machine-learning statements.
11. Either substantially strengthen the Applications section with sourced examples or reduce it to a concise discussion rather than maintaining an underdeveloped full section.
12. Add a clear Scope and Limitations statement explaining what the review covers and any limitations in the available literature.
13. Make the revised paper more analytical. Where appropriate, compare approaches, identify trade-offs, discuss limitations, and explain why one approach may be preferable under conditions.
14. Maintain technical accuracy and explain specialised terminology when first introduced.
15. Do not add unsupported claims simply to make the paper appear more complete.

Citation rules:

- Use only sources that can be verified from the provided research materials or newly retrieved authoritative academic sources.
- Do not fabricate citations.
- Provide a complete reference list at the end.
- Clearly distinguish established findings from interpretations or proposed future directions.

Output:
Produce the complete revised research paper, not a list of changes. After the paper, provide a brief Revision Summary listing the major changes made in response to the critique.

- The output:

Steganography: Principles, Methods, and Contemporary Developments

Revised draft, incorporating critical review

1. Introduction

Steganography is the practice of hiding information within other, seemingly innocuous data so that the presence of the hidden message itself remains undetected. As digital communication has grown in scale, the techniques used to conceal information — and the techniques used to detect that concealment — have evolved from simple manipulations of file formats to applications of coding theory, statistical modelling, and deep learning. This paper reviews the theoretical foundations of steganography, the principal technical methods used for embedding and detection, and several lines of contemporary research, with particular attention to where methods are well established, where they represent a specific research group's proposal rather than a field-wide consensus, and where genuine open problems remain.

2. Scope and Limitations of This Review

This paper is a synthesis rather than an original empirical study. Its coverage is necessarily bounded by the sources consulted, and several caveats apply throughout. First, the historical development of steganography prior to its digital, information-theoretic formalisation is not treated in depth here, since the available material for this review concentrated on the modern theoretical and algorithmic literature rather than early or pre-digital techniques; a fuller historical treatment is a natural extension for future work rather than something this paper attempts. Second, some concepts discussed below — notably the framing of generative steganography as facing a three-way "trilemma," and several claims about audio and video embedding techniques — could not be traced to a specific, independently verifiable publication within the scope of this review and are marked accordingly as requiring further verification, rather than presented as settled, citable findings. Third, the discussion of applications (Section 10) is intentionally brief, reflecting a genuine gap in the sources available for this review rather than an oversight. Readers should treat unmarked claims as supported by the cited literature and treat bracketed verification

notes as an explicit signal that a claim, while plausible and consistent with the broader literature, should not yet be relied upon as a settled academic fact.

3. What is Steganography?

At its core, steganography addresses a single question: how can a message be hidden inside another piece of data — an image, an audio file, a video, or even ordinary text — such that an observer sees nothing unusual? A successful steganographic system produces a "stego-object" (the carrier plus hidden message) that is statistically and perceptually indistinguishable from an ordinary, unmodified "cover object." This is a stronger requirement than simply making a message hard to read; it requires that the very act of communication leave no visible trace. Sections 5 and 8 below make this requirement more precise, first through the theoretical models used to define "indistinguishable," and then through the practical statistical tools used to test whether a given scheme actually achieves it.

4. Steganography, Cryptography, and Watermarking

Steganography, cryptography, and digital watermarking are frequently discussed together, but they address different threat models and pursue different goals, and conflating them is a common source of confusion.

Cryptography assumes that an adversary may see the encrypted message and focuses on making its *content* unreadable without the correct key; the existence of communication is not hidden, only its meaning. Steganography, by contrast, assumes that an adversary is watching a communication channel for anomalies, and its goal is to conceal the *existence* of the message so that no communication appears to be occurring at all. In practice, the two are complementary rather than competing: a message can be encrypted first and then steganographically embedded, so that even if the hidden channel were somehow discovered, the extracted content would still be protected. This layering illustrates why steganography is best understood as addressing a different threat model, rather than as a substitute for encryption.

Digital watermarking embeds information into a carrier in a broadly similar technical manner to steganography, but with an inverted priority structure. A watermark is often expected to survive deliberate attempts to remove or degrade it — resizing, re-compression, cropping — and its presence may even be publicly known; the challenge is protecting its *integrity*, not its secrecy. A steganographic payload, by contrast, is only useful for as long as its presence remains unsuspected, so imperceptibility and undetectability typically take priority over surviving deliberate attack. This distinction — robustness against active, known distortion (watermarking) versus secrecy against a suspicious but not necessarily hostile observer (steganography) — recurs when evaluating the trade-offs discussed in Section 5.

5. Fundamental Concepts: Security Models and the Capacity–Imperceptibility–Robustness Trade-off

5.1 The Prisoners' Problem

The classical formalisation of covert communication is Simmons' **Prisoners' Problem**. Simmons introduced a scenario in which two prisoners, Alice and Bob, held in separate cells, wish to coordinate an escape plan by exchanging messages that pass through a warden, who inspects all communications and will terminate the exchange — and punish the prisoners — the moment anything looks suspicious (Simmons, 1984). This scenario captures the essential adversarial structure of steganography: the communicators are not simply trying to keep a secret from being read; they are trying to avoid triggering suspicion in the first place. Detection alone is a failure, independent of whether the warden can actually decode the hidden content — a distinction that separates steganographic security from cryptographic security, where an adversary seeing ciphertext is an expected and tolerated part of the threat model.

5.2 Cachin's Information-Theoretic Security Model

Building on this adversarial framing, Cachin proposed a way to formally measure how "secure" a steganographic scheme is, by quantifying the statistical similarity between the distribution of cover objects and the distribution of stego-objects (Cachin, 1998). Security in this model is measured using the **Kullback–Leibler (KL) divergence** between the two distributions; **perfect security** is achieved when this divergence is zero, meaning a stego-object is statistically indistinguishable from an ordinary cover object even to an adversary with unlimited computational power and full knowledge of both distributions. This is a demanding theoretical benchmark rather than a routinely achieved practical target: real embedding algorithms are evaluated against it as an ideal, and most practical schemes settle for keeping this divergence small rather than provably zero, particularly at non-trivial embedding rates.

5.3 The Steganographic Triangle and the Security/Secrecy Distinction

Practical steganographic systems are shaped by a persistent, three-way trade-off often referred to as the **steganographic triangle**: embedding **capacity** (how much data can be hidden), **imperceptibility** (how statistically and perceptually indistinguishable the stego-object is from a genuine cover), and **robustness** (whether the hidden message survives further processing of the carrier, such as compression or resizing). Increasing capacity tends to make embedding changes more detectable, whether to a human observer or to a statistical test, and neither capacity nor imperceptibility can generally be increased without affecting how well the hidden message tolerates further transformation of the carrier. Different applications sit at different points on this triangle: a covert communication channel between two parties might prioritise imperceptibility and accept a lower payload, while an application more tolerant of detection risk might push for higher capacity instead.

It is also useful to distinguish, within the general notion of "security," between preventing *detection* (an observer cannot tell a stego-object from a cover object) and preventing *extraction* (even if detected, the actual message content cannot be recovered without the key). A scheme can in principle fail at the first while succeeding at the second — for example, if a statistical test flags an object as suspicious, but the payload itself remains encrypted and unrecoverable without a

key. This distinction is a conceptually useful separation for evaluating exactly what a given steganographic system protects against, though it should be treated as an analytical framing rather than a claim that the field uniformly adopts this precise terminology [SOURCE TO VERIFY].

6. Types of Steganography

Steganography can be applied to any digital medium with sufficient redundancy — meaning more capacity to represent information than is strictly required to convey its apparent content. The major carrier types are as follows.

Image steganography is the most widely used form, owing to the high digital redundancy of images; typical approaches hide information by manipulating pixel intensities directly, or by manipulating the coefficients produced when an image is transformed into a frequency representation (see Section 7).

Audio steganography exploits limitations of human hearing. Reported techniques include phase coding, which embeds data in phase shifts that are inaudible to human listeners, and echo data hiding, which embeds bits by introducing small, controlled echo delays. These specific technique descriptions are drawn from the review material available for this synthesis rather than independently verified against a primary source in this revision and should be treated as indicative rather than definitive [SOURCE TO VERIFY].

Video steganography benefits from very high available capacity and can mask embedded data using temporal dependencies between frames, spreading changes across many frames rather than concentrating them in one image. Reported approaches include modifying motion vectors or intra-frame prediction modes, both internal components of how video compression represents movement and structure between frames [SOURCE TO VERIFY].

Text (linguistic) steganography historically relied on format-level changes — such as whitespace or tab patterns — that are largely invisible to a casual reader. A more recent generative variant uses language models to synthesise the carrier text itself around the hidden payload; this is discussed in more technical detail in Section 9.3, since it is best understood alongside the broader modification-versus-generation distinction introduced there, rather than as a separate historical technique.

7. Technical Methods

7.1 Spatial-Domain Techniques and LSB Substitution

Spatial-domain methods operate directly on a carrier's raw values — for an image, the pixel values themselves. The most common spatial-domain method is **Least Significant Bit (LSB) substitution**, which replaces the lowest-order bit of each cover element (for example, each colour channel of a pixel) with a bit from the secret message. Because a pixel's brightness or colour value is stored as a binary number, and the least significant bit contributes the smallest possible amount to that value, changing it alters the pixel's actual numeric value by at most one unit out of, typically,

256 possible levels — a change generally imperceptible to the human eye. This is why LSB substitution can carry a substantial payload with little visible distortion.

However, this same mechanism is also the source of the method's principal weakness: because the least significant bits of a natural image are not perfectly random to begin with, uniformly replacing them with message bits (which, if the message is compressed or encrypted, will look close to random) disturbs the statistical relationships between neighbouring pixel values in predictable ways. Early detection work demonstrated that this disturbance is directly measurable — for instance, using pair-based statistical tests that compare the frequency of specific value pairs before and after embedding (Fridrich, Goljan, & Du, 2001, as cited in later steganalysis literature reviewed for this paper). This is why naive LSB substitution, despite its simplicity and capacity advantages, is considered highly vulnerable to targeted statistical analysis (see Section 8).

7.2 Transform-Domain Techniques: DCT and DWT

Transform-domain methods take a different approach: rather than modifying raw pixel or sample values directly, they first convert the carrier into a set of **frequency coefficients** and embed the message by altering those coefficients. The rationale is that certain frequency components of an image or audio signal are less perceptually important than others, so embedding in the "right" coefficients can survive processes — most importantly compression — that would disrupt a naive spatial-domain embedding.

The **Discrete Cosine Transform (DCT)** underlies JPEG steganography and is significant precisely because DCT is also the transform used internally by the JPEG compression standard. JPEG-domain steganography works by dividing the image into 8×8 blocks and embedding data in the quantised AC coefficients produced by JPEG's own compression pipeline. Because the message is embedded in coefficients that are already part of the image's compressed representation, DCT-based steganography tends to be more robust than naive spatial-domain embedding against re-compression of the resulting file — the hidden data occupies a representation the carrier will typically retain anyway, rather than raw pixel values that a second compression pass would likely overwrite.

The **Discrete Wavelet Transform (DWT)** offers a different set of properties: it provides a multi-resolution decomposition of the image, representing both coarse structure and fine detail across multiple scales rather than the single fixed 8×8 block size used by DCT. This multi-resolution structure loosely parallels how the human visual system processes both broad shapes and fine texture simultaneously, which is part of the rationale for DWT-based methods' reported robustness. It is worth being cautious here: "more robust" should not be read as "categorically superior" — robustness is only one vertex of the steganographic triangle (Section 5.3), and a scheme designed to maximise robustness may do so at the cost of capacity or imperceptibility. The choice between DCT and DWT, or between transform-domain and spatial-domain embedding generally, depends on which property (capacity, imperceptibility, or robustness) a given application actually needs most.

7.3 Distortion-Minimisation Frameworks: Cost Functions and Syndrome-Trellis Codes

A more principled alternative to simply choosing a domain (spatial or transform) is to explicitly optimise *where within* a cover object embedding changes are made. This is the **distortion-minimisation framework**, which separates the embedding problem into two distinct components: (1) a **cost function** that assigns a numerical distortion cost to changing each element of the cover, and (2) a **coding scheme** that embeds the message while provably minimising total cost, given the assigned costs.

Cost functions. The goal of a cost function is to assign high cost to changes in smooth, easily modelled regions (where any deviation is both visually and statistically obvious) and low cost to changes in complex, textured, or noisy regions (which are harder for both the eye and a statistical model to characterise precisely). One influential, domain-general cost function is **UNIWARD (Universal Wavelet Relative Distortion)**, proposed by Holub, Fridrich, and Denemark (2014), which computes distortion as the sum of relative changes in a directional wavelet filter-bank decomposition of the image; because the filter bank is directional (sensitive to different textures and edges in different orientations), the resulting cost function naturally concentrates permitted changes in textured regions while penalising changes to smooth areas or clean edges. A domain-specific variant, S-UNIWARD, applies this same principle to spatial-domain embedding. UNIWARD was notable for being explicitly designed to generalise across embedding domains (spatial, JPEG, and side-informed JPEG) using a single distortion design, rather than requiring a bespoke cost function for each domain.

Coding: Syndrome-Trellis Codes (STC). Assigning good costs is only half the problem; the embedder still needs a method to actually embed the message while achieving as close to the minimum possible total distortion as the assigned costs allow. **Syndrome-Trellis Codes**, introduced by Filler, Judas, and Fridrich (2011), provide a practical, near-optimal solution to this coding problem. STC frames embedding as a coding-theoretic task: given a cover object, a message, and a cost assigned to changing each element, the STC algorithm searches — efficiently, using a trellis (a graph-based structure borrowed from convolutional coding) — for the specific set of changes that embeds the full message at the lowest possible total cost. This decouples the design of *what makes a good change* (the cost function, e.g., UNIWARD) from *how to find the cheapest way to embed the message given those costs* (the coding scheme, STC), and this separation is what makes the distortion-minimisation framework practically usable: a designer can improve the cost function independently of the coding method, and STC will still find a near-optimal set of changes for whatever costs it is given. This is a meaningfully more principled approach than either naive LSB substitution (which embeds without regard to local cost at all) or fixed-domain methods like plain DCT embedding (which embed in a chosen domain but do not explicitly optimise placement within it).

8. Steganalysis and Detection

Steganalysis is the counterpart discipline to steganography: the study of detecting, and where possible extracting, hidden messages.

8.1 Targeted vs. Universal Steganalysis

A basic distinction in steganalysis is between targeted and universal (or "blind") approaches. Targeted steganalysis exploits specific, known vulnerabilities of a particular embedding algorithm — for example, the pair-based statistical anomalies left by naive LSB substitution — and can be highly effective against that algorithm but generalises poorly to others. Universal steganalysis instead trains a statistical or machine-learning classifier to distinguish cover objects from stego-objects based on general features, without being tailored to one specific embedding method, trading some algorithm-specific sensitivity for broader applicability.

8.2 Rich Models and Ensemble Classifiers

One influential family of universal, feature-based detection tools is the **Spatial Rich Model (SRM)**, introduced by Fridrich and Kodovský (2012). SRM is a *feature-extraction* method, not a complete classifier in itself: it computes a large, diverse set of noise-residual features by applying many different high-pass filters to an image and modelling the resulting neighbouring-pixel dependencies, on the reasoning that embedding — even when visually imperceptible — disturbs local statistical relationships that these filters are designed to expose. Because the resulting feature space is very high-dimensional, SRM was designed from the outset to be paired with an **ensemble classifier** (Kodovský, Fridrich, & Holub, 2012) — a computationally efficient classification method well suited to high-dimensional feature spaces and large training sets — rather than with a single monolithic model. Presenting SRM as if it were, by itself, a complete detector is a simplification worth correcting: the detection pipeline is properly understood as SRM (feature extraction) plus an ensemble classifier (the actual decision-making step).

8.3 Deep-Learning-Based Detection

More recently, convolutional neural networks (CNNs) have been applied directly to steganalysis, learning discriminative features from labelled cover/stego image pairs rather than relying on a human-designed filter bank. A notable architecture in this line of work is **SRNet**, proposed by Boroumand, Chen, and Fridrich (2019), a deep residual network that learns noise-residual features end-to-end without requiring the handcrafted high-pass pre-filters used in SRM. This represents a genuine shift in methodology — from manually engineered statistical features to features discovered by the learning process itself — but it is worth noting that SRNet is one representative architecture among several CNN-based steganalysis proposals developed over roughly the same period, rather than a single, unique solution defining the entire subfield; other architectures pursued similar goals with different design choices. It is also worth noting, in fairness to both approaches, that rich-model-plus-ensemble pipelines remain a meaningful comparison baseline in the literature rather than an approach that CNN-based methods have simply rendered obsolete in every setting; how well a given deep-learning detector generalises to cover data unlike its training data (see Section 11.1) is itself an active concern, not a solved question.

9. Modern Developments

9.1 Deep Learning's Dual Role

Deep learning has influenced steganography on both sides of the adversarial relationship described in Section 5.1: it has reshaped steganalysis, as discussed in Section 8.3, and it has also influenced steganographic *embedding* itself, through learned cost functions, generative embedding schemes, and coverless approaches (discussed below). This dual influence sets up an ongoing methodological contest: as detection methods improve, embedding methods adapt, and vice versa — a dynamic consistent with the adversarial framing of the Prisoners' Problem itself (Section 5.1), now played out with learned rather than purely handcrafted strategies on both sides.

9.2 Generative Steganography

One significant shift in the field is the move from *modification-based* approaches (which start with an existing cover object and alter it, as in Sections 7.1–7.3) to *generation-based* approaches, which synthesise an entirely new piece of media directly from the secret bitstream, rather than starting from a pre-existing cover. Because there is no original, unmodified object being altered, this sidesteps a large part of what pixel-level and residual-based steganalysis (Section 8) are designed to detect — those methods rely, at least implicitly, on comparing a suspected stego-object against what an unmodified cover of that kind "should" look like, and this comparison is less straightforward when no such original cover ever existed.

This advantage comes with real costs that should be weighed against it rather than treated as a one-directional improvement. Some treatments of generative steganography frame its design space as involving a tension between achieving high embedding capacity, achieving low computational complexity (ideally embedding and extraction that scale efficiently with message length), and achieving strong statistical security — with the suggestion that methods optimised for one of these (for example, entropy- or arithmetic-coding-based approaches that achieve strong theoretical capacity) may do so at some cost to the other two. This three-way framing is a useful way to think about the design space, but it should be read as an analytical lens drawn from specific treatments of generative steganography rather than as a universally agreed, field-wide taxonomy; this review could not verify it as an established consensus position within its scope, and it is marked here accordingly [SOURCE TO VERIFY].

9.3 Generative Linguistic Steganography

Within text steganography specifically, the generation-based paradigm takes the form of using large language models to generate fluent carrier text whose word choices are steered, bit by bit, by the secret message being encoded — rather than modifying the formatting of pre-existing text, as in traditional linguistic steganography (Section 6). The appeal is similar to generative image steganography: since the resulting text is generated rather than edited, it lacks the trace of an "original" unmodified version that many detection strategies implicitly rely on. The corresponding cost is a real one: the more tightly the message constrains word choice, the more the model's output can depart from what a human would naturally write, creating a trade-off between payload size and text fluency that mirrors the

capacity–imperceptibility trade-off discussed in Section 5.3. Specific reported implementations and their performance characteristics are drawn from the review material available for this synthesis and are marked for further verification rather than presented as independently confirmed findings [SOURCE TO VERIFY].

9.4 Coverless Steganography

Coverless steganography is a more radical departure still: rather than embedding a message into a cover object by modification or generation, it hides information without altering the carrier at all, by establishing a mapping between the secret data and features of existing, unmodified media — an approach sometimes described as deep semantic mapping. In practice, this typically works by extracting a high-dimensional feature representation (for example, using a pre-trained CNN) from a large database of candidate cover objects, converting that representation into a hash or index, and then selecting or retrieving whichever existing objects' features correspond to the secret bits being sent — the sender and receiver share the same mapping scheme (and, in most designs, access to the same or a synchronised database), so the receiver can recover the message by extracting the same features from the received object(s) and reversing the mapping.

Because no modification occurs, coverless approaches are attractive from a detection standpoint: pixel-level and residual-based steganalysis techniques (Section 8), which are designed to detect the trace left by an embedding *operation*, have little to act on when no such operation took place. However, a recent survey of the field notes that this advantage comes with genuine practical costs. Early coverless image steganography methods reported limited capacity for transmitting bits compared with modification-based embedding, since the achievable payload is constrained by how many distinguishable images or features the mapping scheme and available database can support (Xiang, Tan, Qin, & Tan, 2025). The same survey also documents that many coverless designs, particularly those extended to video, incur substantial auxiliary overhead — the sender and receiver must maintain, synchronise, and search a large reference database, which is itself a meaningful computational and logistical cost rather than a free byproduct of "not modifying the cover" (Xiang, Tan, Qin, & Tan, 2025). Robustness against certain classes of distortion, particularly geometric transformations of images, has also been identified as a persistent weakness of several coverless schemes (Xiang, Tan, Qin, & Tan, 2025). Taken together, coverless steganography is best understood as a genuinely distinct technical strategy with a different risk profile — strong resistance to conventional modification-detecting steganalysis, traded against constraints on capacity, added infrastructure requirements, and, in several reported designs, weaker robustness to certain attacks — rather than as a strictly superior successor to modification-based or generative methods.

10. Applications

The literature reviewed for this paper does not provide substantial detail on deployed, real-world application domains for steganography, and this section is deliberately concise rather than padded with unverifiable examples. In general terms, the technique is relevant wherever covert communication or the discreet embedding of auxiliary information is desired, consistent with the adversarial framing of the

Prisoners' Problem (Section 5.1) from which the field's threat model derives. A fuller account of specific deployed applications, industries, or documented use cases would require sources beyond the scope of this review and is flagged here as an area for further research (Section 12) rather than asserted without support.

11. Limitations and Research Challenges

Several open challenges recur across both steganographic embedding and steganalysis.

11.1 Cover Source Mismatch (CSM)

Cover Source Mismatch refers to the degradation in steganalysis accuracy that occurs when the statistical properties of real-world cover data differ from those of the data a detector was trained on — for example, images from a different camera, processing pipeline, or resizing history than the training set. This is a well-documented problem in the steganalysis literature (Kodovský, Sedighi, & Fridrich, 2014) and is a particularly serious limitation for deep-learning-based detectors of the kind described in Section 8.3, since a model's learned features may reflect idiosyncrasies of its training distribution rather than universal signatures of embedding. This is not a deep-learning-specific problem in origin — mismatch between training and deployment cover sources was recognised as a challenge for statistical, rich-model-based steganalysis as well — but the reliance of deep learning on large, representative training sets arguably makes the issue more consequential in practice.

11.2 Real-Time Detection

Detecting hidden information in high-bandwidth, low-latency communication — such as VoIP streams or live video — remains difficult, since the computational cost of thorough statistical or deep-learning-based analysis is in tension with the speed required to process a continuous, live data stream. This is consistent with the general difficulty of applying computationally intensive detection pipelines (rich models plus ensemble classifiers, or deep residual networks) under strict latency constraints, though this review was not able to verify specific quantitative benchmarks for real-time steganalysis performance and marks this claim for further verification [SOURCE TO VERIFY].

11.3 Lack of Standardisation

There is a recognised lack of unified taxonomy and standardised evaluation frameworks for comparing the growing diversity of generative and coverless approaches (Section 9) against one another and against traditional modification-based methods. Without agreed benchmarks and shared terminology, comparative claims about which techniques "outperform" others are difficult to evaluate rigorously — a caution that applies to claims made in the broader literature on this topic, and one this paper has tried to observe by attributing specific performance claims to their sources rather than asserting them as general truths.

11.4 Active Adversaries

Most information-theoretic treatments of steganographic security, including Cachin's model (Section 5.2), assume a **passive warden**: an observer who monitors communication but does not interfere with it. Resilience against an *active*adversary — one who can perturb, inject noise into, or otherwise manipulate the communication channel, not merely observe it — is comparatively underexplored, particularly in high-capacity embedding settings. This represents a meaningful gap between the dominant theoretical security model and a more demanding, and arguably more realistic, threat model in adversarial network settings.

12. Future Research Directions

The following directions follow specifically from the steganography- and steganalysis-specific gaps identified in Section 11, rather than from generic observations about machine learning that would apply equally to unrelated fields.

Addressing **Cover Source Mismatch** (Section 11.1) specifically requires steganalysis methods that remain accurate when the training cover distribution and the deployment cover distribution diverge — for instance, detectors explicitly evaluated across multiple independent image sources, processing pipelines, or acquisition devices, rather than solely on a single benchmark dataset such as those historically used in the SRM and SRNet literature (Sections 8.2–8.3). Progress on **real-time detection** (Section 11.2) would require detection pipelines — whether rich-model-based or deep-learning-based — explicitly optimised for the latency constraints of streaming media such as VoIP or live video, which is a different design target from the accuracy-first, offline-analysis setting in which most current detectors (including SRM and SRNet) were developed and evaluated. The absence of **standardised evaluation frameworks** (Section 11.3) is a particularly acute gap for generative and coverless methods (Section 9), where capacity, robustness, and detectability are measured inconsistently across studies; developing shared benchmarks that allow direct comparison between modification-based, generative, and coverless approaches under matched conditions (same payload size, same cover source, same detector) would materially improve the field's ability to make defensible comparative claims. Finally, extending steganographic security models beyond the **passive-warden assumption** (Section 11.4) — for example, by formally analysing security against an adversary permitted to add noise or otherwise perturb the channel, building on the passive-warden foundation laid by Cachin's model (Section 5.2) — is a specific theoretical extension motivated directly by the gap between current security models and increasingly adversarial real-world network conditions, rather than a general call for "more robust methods" untethered to steganography's particular threat model. These directions are offered as reasonable extensions of the specific gaps identified above, not as descriptions of confirmed ongoing research programmes.

13. Conclusion

Steganography occupies a distinct position among information-security disciplines: where cryptography protects the content of a message and watermarking protects the integrity of an embedded signal, steganography protects the fact that a message exists at all (Section 4). This paper has traced the field from its foundational theoretical models — the Prisoners' Problem (Simmons, 1984) and Cachin's KL-

divergence-based security framework (Cachin, 1998) — through the practical trade-offs captured by the steganographic triangle, to concrete embedding methods in the spatial and transform domains, the more principled distortion-minimisation framework built on explicit cost functions such as UNIWARD and near-optimal coding via Syndrome-Trellis Codes (Filler, Judas, & Fridrich, 2011; Holub, Fridrich, & Denemark, 2014), and the corresponding detection techniques developed in steganalysis, from rich models paired with ensemble classifiers (Fridrich & Kodovský, 2012; Kodovský, Fridrich, & Holub, 2012) to end-to-end deep residual networks such as SRNet (Boroumand, Chen, & Fridrich, 2019).

The recent influence of deep learning is evident on both sides of this adversarial relationship: embedding techniques have moved toward generative and coverless paradigms that sidestep some traditional, modification-based detection strategies, while steganalysis has moved toward learned, end-to-end architectures. Neither shift should be read as having settled the field's underlying tensions. Generative and coverless methods trade the advantage of evading modification-based detection for real costs in capacity, computational overhead, database dependence, or robustness to certain distortions (Sections 9.2–9.4); deep-learning-based detectors remain vulnerable to Cover Source Mismatch in ways that are not yet fully resolved (Section 11.1). Capacity, imperceptibility, robustness, security, and — in the generative setting — computational efficiency remain in genuine competition, and open challenges around distributional mismatch, real-time analysis, evaluation standardisation, and active adversaries indicate that steganography remains an active area of research with unresolved theoretical and practical questions, rather than a technically settled problem.

References

Boroumand, M., Chen, M., & Fridrich, J. (2019). Deep residual network for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 14(5), 1181–1193.

Cachin, C. (1998). An information-theoretic model for steganography. In *Information Hiding: 2nd International Workshop*, Lecture Notes in Computer Science, Vol. 1525 (pp. 306–318). Springer.

Filler, T., Judas, J., & Fridrich, J. (2011). Minimizing additive distortion in steganography using syndrome-trellis codes. *IEEE Transactions on Information Forensics and Security*, 6(3), 920–935.

Fridrich, J., & Kodovský, J. (2012). Rich models for steganalysis of digital images. *IEEE Transactions on Information Forensics and Security*, 7(3), 868–882.

Holub, V., Fridrich, J., & Denemark, T. (2014). Universal distortion function for steganography in an arbitrary domain. *EURASIP Journal on Information Security*, 2014, Article 1.

Kodovský, J., Fridrich, J., & Holub, V. (2012). Ensemble classifiers for steganalysis of digital media. *IEEE Transactions on Information Forensics and Security*, 7(2), 432–444.

Kodovský, J., Sedighi, V., & Fridrich, J. (2014). Study of cover source mismatch in steganalysis and ways to mitigate its impact. In *Proceedings of SPIE: Media Watermarking, Security, and Forensics*, Vol. 9028.

Simmons, G. J. (1984). The prisoners' problem and the subliminal channel. In D. Chaum (Ed.), *Advances in Cryptology: Proceedings of CRYPTO '83* (pp. 51–67). Plenum Press.

Xiang, X., Tan, Y., Qin, J., & Tan, Y. (2025). Advancements and challenges in coverless image steganography: A survey. *Signal Processing*, 228, Article 109761.

Additional claims marked [SOURCE TO VERIFY] throughout this paper originate from the review material available for this synthesis but could not be independently traced to a specific, citable primary source within the scope of this revision. They are retained because they are plausible and consistent with the broader literature, but they should not be treated as verified academic findings until a specific source is located and confirmed.

- *Human Review:*
  - I need to review the verbosity, facts, citations and sources.
- *Failure points:*
  - The initial synthesis did not provide usable resources.
  - The sources are not properly cited in the first draft.
  - Some research gaps might stem from hallucinations.
  - Integration was simplified.
  - AI mixed the source types in gathering.
- Time Comparison:
  - Manual: 8+ hours
  - AI: 1.5 hours
  - Improvement: 81% reduction in active point

Run 5:

- *Topic:* Understanding the working and architecture UAVs
- *Gather (NotebookLM):*
    - The prompt given is: Research Task: Understanding the Working and Architecture of UAVs. I am researching Unmanned Aerial Vehicles (UAVs) as a potential topic for my Final Year Project (FYP). I want to understand how UAVs work from both a conceptual and technical perspective before deciding on a specific research or implementation direction. Conduct a comprehensive academic and technical literature search using reliable, authoritative sources, prioritising:

        - Peer-reviewed research papers
        - IEEE and other reputable engineering publications
        - University research
        - Government/aviation authorities
        - Established aerospace and robotics sources
        - High-quality technical documentation

        Avoid relying primarily on blogs, commercial marketing pages, or unsourced summaries.

        Research the following:

        1. What is a UAV?

        - Definition of UAV/UAS
        - Difference between UAV, UAS, drone, RPAS and autonomous aerial vehicle
        - Main categories and classifications of UAVs

        2. How does a UAV work?
        Explain the complete working process from:

        - User/mission commands
        - Flight controller
        - Sensor input
        - State estimation
        - Control algorithms
        - Motor/actuator commands
        - Propulsion
        - Aircraft movement

        Explain how these components interact as a complete feedback-control loop.

        3. Main UAV components
        Research the purpose and operation of:

        - Airframe

- Motors
- Propellers
- Electronic Speed Controllers (ESCs)
- Flight controller
- Battery and power-management system
- GPS/GNSS
- IMU
- Accelerometer
- Gyroscope
- Magnetometer
- Barometer
- Cameras and other payload sensors
- Communication/data links
- Ground control station

4. Flight control
Explain how a UAV controls:

- Thrust
- Roll
- Pitch
- Yaw
- Altitude
- Position
- Stability

Explain the role of PID control and other important control approaches used in UAVs.

5. Navigation and localisation
Research:

- GPS/GNSS
- IMU-based navigation
- Sensor fusion
- Kalman filters / Extended Kalman Filters
- Visual odometry
- SLAM
- GPS-denied navigation

Explain why UAVs need multiple sensors rather than relying on a single navigation system.

6. Communication
Explain how UAVs communicate with:

- Ground control stations
- Remote controllers
- Satellites/GNSS
- Other UAVs where applicable

Research communication protocols, telemetry, command-and-control links, latency, range and reliability.

7. Autonomous UAVs
Investigate the difference between:

- Remotely piloted UAVs
- Semi-autonomous UAVs
- Fully autonomous UAVs

Explain how AI, computer vision, path planning and reinforcement learning can contribute to UAV autonomy.

8. UAV software architecture
Research commonly used UAV software ecosystems and architectures, including where relevant:

- PX4
- ArduPilot
- MAVLink
- ROS/ROS 2
- Ground Control Stations

Explain how these systems communicate and interact rather than simply listing them.

9. Types of UAVs
Compare:

- Multirotor
- Fixed-wing
- Hybrid VTOL

Explain the advantages, disadvantages, endurance, manoeuvrability, payload capacity and typical applications of each.

10. Applications
Research major UAV applications such as:

- Search and rescue
- Disaster management
- Agriculture
- Infrastructure inspection
- Mapping and surveying
- Environmental monitoring
- Delivery/logistics
- Defence/security

Focus on technically meaningful applications rather than simply listing industries.

11. Current challenges
Research major technical problems including:

- Limited battery endurance
- Payload constraints
- Navigation uncertainty
- GPS-denied environments
- Obstacle avoidance
- Autonomous decision-making
- Communication loss
- Cybersecurity
- Weather/environmental effects
- Collision avoidance
- Regulatory constraints

12. AI and UAV research
Since this research may inform an AI/Computer Science FYP, investigate current research involving:

- Computer vision
- Object detection
- Autonomous navigation
- Path planning
- Obstacle avoidance
- Reinforcement learning
- Multi-UAV coordination
- Edge AI
- Vision-based control

Identify potential research gaps that could realistically become a Final Year Project.

Source requirements

For each important claim, identify the relevant source and provide:

- Author(s)
- Year
- Title
- Journal/conference/organisation
- DOI or stable link where available
- A short explanation of what the source contributes

Prioritise recent research from approximately 2020–2026, while also including important foundational sources where necessary.

Do not invent citations or fill missing bibliographic information from assumptions.

Final research output

Organise the gathered research into:

A. Foundational UAV concepts
B. UAV hardware and components
C. Flight-control and feedback systems
D. Navigation and localisation
E. Communication and software architecture
F. Autonomous UAVs and AI
G. UAV types and comparison
H. Applications
I. Current technical challenges
J. Potential FYP research directions
K. 15–25 strongest academic/technical sources

At this stage, do not write the final paper or synthesise the research into an argument. The purpose of this stage is Gathering: collect reliable evidence and establish a strong research base that can be synthesised in the next stage.

- The output is:

This is an impressively thorough research plan! Covering everything from PID control loops and SLAM to ROS 2 and Edge AI will give your Final Year Project a very strong foundation.

To provide the **15–25 academic sources** with specific DOIs and authors you requested, we first need to populate your notebook with those high-quality papers. I have just initiated a **Fast Research** search for recent (2020–2026) IEEE and engineering publications covering your specific technical points (PID, Kalman Filters, and software architectures).

Check the "Source Panel" on the left—once the results appear, you can click "Import" on the ones that look most relevant to your FYP. Once they are added to the notebook, I can synthesize them into the organized sections (A-K) you've outlined.

Because your requirements are so specific regarding academic rigor, would you like me to also start a **Deep Research** session? It takes about 3-5 minutes but will perform a much more comprehensive crawl to find those specific DOIs and peer-reviewed studies you need.

- *Synthesise (NotebookLM):*
  - The prompt is: Using only the academic and technical sources gathered in this research notebook, synthesise the literature into a coherent technical foundation for my FYP on how a modern UAV works as an integrated autonomous system. Do not write the final paper yet. Do not invent information, citations, authors, DOIs, results, or specifications. Cover:

1. UAV architecture — airframe, motors, ESCs, propellers, battery, flight controller, sensors, communications, and the distinction between flight control and higher-level autonomy.
2. Flight control — roll, pitch, yaw, thrust, PID feedback, limitations, and advanced control methods.
3. Sensors & state estimation — IMU, accelerometer, gyroscope, magnetometer, barometer, GPS/GNSS, cameras, sensor fusion, and Kalman Filter/EKF/UKF.
4. Localisation & mapping — SLAM, visual SLAM, visual-inertial navigation, assumptions, strengths and limitations.
5. Navigation — global/local path planning, obstacle avoidance, and algorithms such as A*, RRT/RRT* or potential fields where supported.
6. Computer vision & perception — detection, tracking, obstacle perception, deep-learning models such as CNN/YOLO where supported, and onboard computational challenges.
7. Edge AI & onboard computing — why onboard inference matters, latency, bandwidth, energy, reliability, privacy, hardware acceleration, and accuracy-vs-efficiency trade-offs.
8. Software architecture & ROS 2 — nodes, topics, services/actions, middleware, integration of perception/localisation/planning/control, and resource limitations.
9. Communication, safety & reliability — latency, packet loss, bandwidth, connection loss, fail-safes, cybersecurity and reliability concerns where supported.
10. System integration — explain the complete pipeline: Sensors → State Estimation → Perception → Localisation/Mapping → Path Planning → Control → Actuation, including real-time information flow and different processing frequencies where supported.
11. Current trends and research gaps — autonomous navigation, visual-inertial systems, SLAM, deep-learning perception, Edge AI, ROS 2, lightweight models, GPS-denied operation, computational/energy limits, real-time processing, localisation drift, safety, communication and sim-to-real issues.
12. Critical synthesis — compare approaches rather than summarising papers individually. Identify agreements, disagreements, and trade-offs between accuracy, latency, computational cost, energy, robustness and autonomy.
13. FYP takeaways — identify foundational concepts, practical technologies, advanced techniques, and potential research questions.

    For major claims, provide author/year and DOI where available. Clearly distinguish peer-reviewed papers, surveys, technical documentation and other sources. Never fabricate missing bibliographic information. End with:

    - Key Findings
    - Areas of Agreement
    - Contradictions/Trade-offs
    - Research Gaps
    - Most Important Sources

        - Concepts Requiring Further Research
        - Keep it technically detailed but understandable to a Computer Science/AI student.
    - The output is:

A modern **Unmanned Aircraft System (UAS)** is an integrated network composed of the **Unmanned Aerial Vehicle (UAV)**, a ground control station (GCS), a human operator, and the communication data links that bridge them. While the term "drone" is commonly used to refer to the aircraft, technical literature distinguishes the **UAV** as the physical flying machine comprising the airframe, propulsion system (motors, ESCs, propellers), and onboard avionics.

UAV Architecture and Hardware

The physical performance of a UAV is dictated by **SWaP-C** (Size, Weight, Power, and Cost) considerations.

- **Propulsion:** Most systems utilize **Brushless DC (BLDC) motors** due to their high speed-to-weight ratio and efficiency. These are controlled by **Electronic Speed Controllers (ESCs)**, which regulate the voltage and current to adjust motor speed and thrust.
- **Flight Controller (FC):** Acting as the "brain," the FC accepts remote commands and interfaces with sensors to manage stability and flight parameters.
- **Onboard Computing:** Modern autonomous systems often split responsibilities between a **Flight Control Unit (FCU)** for low-level, high-rate stability tasks and a **Companion Computer** (e.g., NVIDIA Jetson, Raspberry Pi) for high-level autonomy like AI perception and path planning.

Flight Control and Feedback Systems

Autonomous flight relies on a **closed-loop feedback system** where the software estimates the drone’s state and adjusts motor outputs to minimize error.

- **PID Control:** The **Proportional-Integral-Derivative (PID)** controller is the industry standard for attitude and rate control. While simple to tune, it is a linear method that may show degraded performance in nonlinear operating points.
- **Advanced Methods: Sliding Mode Control (SMC)** offers faster response and better disturbance rejection than PID but suffers from **chattering** (high-frequency actuator oscillations). **Model Predictive Control (MPC)** is increasingly used for trajectory tracking as it iteratively predicts future system outputs using cost functions, though it requires significantly more computational power than PID.

Sensors and State Estimation

Accurate navigation requires blending data from multiple sensors to overcome individual limitations.

- **Primary Sensors:** An **Inertial Measurement Unit (IMU)** provides high-rate (100–1000 Hz) data on acceleration and rotation but suffers from **drift** over time. **GNSS/GPS** provides absolute positioning at lower rates (1–10 Hz) to anchor the system and reset IMU errors.
- **Fusion Algorithms:** The **Extended Kalman Filter (EKF)** is the traditional workhorse for nonlinear state estimation. However, recent research explores **Transformer-based fusion architectures** (inspired by Perceiver IO), which can outperform EKFs in "urban canyons" by dynamically weighting sensors like LiDAR and cameras when GPS signals are degraded.

Localization, SLAM, and Mapping

In GPS-denied environments, UAVs must use **Simultaneous Localization and Mapping (SLAM)** to build a map of their surroundings while tracking their own position.

- **Visual SLAM (vSLAM):** Algorithms like **ORB-SLAM** use camera features to reconstruct 3D structures but can lose track during swift maneuvers or in low-texture environments.
- **LiDAR SLAM:** Algorithms like **LOAM** or **LeGO-LOAM** use laser pulses to create high-density 3D maps and are generally more robust to lighting changes than vSLAM.
- **Active SLAM:** This advanced approach integrates path planning into the SLAM loop, allowing the robot to autonomously choose trajectories that minimize map uncertainty.

Navigation and Path Planning

Autonomous navigation involves a hierarchy of planning layers.

- **Global Planning:** Determines the overall route to a goal.
- **Local Planning:** Handles obstacle avoidance in real-time. The **Model Predictive Path Integral (MPPI)** controller is a complex version of MPC that uses sampling-based methods to choose optimal trajectories even with non-convex objective functions.
- **Obstacle Avoidance:** Modern stacks utilize **Computer Vision** (e.g., YOLO for object detection) paired with depth sensors to identify and navigate around dynamic barriers.

Edge AI and Software Architecture

The requirement for **low latency** and **privacy** drives the need for onboard **Edge AI** rather than cloud-based processing.

- **ROS 2:** The **Robot Operating System 2** provides a modular middleware for UAV software, using nodes, topics, and services to integrate perception, planning, and control.
- **Autopilot Ecosystems: PX4** and **ArduPilot** provide full-featured flight stacks, while **ROSflight** offers a leaner, researcher-focused alternative that moves most autonomy logic to a Linux-based companion computer.

- **Integration Bridges:** Tools like **uXRCE-DDS** act as a bridge between the high-level ROS 2 environment and the low-level PX4 firmware.

System Integration: The Autonomous Pipeline

A fully integrated autonomous UAV operates through a continuous execution pipeline:

1. **Sensors:** Collect raw data (IMU, GPS, LiDAR, Vision).
2. **State Estimation:** Fuse data (EKF/Transformer) to determine pose and velocity.
3. **Perception & Mapping:** Build environmental representation (SLAM) and detect obstacles (YOLO).
4. **Path Planning:** Compute a safe trajectory (Nav2/MPPI).
5. **Control:** Generate torque/thrust commands (PID/MPC).
6. **Actuation:** The **Motor Mixer** translates torque commands into PWM signals for the **ESCs**, which drive the **Motors**.

Key Findings

- **UAS vs. UAV:** Professional research must treat the UAV as just one part of a multi-component **system** (UAS).
- **Hybrid Architectures:** Successful autonomous UAVs typically split tasks between an **FCU** (for safety-critical flight) and a **Companion Computer** (for AI-heavy logic).
- **Onboard Edge AI:** Offloading compute to the edge (onboard or local machine) is critical for reducing the latency that plagues cloud-based UAV control.

Areas of Agreement

- **Multi-Sensor Necessity:** No single sensor is sufficient; fusion (e.g., IMU+GPS+Vision) is mandatory for robust navigation.
- **IMU Drift:** All sources acknowledge that without external correction (GPS or SLAM), IMU-based dead reckoning will fail quickly due to accumulative drift.
- **ROS 2 as a Meta-OS:** There is a broad consensus that ROS 2 is the standard for building modular, distributed robotics applications.

Contradictions/Trade-offs

- **PID vs. SMC:** PID is reliable and stable but slow; SMC is faster but induces "chattering" that stresses motors.
- **Accuracy vs. Compute:** MPC and Graph-based SLAM provide superior accuracy but are often too computationally heavy for small onboard microcontrollers, requiring expensive companion computers or edge offloading.

- **LiDAR vs. Vision:** LiDAR is highly accurate and light-independent but expensive and power-hungry; Vision is cheaper but struggles in low light and uniform textures.

Research Gaps

- **Long-term SLAM Stability:** Current open-source SLAM algorithms lack proven reliability for extremely long-duration autonomous missions in military or dynamic environments.
- **AI Certification:** There is a lack of established standards for certifying deep-learning models (like Transformer-based fusion) for safety-critical civilian airspace.
- **Swarm Coordination:** While individual UAV autonomy is maturing, real-time multi-agent cooperation via edge computing remains a nascent field.

Most Important Sources

- **Achilleas Santi Seisa et al. (2021/arXiv):** Detailed MPC and Edge Computing architectures.
- **Yash K Shah (2025/Preprint):** Advanced Transformer-based sensor fusion for UAM.
- **Keerthana Kannan et al. (NASA ARC/AFRC):** Performance comparisons of Visual and LiDAR SLAM in flight.
- **Davide Carminati (Politecnico di Torino):** Detailed PID vs. SMC inner/outer loop architectures.
- **Jacob Moore et al. (2025/arXiv):** ROSflight 2.0 and the lean autopilot design philosophy.

Concepts Requiring Further Research

- **Adaptive SMC:** Converting standard Sliding Mode Control into an adaptive system that accounts for payload variations.
- **Sim-to-Real Transition:** Reducing the effort needed to move AI models from Gazebo simulations to physical hardware.
- **Cybersecurity in UAS:** Protecting the sensitive data links and sensor inputs from spoofing or jamming attacks in autonomous missions.

- *Draft (Claude):*
    - The prompt is: Using the synthesised research and the academic/technical sources gathered in this research notebook, write a technical research paper on how modern autonomous UAVs work as integrated systems. The paper should be written for a Computer Science/AI student and should explain the technology clearly while maintaining academic and technical depth. Structure the paper around:

        1. Introduction and scope
        2. UAV/UAS fundamentals and system architecture
        3. Flight dynamics and control — including PID and advanced control approaches
        4. Sensors, sensor fusion and state estimation

5. Kalman Filtering — KF, EKF and UKF
6. Localisation, mapping and SLAM
7. Path planning and autonomous navigation
8. Computer vision and perception
9. Edge AI and onboard computing
10. UAV software architecture and ROS 2
11. Communication, safety and reliability
12. Integrated autonomous UAV pipeline
13. Current research trends
14. Limitations and research gaps
15. Critical comparison of approaches
16. FYP-relevant implications
17. Conclusion

Explain how the complete system works together, following the pipeline: Sensors → State Estimation → Perception/Mapping → Localisation → Path Planning → Control → Actuation Include appropriate technical comparisons, such as:

- PID vs advanced controllers
- IMU/GNSS/vision/LiDAR
- EKF vs UKF vs other approaches supported by the sources
- Visual SLAM vs LiDAR SLAM
- Global vs local path planning
- Cloud vs Edge AI
- traditional perception vs deep-learning-based perception
- PX4/ArduPilot/ROS 2 where supported

Do not simply summarise papers individually. Synthesize the literature, explaining agreements, disagreements, trade-offs and limitations. Citation requirements:

- Cite major technical claims using the actual sources in the research notebook.
- Use author/year citations where available.
- Include DOI information where available.
- Do not invent citations, authors, DOIs, results, specifications or claims.
- Clearly distinguish peer-reviewed research from preprints, surveys and technical documentation.
- If the available sources do not support a claim, omit it or explicitly identify it as requiring further research.

Pay particular attention to avoiding unsupported claims in the synthesis, including broad statements such as "ROS 2 is the standard," "PID is the industry standard," or claims that one sensor/algorithm is universally superior. The paper should be coherent rather than simply following the source order, use appropriate headings and transitions, and maintain a formal academic tone. Do not critique or revise the paper yet. This is strictly the DRAFT stage of the workflow.

- The output is:

How Modern Autonomous UAVs Work as Integrated Systems: A Technical Synthesis

Draft — for review

---

1. Introduction and Scope

Unmanned Aerial Vehicles (UAVs) are frequently discussed in popular usage as though the aircraft itself were the whole story. From an engineering and computer science standpoint, however, the aircraft is only one component of a larger **Unmanned Aircraft System (UAS)**, which additionally comprises a ground control station (GCS), a human operator, and the communication data links connecting them. This distinction — UAS as the system, UAV as the airframe — is not merely terminological; it shapes how responsibilities for safety, autonomy, and computation are distributed across the architecture.

This paper synthesises the research notebook compiled on autonomous UAV technology to explain how a modern autonomous UAV functions as an *integrated* system rather than a collection of independent subsystems. The scope covers hardware architecture, control theory, sensing and state estimation, localisation and mapping, path planning, perception, onboard computing, software middleware, and the safety/communication layer that binds them together. Where the underlying literature disagrees or leaves questions open, this is reported explicitly rather than resolved artificially. The paper is intended for a Computer Science/AI-oriented reader who requires both conceptual clarity and technical precision, and is written as the **draft stage** of a larger workflow; critique and revision are intentionally deferred.

---

2. UAV/UAS Fundamentals and System Architecture

A UAV's physical design is governed by **SWaP-C** constraints — Size, Weight, Power, and Cost — which trade off directly against performance, endurance, and computational capability. At the hardware level, thrust is generated by **Brushless DC (BLDC) motors**, selected for their favourable speed-to-weight ratio and efficiency, and regulated by **Electronic Speed Controllers (ESCs)** that adjust voltage and current to control motor speed and thrust.

Coordination of these actuators is the role of the **Flight Controller (FC)**, which accepts commands and interfaces with onboard sensors to maintain stability. As autonomy requirements have grown, a structural split has emerged between:

- A **Flight Control Unit (FCU)**, dedicated to low-level, high-rate stability and safety-critical control, and

- A **Companion Computer** (e.g. NVIDIA Jetson, Raspberry Pi), which handles higher-level autonomy tasks such as AI-based perception and path planning.

This FCU/companion-computer split is a recurring architectural theme across the literature: safety-critical, deterministic control is kept on a dedicated real-time unit, while computationally heavy and less time-critical reasoning is offloaded to a general-purpose processor. This separation of concerns is functionally analogous to a real-time operating system kernel handling scheduling guarantees while user-space processes handle higher-level logic — an analogy useful for CS students approaching UAV architecture for the first time.

---

3. Flight Dynamics and Control

Autonomous flight is fundamentally a **closed-loop feedback problem**: the system estimates its own state, compares it against a desired reference, and adjusts motor outputs to minimise the resulting error.

PID control. The Proportional–Integral–Derivative (PID) controller remains the most widely deployed method for attitude and rate control in the literature reviewed. Its appeal lies in simplicity of implementation and tuning, but it is a linear control method, and the notebook's synthesis notes degraded performance at nonlinear operating points — for example during aggressive manoeuvres or under significant external disturbance. It is worth being precise here: the sources support PID as *the most commonly used* method in practice, but the notebook does not provide evidence sufficient to call it a formally standardised requirement across the industry, so that stronger claim is avoided in this synthesis.

Sliding Mode Control (SMC). SMC is presented as offering faster response and improved disturbance rejection relative to PID (Carminati, Politecnico di Torino — source type not specified as peer-reviewed in the notebook and should be treated with appropriate caution). Its principal drawback is **chattering**: high-frequency oscillation of the control signal that translates into physical actuator stress and, potentially, premature mechanical wear.

Model Predictive Control (MPC). MPC is increasingly applied to trajectory tracking. It operates by iteratively predicting future system outputs against a cost function and choosing control actions accordingly. This predictive, optimisation-based approach comes at a computational cost substantially higher than PID, which is significant given the SWaP-C constraints discussed above — an onboard microcontroller sized for PID loops may not be capable of running MPC at the same rate without additional computing hardware (Seisa et al., 2021, arXiv preprint).

Comparative summary. PID, SMC, and MPC do not represent a strict quality ordering but a set of trade-offs along the axes of *responsiveness*, *robustness to nonlinearity/disturbance*, *actuator wear*, and *computational demand*:

Controller	Strength	Weakness
PID	Simple, low compute, well understood	Degrades in nonlinear regimes
SMC	Fast, robust to disturbance	Chattering, actuator stress
MPC	Handles trajectory optimisation well	High computational cost

The literature also identifies **Model Predictive Path Integral (MPPI)** control — a sampling-based variant of MPC capable of handling non-convex objective functions — as relevant primarily to the *local planning* layer rather than inner-loop attitude control; this is discussed further in Section 7.

---

4. Sensors, Sensor Fusion, and State Estimation

No single sensor provides both the accuracy and the update rate required for stable autonomous flight, which is why sensor fusion is treated in the literature as a necessity rather than an optimisation.

- The **Inertial Measurement Unit (IMU)** provides high-rate data (100–1000 Hz) on acceleration and angular rate, but its integration-based position estimates accumulate error over time — a phenomenon known as **drift**.
- **GNSS/GPS** provides absolute position at a much lower rate (1–10 Hz), which is used to anchor the IMU's relative estimate and periodically reset accumulated drift.

This is one of the clearest points of consensus across the notebook's sources: without an external correction signal — whether GPS or a SLAM-derived estimate — IMU-only dead reckoning degrades and will eventually fail. The practical implication is that GNSS-denied environments (discussed in Section 6) require a substitute correction source, typically vision or LiDAR.

Fusion algorithms. The Extended Kalman Filter (EKF) is described as the traditional workhorse of nonlinear state estimation (see Section 5). More recent research explores **Transformer-based fusion architectures**, inspired by the Perceiver IO architecture, which dynamically weight heterogeneous sensor inputs (e.g. LiDAR and camera) and are reported to outperform EKF-based approaches in "urban canyon" conditions where GPS signal quality is degraded (Shah, 2025, preprint). It is important to flag that this is preprint-stage research rather than a peer-reviewed, widely reproduced result, and should be read with corresponding caution regarding generalisability.

---

5. Kalman Filtering: KF, EKF, and UKF

The research notebook centres its discussion of state estimation on the **Extended Kalman Filter (EKF)**, describing it as the traditional method for nonlinear state estimation in UAV systems. The EKF operates by linearising the nonlinear system

model around the current state estimate at each timestep, which allows the standard linear Kalman Filter (KF) machinery to be applied approximately.

The notebook does not provide direct comparative data, performance figures, or citations specifically evaluating the **Unscented Kalman Filter (UKF)** against the EKF in a UAV context. Consequently, this paper does not assert a specific performance ordering between EKF and UKF; doing so would exceed what the source material supports. What can be said, drawing on general control-theoretic understanding consistent with the notebook's framing of EKF's linearisation limitation, is that the EKF's approximation via first-order linearisation is a known source of estimation error under strong nonlinearity — which is precisely the gap that the notebook identifies Transformer-based fusion as attempting to address in degraded-GPS conditions. A rigorous, sourced EKF-vs-UKF-vs-Transformer-fusion comparison for UAV state estimation is flagged here as an area requiring further research beyond what this notebook provides.

---

6. Localisation, Mapping, and SLAM

In environments where GNSS is unavailable or unreliable — indoors, in dense urban canyons, or under jamming — UAVs must rely on **Simultaneous Localization and Mapping (SLAM)** to build a map of the environment while concurrently estimating their own position within it.

- **Visual SLAM (vSLAM)**, exemplified by **ORB-SLAM**, extracts and tracks visual features from camera imagery to reconstruct 3D structure. Its principal weaknesses are loss of tracking during aggressive manoeuvres and failure in low-texture environments where distinctive visual features are sparse.
- **LiDAR SLAM**, exemplified by **LOAM** and **LeGO-LOAM**, uses laser range measurements to build high-density 3D maps and is described as generally more robust to lighting changes than vSLAM.
- **Active SLAM** extends both approaches by integrating path planning directly into the SLAM loop, allowing the vehicle to autonomously select trajectories that reduce map uncertainty rather than passively mapping whatever it happens to fly through.

Comparative trade-off. The notebook's synthesis frames this not as "LiDAR is better than vision" but as a cost/robustness trade-off: LiDAR sensors are accurate and largely independent of ambient lighting, but are more expensive and power-hungry — a meaningful constraint given SWaP-C limits (Section 2). Vision-based systems are cheaper and lighter but are comparatively fragile in low light and in visually uniform or texture-poor environments. Performance comparisons between visual and LiDAR SLAM specifically in flight are attributed in the notebook to NASA ARC/AFRC research (Kannan et al.), which should be read as institutional/technical-report-level evidence rather than necessarily peer-reviewed journal publication, absent further detail in the notebook.

---

7. Path Planning and Autonomous Navigation

Autonomous navigation is organised hierarchically:

1. **Global planning** determines the overall route toward a distant goal, typically over a known or previously mapped environment.
2. **Local planning** handles real-time obstacle avoidance and short-horizon trajectory adjustment. The **Model Predictive Path Integral (MPPI)** controller — a sampling-based extension of MPC — is highlighted as suited to this layer because it can optimise over non-convex objective functions, which arise naturally when obstacles create disjoint feasible regions in trajectory space.
3. **Obstacle avoidance** is increasingly implemented through computer-vision object detectors (e.g. YOLO) combined with depth sensing, to identify and react to dynamic obstacles that were not present in any prior map.

The distinction between global and local planning is not a matter of one being superior; they address different problems at different timescales — global planning assumes a relatively static, already-known environment, while local planning must react to obstacles and disturbances discovered in real time. Robust autonomy in the sources reviewed depends on both layers operating together rather than either in isolation.

---

8. Computer Vision and Perception

Perception in the reviewed literature spans two broad families of technique: geometric/classical computer vision (as used within vSLAM's feature-based tracking) and deep-learning-based object detection (e.g. YOLO-family models) used for dynamic obstacle and object recognition. The notebook indicates these are typically **paired rather than substituted for one another** — classical geometric vSLAM techniques handle structural mapping and self-localisation, while deep-learning detectors handle semantic recognition of specific object classes for avoidance and, potentially, mission-relevant target identification.

The notebook does not provide a direct, sourced comparison of classical versus deep-learning perception accuracy, latency, or robustness in a UAV-specific benchmark; therefore this paper does not make a claim of general superiority of one approach over the other, and instead notes that they serve distinct roles (structural mapping vs. semantic detection) within the same pipeline.

---

9. Edge AI and Onboard Computing

Latency and privacy considerations drive UAV systems toward **onboard ("edge") AI processing** rather than reliance on cloud-based computation. Round-tripping sensor data to a remote server and back introduces communication latency that is incompatible with the millisecond-scale control loops required for stable flight, and raises data-transmission and privacy concerns for sensitive missions.

This is why the companion-computer architecture (Section 2) exists: it allows AI-heavy perception and planning workloads to run locally, close to the sensors and control loop, at the cost of onboard SWaP-C budget rather than network dependency. The trade-off is not "cloud vs. edge" in the abstract but a direct consequence of the latency and connectivity constraints of an airborne, often communications-limited platform; the notebook does not provide quantitative latency figures comparing cloud and edge deployments specifically, so this remains a qualitative but well-supported architectural rationale rather than a benchmarked result.

---

10. UAV Software Architecture and ROS 2

The **Robot Operating System 2 (ROS 2)** provides modular middleware for integrating perception, planning, and control through a node/topic/service communication model. The notebook describes broad consensus among its sources that ROS 2 functions as a widely used foundation for building modular, distributed robotics software — a point of agreement rather than a specific technical claim about market dominance, and it is reported here at that level of confidence rather than as an absolute industry standard.

Two autopilot ecosystems are discussed:

- **PX4** and **ArduPilot** offer full-featured, integrated flight stacks.
- **ROSflight**, by contrast, is presented as a leaner, research-oriented alternative that deliberately moves most autonomy logic off the flight controller and onto a Linux-based companion computer (Moore et al., 2025, arXiv preprint, describing ROSflight 2.0).

Bridging tools such as **uXRCE-DDS** connect the high-level ROS 2 environment to low-level PX4 firmware, allowing companion-computer-hosted autonomy logic to exchange data with the safety-critical flight controller without requiring the FCU itself to run ROS 2.

PX4/ArduPilot vs. ROSflight trade-off. PX4 and ArduPilot provide more complete, production-oriented feature sets out of the box, which suits deployment-focused projects. ROSflight's leaner design, pushing autonomy logic to the companion computer, is positioned as more suited to research contexts where flexibility and transparency of the autonomy stack matter more than turnkey completeness. The notebook does not provide benchmarked performance data comparing these ecosystems directly, so this comparison should be read as an architectural-philosophy distinction rather than a quantitative one.

---

11. Communication, Safety, and Reliability

The UAS as a whole — not merely the UAV — depends on communication data links connecting the aircraft, ground control station, and operator. The notebook

identifies **cybersecurity of these data links and sensor inputs** — protection against spoofing or jamming — as an explicit research gap rather than a solved problem, which is a materially important caveat for any system intended to operate autonomously without constant human oversight. This paper does not speculate on specific mitigation techniques beyond what is in the notebook, since none are detailed there; it is flagged as requiring further research (Section 14).

---

12. Integrated Autonomous UAV Pipeline

Bringing the preceding sections together, a fully integrated autonomous UAV executes a continuous pipeline:

1. **Sensors** collect raw data from the IMU, GNSS, LiDAR, and vision systems (Section 4).
2. **State Estimation** fuses this raw data — via EKF or, in emerging research, Transformer-based fusion — to produce a pose and velocity estimate (Sections 4–5).
3. **Perception and Mapping** builds an environmental representation through SLAM and detects obstacles via deep-learning detectors such as YOLO (Sections 6, 8).
4. **Path Planning** computes a safe trajectory using global planning together with local, sampling-based methods such as MPPI (Section 7).
5. **Control** converts the planned trajectory into torque/thrust commands via PID, SMC, or MPC (Section 3).
6. **Actuation** — the **Motor Mixer** — translates these torque commands into PWM signals sent to the ESCs, which in turn drive the motors (Section 2).

This pipeline is not a one-directional pipeline in the strict software-engineering sense but a closed loop: actuation changes the vehicle's physical state, which is re-measured by sensors, re-estimated, re-planned, and re-controlled continuously at rates ranging from the sub-millisecond attitude loop up to the slower, second-scale replanning loop. The FCU/companion-computer split described in Section 2 maps directly onto this pipeline: the FCU typically owns state estimation (at least at the inner-loop level) and control/actuation, while the companion computer owns perception, mapping, and higher-level planning — with the two communicating over bridges such as uXRCE-DDS (Section 10).

---

13. Current Research Trends

Several trends recur across the notebook's synthesis of recent (2021–2025) sources:

- **Learning-based sensor fusion** (Transformer/Perceiver IO-inspired architectures) as a candidate replacement or supplement to EKF-based fusion in GPS-degraded conditions (Shah, 2025, preprint).

- **Edge computing architectures** explicitly designed around MPC-class controllers, addressing the tension between control sophistication and onboard compute limits (Seisa et al., 2021, arXiv).
- **Leaner, research-first autopilot stacks** (ROSflight 2.0) that push more autonomy logic to companion computers rather than embedding it in flight-controller firmware (Moore et al., 2025, arXiv).
- **Empirical, flight-tested comparison** of visual versus LiDAR SLAM approaches under institutional research programmes (Kannan et al., NASA ARC/AFRC).

A consistent thread across these trends is the movement of intelligence away from fixed, tightly-coupled flight-controller firmware and toward more flexible, learning-capable, and modular companion-computer software — while safety-critical, high-rate control remains comparatively conservative (PID/EKF-dominated) at the innermost loop.

---

14. Limitations and Research Gaps

Drawn directly from the notebook's own assessment, the following are identified as open problems rather than solved engineering questions:

- **Long-term SLAM stability.** Current open-source SLAM algorithms lack demonstrated reliability over extremely long-duration autonomous missions, particularly in military or highly dynamic environments.
- **AI certification.** No established standards currently exist for certifying deep-learning-based components (such as Transformer-based fusion) for use in safety-critical civilian airspace — a nontrivial barrier to regulatory deployment regardless of technical performance.
- **Swarm coordination.** While single-UAV autonomy is comparatively mature, real-time multi-agent coordination via edge computing is described as a nascent field with limited established results.
- **Cybersecurity of data links.** Spoofing and jamming resistance for sensitive data links and sensor inputs is flagged as unresolved.
- **EKF vs. UKF vs. learned fusion.** As noted in Section 5, the notebook does not provide a rigorous comparative evaluation; this remains an explicit gap in this synthesis rather than an assumed conclusion.

---

15. Critical Comparison of Approaches

Rather than repeating individual trade-offs already discussed, this section consolidates the recurring pattern across the paper: **almost every design choice in autonomous UAV systems is a trade-off between capability/accuracy and resource cost (compute, power, weight, or money), not a strict superiority relationship.**

Dimension	Lower-cost / lower-compute option	Higher-capability / higher-cost option	Nature of trade-off
Control	PID	SMC / MPC	Responsiveness and disturbance rejection vs. compute and actuator wear
State estimation	EKF	Transformer-based fusion	Established, lightweight vs. emerging, unproven at scale (preprint-stage)
Localisation	vSLAM	LiDAR SLAM	Cost/weight/power vs. lighting robustness and map density
Planning	Global-only	Global + local (MPPI)	Simplicity vs. real-time reactivity to dynamic obstacles
Compute	Cloud offload	Onboard edge AI	Bandwidth/power savings vs. latency and reliability
Autopilot stack	PX4/ArduPilot	ROSflight	Turnkey completeness vs. research flexibility

This framing matters pedagogically: a CS/AI student approaching UAV autonomy should not look for "the best algorithm" in each category, but should instead learn to reason about which constraints (SWaP-C, latency, environment, regulatory context) dominate a given application, since the literature synthesised here consistently supports context-dependent trade-offs rather than universal winners.

---

16. Final-Year-Project (FYP) Relevant Implications

For a student undertaking a Final Year Project in this space, several implications follow directly from the synthesis above:

- A project built on **PX4 or ArduPilot** benefits from mature tooling and community support but offers less flexibility for modifying core autonomy logic; a project built on **ROSflight** offers more room to experiment with custom estimation/control logic on the companion computer, at the cost of a less turnkey starting point (Moore et al., 2025).
- Given SWaP-C constraints, an FYP attempting real-time **MPC or MPPI** on constrained hardware should budget explicitly for companion-computer compute, rather than assuming a microcontroller-class flight controller can absorb the additional load (Seisa et al., 2021).
- Any FYP proposing **learned sensor fusion** (Transformer-based or otherwise) should be scoped with the understanding that this is preprint-stage, not yet an established or widely reproduced technique in the literature reviewed (Shah, 2025) — a reasonable and interesting research direction, but not one to present as settled practice.
- Comparative SLAM work (vSLAM vs. LiDAR SLAM) is a well-precedented FYP direction, with NASA ARC/AFRC-attributed research (Kannan et al.) offering a relevant point of comparison for methodology.

- Any project involving multi-UAV coordination should be scoped conservatively, given that swarm coordination via edge computing is explicitly identified as a nascent research area rather than an area with mature, off-the-shelf solutions.

---

17. Conclusion

A modern autonomous UAV is best understood not as a single intelligent aircraft but as a tightly coupled pipeline — sensors, state estimation, perception and mapping, path planning, control, and actuation — executing in a continuous closed loop, distributed across a safety-critical flight controller and a more flexible companion computer. The literature synthesised in this paper shows broad agreement on certain foundations (the necessity of multi-sensor fusion, the inevitability of IMU drift without external correction, and the practical utility of ROS 2 as a modular middleware) while showing genuine, unresolved trade-offs elsewhere (PID vs. SMC vs. MPC; vSLAM vs. LiDAR SLAM; PX4/ArduPilot vs. ROSflight; cloud vs. edge compute). Emerging techniques — particularly learned sensor fusion and sampling-based local planning — are promising but, in several cases, still at the preprint or early-research stage rather than established engineering practice. For students and researchers entering this field, the central lesson from this synthesis is that autonomous UAV design is an exercise in navigating explicit, context-dependent trade-offs rather than selecting universally "best" components.

Notes on Sourcing

This draft is built exclusively from the research notebook supplied for this task. Author/year attributions are reproduced as given in the notebook (Seisa et al., 2021; Shah, 2025; Kannan et al.; Carminati; Moore et al., 2025). **No DOI information was provided in the source notebook for any of these works**, and none has been invented for this draft; DOI/full bibliographic detail should be added at the revision stage once verified against the original sources. Preprint-stage sources (Shah, 2025; Seisa et al., 2021 as arXiv; Moore et al., 2025 as arXiv) are distinguished from institutional technical work (Kannan et al., NASA ARC/AFRC) and from Carminati's source, whose peer-review status is not specified in the notebook and should be verified before final submission. Claims not directly supported by the notebook (e.g., a specific EKF-vs-UKF performance ordering, quantitative cloud-vs-edge latency figures, and classical-vs-deep-learning perception benchmarks) have been explicitly flagged as gaps rather

- *Critique (Claude):*
    - The prompt is: Critically review the UAV research paper produced in the previous stage. Do not rewrite it yet. Evaluate:

        - Research quality: source credibility, citations, evidence, unsupported claims and citation mismatches.

- Technical accuracy: UAV architecture, flight control/PID, sensors, sensor fusion, KF/EKF/UKF, SLAM, path planning, computer vision, Edge AI, ROS 2, communication and safety.
- Technical depth: whether the paper explains how these technologies work rather than only defining them.
- Integration: whether it clearly connects Sensors → State Estimation → Perception/Mapping → Localisation → Path Planning → Control → Actuation.
- Critical analysis: quality of comparisons and trade-offs involving accuracy, latency, computation, energy, robustness and autonomy.
- Research gaps and FYP relevance: whether these are supported by the literature rather than speculative.
- Structure and writing: repetition, missing content, weak transitions, excessive jargon and imbalanced sections.

Do not invent sources or information.

Finish with:

1. Overall score /10
2. Top weaknesses ranked by importance
3. Strongest and weakest sections
4. Missing technical concepts
5. Claims requiring verification
6. Specific revision priorities

- The output is:

Critical Review: "How Modern Autonomous UAVs Work as Integrated Systems"

Research Quality

Source base is thin and entirely secondary. The paper draws on exactly five named sources, and all of them are known only through the summaries embedded in the research notebook — not through direct access to the original papers. This means every author/year citation in the draft is a *citation of a citation*: I attributed claims to Seisa et al. (2021), Shah (2025), Kannan et al., Carminati, and Moore et al. (2025) based on the notebook's one-line description of what each source covers, not on verified reading of the primary text. The topic-to-source pairings are internally consistent with the notebook (e.g., Carminati → PID/SMC, Seisa → MPC/Edge), so there are no outright **mismatches**, but the underlying evidentiary chain is weak — this is a genuine research-quality limitation that should be disclosed more prominently than the single footnote it currently gets.

The notebook's own "consensus" and "agreement" framing is treated as more authoritative than it should be. Statements like "broad consensus that ROS 2 is..." or "clearest points of consensus" originate from the notebook's own synthesized "Areas of Agreement" section — which is itself a secondary synthesis, not a systematic multi-paper review. The paper is careful with hedging language in most

places, but it still implicitly borrows the notebook's confidence level rather than independently interrogating it.

Unsupported or drifted claims that need flagging:

- Section 5 introduces the claim that "first-order linearisation is a known source of estimation error under strong nonlinearity" — this is general control-theory knowledge added by the writer, not something the notebook states. It's reasonable and standard, but it isn't sourced, and the paper should either cite a control-theory reference or mark it explicitly as background knowledge rather than blending it into the source-driven narrative.
- Section 12 invents specific numeric loop-rate language ("sub-millisecond attitude loop," "second-scale replanning loop") and Section 9 does the same ("millisecond-scale control loops"). None of these figures come from the notebook, which only gives IMU (100–1000 Hz) and GPS (1–10 Hz) rates. This is the most concrete instance of unsupported specificity in the draft and should be softened or removed.
- The Section 2 RTOS-kernel analogy is a pedagogical device, not a sourced claim — it's clearly framed as an analogy, which is acceptable, but it sits close enough to cited material that a careless reader could mistake it for part of the evidence base.

Technical Accuracy

Core control theory (PID/SMC/MPC), sensor fusion logic (IMU drift + GNSS correction), EKF's linearisation mechanism, SLAM family distinctions (ORB-SLAM/LOAM/LeGO-LOAM), and the global/local planning hierarchy are all described correctly and consistently with standard robotics/controls understanding — no factual errors detected in these areas.

Section 5 is the weakest technically. It is titled "Kalman Filtering: KF, EKF, and UKF" but only substantively explains the EKF. The base KF is mentioned only as "standard linear Kalman Filter machinery" with no actual mechanism (predict/update equations, state/covariance propagation). The UKF is not explained at all — not even its core mechanism (sigma-point sampling, unscented transform) — because the notebook doesn't cover it. The section essentially says "we can't compare EKF and UKF" without ever describing what a UKF *is*. For a section whose heading promises three algorithms, this is a significant depth shortfall, not just a sourcing limitation — the paper could have explained the UKF's mechanism from general knowledge (clearly labelled as background, not notebook-derived) even while declining to make a comparative performance claim.

Section 11 (Communication, Safety, and Reliability) is technically underdeveloped. It never discusses MAVLink, command-and-control (C2) link architecture, telemetry redundancy, failsafe behaviors (RTL, geofencing, loss-of-link procedures), or certification/regulatory safety cases — all standard components of "UAS safety and reliability" in the field. The section reduces almost entirely to one gap statement about cybersecurity. This is bounded by what the notebook contains, but the paper doesn't flag clearly enough that this section is *disproportionately thin relative to its topic's real scope*.

Technical Depth

Depth is uneven. Sections 3 (control), 4 (fusion), 6 (SLAM), 7 (planning), 10 (ROS 2/autopilots), and 15 (comparative trade-offs) explain *mechanisms*, not just definitions — e.g., how EKF linearises, why chattering occurs, why MPPI suits non-convex local planning. By contrast, Sections 5 (KF/UKF), 8 (CV/perception), 9 (Edge AI), and 11 (comms/safety) are largely definitional or gap-acknowledging rather than explanatory. Section 8 in particular defines classical vs. deep-learning perception but never explains *how* YOLO-style detection or ORB feature extraction actually works mechanically — it states that they're paired without unpacking either technique.

Integration (Pipeline Coherence)

The explicit pipeline walkthrough in Section 12 is one of the paper's stronger integrative moves — it maps pipeline stages back onto the FCU/companion-computer split established in Section 2, which is a genuine synthesis move rather than restatement.

However, there is a structural mismatch against the brief's requested pipeline: **the brief specifies Sensors → State Estimation → Perception/Mapping → Localisation → Path Planning → Control → Actuation (a distinct "Localisation" stage), but the paper (following the notebook's own six-stage pipeline) never separates Localisation from State Estimation/SLAM.** Localisation is folded silently into "Perception and Mapping" and Section 4's state estimation discussion, with no explicit paragraph clarifying the distinction between *pose estimation* (EKF, IMU+GPS) and *localisation within a map* (SLAM-derived position). This conflation is a real integration weakness relative to what was asked for.

Critical Analysis / Trade-offs

Section 15's comparison table is the strongest piece of critical synthesis in the paper — it correctly reframes the whole document around cost/capability trade-offs rather than "best algorithm" claims, which was an explicit requirement. That said, the requested trade-off dimensions of **energy** and **latency** are addressed only qualitatively ("power-hungry," "latency-incompatible") with no quantified figures anywhere — a limitation inherited directly from the notebook, which contains no numbers on power draw, detection latency, or estimation error. The paper should state this limitation explicitly rather than leaving quantitative absence implicit.

Research Gaps and FYP Relevance

The five research gaps in Section 14 map directly and traceably onto the notebook's own "Research Gaps" list — this section is well-grounded, not speculative. Section 16 (FYP implications), by contrast, is inherently the most speculative section in the paper by construction, since the notebook contains no FYP-specific content at all. The implications drawn are reasonable extrapolations and are appropriately framed as "implications" rather than findings, but a reader could still mistake the confidence of phrasing (e.g., "a project built on PX4... benefits from...") for sourced fact rather than reasoned inference.

Structure and Writing

- **Repetition:** SWaP-C is invoked in Sections 2, 3, 9, and 15. This reinforces the trade-off theme deliberately, but by the fourth repetition it reads as padding rather than reinforcement.
- **Imbalance:** Sections 5 and 11 are markedly shorter and thinner than Sections 3, 4, 6, 7, and 12, creating an uneven reading experience relative to their headline importance.
- **Transitions:** Generally solid; Section 12 in particular does real synthesis work rather than just restating prior sections.
- **Jargon:** Appropriately pitched for a CS/AI audience; no unexplained acronyms.
- **Missing content relative to headings:** Section 5 doesn't deliver on its own title (KF and UKF barely appear); Section 11 doesn't deliver a "safety and reliability" discussion beyond one gap statement.

---

1. Overall Score: 6.5/10

Solid architectural synthesis and appropriately cautious epistemic hedging, undercut by an entirely secondary source base, one significantly underdeveloped section (KF/UKF), a thin safety/communications section, a few unsourced numeric specifics, and an unaddressed structural gap around "Localisation" as a distinct pipeline stage.

2. Top Weaknesses (Ranked)

1. **Entirely secondary sourcing** — all citations trace to a synthesized notebook, not primary literature; citation accuracy cannot be independently verified.
2. **Section 5 fails to deliver on its own title** — KF and UKF are not actually explained, only EKF is.
3. **Unsupported numeric specificity** in Sections 9 and 12 (loop rates, "millisecond-scale") not found anywhere in the source material.
4. **Localisation is not treated as a distinct pipeline stage**, despite the brief explicitly requesting Sensors → State Estimation → Perception/Mapping → **Localisation** → Path Planning → Control → Actuation.
5. **Section 11 (comms/safety) is disproportionately thin** — no MAVLink, no failsafe/redundancy discussion, no certification framing beyond one sentence.
6. **Quantitative trade-off data is absent** (latency, energy, error rates) across the whole paper, with the limitation not stated as clearly as it should be.

3. Strongest and Weakest Sections

- **Strongest:** Section 15 (Critical Comparison of Approaches) — genuinely synthesizes trade-offs rather than restating facts; Section 12 (Integrated Pipeline) — ties architecture back to the FCU/companion-computer split effectively.

- **Weakest:** Section 5 (Kalman Filtering: KF, EKF, UKF) — thinnest technical content relative to its title; Section 11 (Communication, Safety, and Reliability) — least developed topic overall.

4. Missing Technical Concepts

- UKF mechanism (sigma-point/unscented transform)
- Base KF predict/update equations
- MAVLink and C2 link architecture
- Failsafe behaviors: RTL, geofencing, loss-of-link handling
- DDS/QoS specifics within ROS 2 (beyond node/topic/service)
- uORB (PX4's internal messaging) or equivalent ArduPilot internals
- How YOLO-style detection or ORB feature matching actually works mechanically
- Battery/power-system-level SWaP-C trade-offs (only discussed abstractly)

5. Claims Requiring Verification

- All five author/year attributions — need checking against the actual primary papers, not just the notebook's one-line descriptions.
- Carminati's peer-review status (already flagged in the draft as unknown).
- Whether "Perceiver IO"-inspired framing accurately represents Shah (2025)'s actual architecture.
- The institutional/publication venue implied by "NASA ARC/AFRC" for Kannan et al. (technical report vs. peer-reviewed paper).
- The added general-knowledge claim about EKF linearisation error in Section 5.
- The invented loop-rate figures in Sections 9 and 12.

6. Specific Revision Priorities

1. Rewrite Section 5 to actually explain KF and UKF mechanisms (clearly labelled as general background where not notebook-sourced), while keeping the "no comparative data available" caveat intact.
2. Remove or explicitly re-label the unsourced numeric claims in Sections 9 and 12.
3. Add an explicit paragraph distinguishing "localisation" from "state estimation" and "mapping," even if the pipeline ultimately keeps them structurally merged — at minimum, acknowledge the brief's requested stage explicitly.
4. Expand Section 11 with whatever comms/safety concepts can be responsibly added as general background (clearly separated from notebook-sourced claims), or explicitly state that this is a scope limitation of the source material.
5. Add a methodological caveat early in the paper (Section 1 or a new "Sourcing Note") disclosing that all citations are secondary, derived from a synthesized notebook rather than primary-source verification.
6. Reduce repeated SWaP-C invocations to 2 occurrences, consolidating the trade-off framing.

- *Revise (Claude):*
    - The prompt is: Revise the UAV research paper using the previous draft and critique. Correct the identified technical inaccuracies, unsupported claims, citation issues, repetition and structural weaknesses. Strengthen technical explanations and make the paper more analytical by comparing approaches and clearly explaining trade-offs between accuracy, computational cost, latency, energy, robustness and autonomy. Ensure the paper clearly explains the integrated pipeline: Sensors → State Estimation → Perception/Mapping → Localisation → Path Planning → Control → Actuation. Strengthen the discussion of UAV architecture, PID/control, sensor fusion, KF/EKF/UKF, SLAM, navigation, computer vision, Edge AI, ROS 2, communication, safety and FYP relevance where supported by the sources. Use only sources available in the research notebook. Do not invent citations, authors, DOIs, results, specifications or technical claims. Clearly distinguish established findings from inference and remove unsupported claims. Maintain a formal academic tone and preserve the original research objective. Output:
        - The complete revised paper
        - A brief Revision Summary explaining the major changes made.
    - The output is:

How Modern Autonomous UAVs Work as Integrated Systems: A Technical Synthesis

Revised Draft

Sourcing Note

This paper is based on a single research notebook summarising five sources: Seisa et al. (2021, arXiv preprint), Shah (2025, preprint), Kannan et al. (NASA ARC/AFRC technical work), Carminati (Politecnico di Torino; peer-review status not specified in the notebook), and Moore et al. (2025, arXiv preprint, on ROSflight 2.0). All author/year citations in this paper are therefore **secondary** — derived from the notebook's descriptions of these works rather than from direct reading of the original papers. No DOI information was available in the source material and none has been invented. Where the notebook does not contain sufficient information to support a technical explanation, this paper supplements with **standard, textbook-level robotics/control-theory background**, explicitly labelled as such and kept separate from claims attributed to the five named sources. Claims that neither the notebook nor standard background can support are identified as open questions rather than asserted.

---

1. Introduction and Scope

Unmanned Aerial Vehicles (UAVs) are commonly discussed as though the aircraft itself were the whole story. From an engineering and computer science standpoint, however, the aircraft is one component of a larger **Unmanned Aircraft System (UAS)**, comprising the UAV airframe, a ground control station (GCS), a human operator, and the communication data links connecting them. This distinction shapes how responsibilities for safety, computation, and autonomy are distributed across the system.

This paper explains how a modern autonomous UAV functions as an *integrated* system, following the pipeline **Sensors → State Estimation → Perception/Mapping → Localisation → Path Planning → Control → Actuation**. Each stage is treated both individually and in terms of how it depends on and feeds the others. Where the underlying literature disagrees, is incomplete, or does not extend to a comparison the reader might expect (e.g., EKF vs. UKF), this is stated explicitly rather than resolved by assumption. This is the **revised draft**, incorporating corrections from a prior critique stage; changes are summarised at the end of the paper.

---

2. UAV/UAS Fundamentals and System Architecture

A UAV's physical design is governed by **SWaP-C** constraints — Size, Weight, Power, and Cost — which recur throughout this paper because they underlie most of the trade-offs discussed in later sections. Thrust is generated by **Brushless DC (BLDC) motors**, chosen for their favourable speed-to-weight ratio and efficiency, and regulated by **Electronic Speed Controllers (ESCs)** that adjust voltage and current to control motor speed and thrust.

The **Flight Controller (FC)** coordinates these actuators, accepting commands and interfacing with onboard sensors to maintain stability. As autonomy requirements have grown, a structural split has emerged between:

- A **Flight Control Unit (FCU)**: a dedicated, real-time-capable unit responsible for low-level, high-rate stability and safety-critical control.
- A **Companion Computer** (e.g. NVIDIA Jetson, Raspberry Pi): a general-purpose processor responsible for computationally heavier, less time-critical autonomy tasks such as AI-based perception and path planning.

This division of labour is a recurring architectural theme in the notebook's sources: safety-critical, deterministic control stays on a dedicated unit, while heavier reasoning is offloaded to general-purpose hardware. Readers with a systems background may find it useful to think of this in terms of separating a hard real-time control loop from a soft real-time or best-effort computation layer — a general systems-design principle, not a claim attributed to any specific source in the notebook.

---

3. Flight Dynamics and Control

Autonomous flight is a **closed-loop feedback problem**: the system estimates its own state, compares it to a desired reference, and adjusts motor outputs to minimise the resulting error.

PID control. The Proportional–Integral–Derivative (PID) controller is described in the notebook's sources as the most commonly deployed method for attitude and rate control. Its proportional term responds to the current error, the integral term accumulates past error to eliminate steady-state offset, and the derivative term responds to the rate of change of error to dampen oscillation. PID's appeal is simplicity of implementation and tuning; however, as a linear control method it can show degraded performance at nonlinear operating points — for example during aggressive manoeuvres or under significant external disturbance. This paper treats PID as *widely used in practice* according to the sources reviewed, not as a formally standardised industry requirement, since the notebook does not support the stronger claim.

Sliding Mode Control (SMC). SMC is reported to offer faster response and improved disturbance rejection relative to PID (Carminati). It achieves this by driving the system state onto a predefined "sliding surface" and maintaining it there despite disturbances, at the cost of **chattering** — high-frequency oscillation of the control signal caused by the discontinuous switching term at the core of the method. Chattering translates into physical actuator stress and potential premature wear.

Model Predictive Control (MPC). MPC iteratively predicts future system outputs over a finite horizon and selects control actions that minimise a defined cost function, subject to system constraints. This predictive, optimisation-based approach is computationally more demanding than PID, which matters directly under the SWaP-C constraints of Section 2: hardware sized for a PID control loop may not support MPC at the same rate without additional onboard compute (Seisa et al., 2021).

Comparative trade-offs. PID, SMC, and MPC do not form a strict quality ordering but trade off along several axes:

Controller	Responsiveness	Robustness to nonlinearity/disturbance	Actuator wear	Computational cost
PID	Moderate	Degrades in nonlinear regimes	Low	Low
SMC	High	High	Elevated (chattering)	Low–moderate
MPC	High (predictive)	High	Low	High

The notebook does not provide quantified figures (e.g. settling time, tracking error, or power draw) for any of these controllers, so this comparison remains qualitative. **Model Predictive Path Integral (MPPI)** control, a sampling-based variant of MPC capable of handling non-convex objective functions, is discussed separately in Section 7 as it belongs to the local path-planning layer rather than inner-loop attitude control.

---

4. Sensors, Sensor Fusion, and State Estimation

No single sensor provides both the accuracy and update rate required for stable autonomous flight, making sensor fusion a necessity rather than an optimisation.

- The **Inertial Measurement Unit (IMU)** provides high-rate data (100–1000 Hz) on linear acceleration and angular rate. Position and velocity are obtained by integrating this data over time, which causes **drift**: small sensor biases and noise accumulate into growing position error.
- **GNSS/GPS** provides absolute position at a lower rate (1–10 Hz), used to correct the IMU's accumulated drift periodically.

A clear point of consensus across the notebook's sources is that IMU-only dead reckoning degrades and eventually fails without an external correction signal — whether from GPS or from a SLAM-derived position estimate. This is why GNSS-denied environments (Section 6) require a substitute correction source.

Fusion algorithms. The Extended Kalman Filter (EKF) is described as the traditional approach to nonlinear state estimation in this domain (elaborated in Section 5). More recent research explores **Transformer-based fusion architectures**, inspired by the Perceiver IO architecture, which dynamically weight heterogeneous sensor inputs such as LiDAR and camera data and are reported to outperform EKF-based approaches in GPS-degraded "urban canyon" conditions (Shah, 2025). This is preprint-stage research, not yet peer-reviewed or independently reproduced according to the notebook, and should be read with corresponding caution about generalisability.

---

5. Kalman Filtering: KF, EKF, and UKF

Because the notebook discusses the EKF specifically but does not provide comparative treatment of the base KF or the UKF, this section separates **notebook-derived claims** from **standard estimation-theory background** needed to make the discussion technically complete; the latter is general robotics/control knowledge, not attributed to any of the five sources.

Kalman Filter (KF, background). The standard KF is an optimal estimator for *linear* systems with Gaussian noise. It operates in two steps at each timestep: a **prediction** step, which propagates the previous state estimate and its uncertainty (covariance) forward using a linear motion model, and an **update** step, which corrects that prediction using a new sensor measurement, weighted by the relative uncertainty of the prediction versus the measurement (the Kalman gain). UAV dynamics are nonlinear (e.g., rotational kinematics), which is why the plain KF is not directly applicable to full UAV state estimation.

Extended Kalman Filter (EKF, notebook-derived). The EKF adapts the KF to nonlinear systems by linearising the nonlinear motion and measurement models

around the current state estimate at each timestep (typically via a first-order Taylor expansion / Jacobian), then applying the standard linear KF prediction/update equations to this local linear approximation. This makes the EKF computationally efficient and straightforward to implement, which likely explains its status as the "traditional workhorse" described in the notebook. Its known limitation, consistent with general estimation theory, is that the linearisation introduces approximation error that grows with the degree of nonlinearity and with larger timesteps between updates — this is a standard property of first-order linearisation, not a notebook-sourced finding, but it is consistent with the notebook's framing of the EKF as the conventional rather than the most capable method.

Unscented Kalman Filter (UKF, background). The UKF avoids explicit linearisation. Instead, it propagates a small, deterministic set of sample points (the "sigma points") through the true nonlinear model directly, then reconstructs the estimated mean and covariance from the transformed points (the "unscented transform"). This can capture nonlinear effects more accurately than a first-order linearisation, at the cost of propagating multiple sigma points through the model each timestep rather than a single linearised update — a higher, though generally still modest, computational cost relative to the EKF.

What can and cannot be concluded. The notebook does not contain a direct, sourced comparison of EKF versus UKF performance, computational cost, or accuracy in a UAV-specific context, and none is invented here. What can be responsibly stated is that the EKF's linearisation-based approximation is the known trade-off the UKF's sigma-point approach is designed to address in general nonlinear estimation problems; whether this translates into a meaningful practical advantage for UAV state estimation specifically — and at what computational cost — is identified here as an open question requiring further, UAV-specific literature beyond this notebook (see Section 14).

---

6. Localisation, Mapping, and SLAM

This section addresses three related but distinct problems: **localisation** (determining the vehicle's position relative to a reference frame), **mapping** (building a representation of the surrounding environment), and **SLAM**, which performs both simultaneously when neither an external position reference (GPS) nor a prior map is available.

Localisation vs. state estimation. Section 4/5's EKF-based state estimation produces a *continuously updated pose estimate* by fusing IMU and GPS data — this is localisation with respect to a global (GNSS) reference frame. In GNSS-denied settings, localisation instead means determining the vehicle's position *relative to a map it is simultaneously building*, which is the specific problem SLAM solves. The two are connected — SLAM-derived position estimates can themselves be fed back into the fusion filter described in Section 4/5 as a substitute correction source when GPS is unavailable — but they are not the same computation, and this paper treats them as distinct stages of the pipeline accordingly.

Simultaneous Localization and Mapping (SLAM). In GNSS-denied environments — indoors, in dense urban canyons, or under jamming — SLAM builds a map of the environment while concurrently estimating the vehicle's position within it.

- **Visual SLAM (vSLAM)**, exemplified by **ORB-SLAM**, extracts and tracks distinctive visual features across camera frames to reconstruct 3D structure and estimate motion between frames. Its principal weaknesses are loss of tracking during aggressive manoeuvres (where features move too quickly between frames to match reliably) and failure in low-texture environments where distinctive features are sparse.
- **LiDAR SLAM**, exemplified by **LOAM** and **LeGO-LOAM**, uses laser range measurements to build high-density 3D point-cloud maps and is reported to be generally more robust to lighting changes than vSLAM, since it does not depend on ambient light or visual texture.
- **Active SLAM** extends both approaches by integrating path planning into the SLAM loop itself, allowing the vehicle to autonomously select trajectories that reduce map uncertainty, rather than passively mapping whatever it happens to fly through.

Comparative trade-off. The notebook frames this as a cost/robustness trade-off rather than a superiority claim: LiDAR sensors are accurate and largely light-independent, but more expensive and power-hungry, which matters directly under the SWaP-C constraints introduced in Section 2. Vision-based systems are cheaper and lighter but comparatively fragile in low light and texture-poor environments. Flight-tested comparisons between visual and LiDAR SLAM are attributed in the notebook to NASA ARC/AFRC research (Kannan et al.), which should be read as institutional/technical research rather than necessarily peer-reviewed journal publication, since the notebook does not specify a publication venue.

---

7. Path Planning and Autonomous Navigation

Autonomous navigation is organised hierarchically, and — building on the localisation distinction in Section 6 — planning operates on top of whichever localisation estimate is currently available (GNSS-anchored or SLAM-relative).

1. **Global planning** determines the overall route to a distant goal, typically assuming a known or previously mapped environment. It addresses a comparatively static problem: given a map, find a feasible or optimal route.
2. **Local planning** handles real-time obstacle avoidance and short-horizon trajectory adjustment as new information arrives. The **Model Predictive Path Integral (MPPI)** controller — a sampling-based extension of MPC — is suited to this layer because it can optimise over non-convex objective functions, which arise naturally when obstacles create disjoint feasible regions in trajectory space that gradient-based methods struggle with.
3. **Obstacle avoidance** increasingly combines computer-vision object detectors (e.g. YOLO) with depth sensing to identify and react to dynamic obstacles absent from any prior map.

Global and local planning are not competing alternatives but operate at different timescales and address different problems: global planning assumes the environment is largely static and known, while local planning must react to obstacles and disturbances discovered in real time that a global plan could not have anticipated. Robust autonomy, per the sources reviewed, depends on both layers operating together.

---

8. Computer Vision and Perception

UAV perception in the reviewed literature combines two complementary technique families rather than treating one as a replacement for the other.

Feature-based geometric vision (as used in vSLAM) identifies distinctive, repeatable image features — corners, edges, or texture patterns — and tracks their pixel-position changes across consecutive frames. Given known camera geometry, this pixel-motion information can be used to estimate both the camera's own motion (for localisation) and the 3D position of the tracked features (for mapping). This is a structural/geometric task: recovering *where things are*, including the vehicle itself.

Deep-learning-based object detection (e.g., YOLO-family models) instead performs semantic recognition: a trained neural network processes an image and outputs bounding boxes and class labels for recognised object categories (e.g., "person," "vehicle," "obstacle"). This is a recognition task: identifying *what things are*, generally without requiring the same frame-to-frame geometric tracking that vSLAM relies on.

These two families are typically paired rather than substituted for one another: geometric vSLAM handles structural self-localisation and mapping, while deep-learning detectors handle recognition of specific object classes for avoidance or mission-relevant identification, feeding into the local planning layer described in Section 7. The notebook does not provide a direct, sourced comparison of classical versus deep-learning perception in terms of accuracy, latency, or robustness for UAV applications, so no general superiority claim is made here; their roles are complementary rather than substitutable within the pipeline.

---

9. Edge AI and Onboard Computing

Latency and privacy considerations drive UAV systems toward **onboard ("edge") AI processing** rather than cloud-based computation. Sending sensor data to a remote server and back introduces round-trip communication latency, and this delay is directly at odds with the requirement that control loops respond promptly to changes in vehicle state — precisely how much delay is tolerable depends on the specific control loop's dynamics and is not quantified in the notebook, so no specific timing figure is claimed here. Transmitting raw sensor data off-platform also raises data-exposure and privacy concerns for sensitive missions.

This latency and privacy rationale is the underlying reason for the companion-computer architecture introduced in Section 2: AI-heavy perception and planning workloads run locally, near the sensors and control loop, at the cost of onboard SWaP-C budget (additional weight, power draw, and hardware cost) rather than dependence on network connectivity and bandwidth. The notebook does not provide quantitative comparisons of cloud versus edge latency or energy consumption specifically for UAV workloads; this trade-off is therefore presented here as a well-motivated architectural rationale rather than a benchmarked result, and further quantification is identified as a gap in Section 14.

---

10. UAV Software Architecture and ROS 2

The **Robot Operating System 2 (ROS 2)** provides modular middleware for integrating perception, planning, and control through a node/topic/service communication model, in which independent processes ("nodes") exchange data over named channels ("topics") or request/response interactions ("services"). The notebook reports broad agreement among its sources that ROS 2 is a widely used foundation for building modular, distributed robotics software; this is reported here as a documented point of agreement among the reviewed sources, not as a claim of formal industry standardisation.

Two autopilot ecosystem approaches are discussed:

- **PX4** and **ArduPilot** provide full-featured, integrated flight stacks with autonomy functionality built into the flight-controller firmware itself.
- **ROSflight**, in contrast, is a leaner, research-oriented alternative that deliberately moves most autonomy logic off the flight controller and onto a Linux-based companion computer (Moore et al., 2025, describing ROSflight 2.0).

Bridging tools such as **uXRCE-DDS** connect the high-level ROS 2 environment to low-level PX4 firmware, allowing companion-computer-hosted autonomy logic to exchange data with the safety-critical flight controller without requiring the FCU itself to run ROS 2 — a practical instance of the FCU/companion-computer separation from Section 2.

Comparative trade-off. PX4 and ArduPilot offer more complete, production-oriented feature sets suited to deployment-focused work. ROSflight's leaner design, which pushes autonomy logic to the companion computer, is positioned as better suited to research contexts prioritising flexibility and transparency of the autonomy stack over turnkey completeness. The notebook does not provide benchmarked performance data comparing these ecosystems, so this remains an architectural-philosophy distinction rather than a quantitative one.

---

11. Communication, Safety, and Reliability

The UAS as a whole depends on communication data links connecting the aircraft, ground control station, and operator, in addition to the onboard hardware and software discussed in prior sections. Two aspects are relevant here.

Reliability under degraded conditions (general background, not notebook-sourced). UAS in practice typically implement predefined failsafe behaviours for loss of a command link or GNSS signal — for example, returning to a launch point, holding position, or landing in place — and telemetry redundancy where mission requirements demand it. These are standard practices in the broader UAS field; the notebook does not describe specific failsafe implementations, so no particular mechanism is attributed to the reviewed sources, and this paragraph is included only as general context necessary for a complete discussion of the topic, not as a notebook-derived finding.

Cybersecurity of data links and sensors (notebook-derived gap). The notebook explicitly identifies protection of data links and sensor inputs against spoofing or jamming as an open research gap, not a solved problem. This is a materially important caveat for any UAV intended to operate autonomously with reduced human oversight, since a compromised link or a spoofed sensor feed (e.g., false GNSS signals) directly undermines the state-estimation and localisation stages discussed in Sections 4–6. No specific mitigation techniques are detailed in the notebook, so none are proposed here; this is carried forward as a research gap in Section 14.

---

12. Integrated Autonomous UAV Pipeline

Bringing the preceding sections together, a fully integrated autonomous UAV executes a continuous, closed-loop pipeline:

1. **Sensors** collect raw data from the IMU, GNSS, LiDAR, and vision systems (Section 4).
2. **State Estimation** fuses IMU and GNSS data — via EKF, or in emerging research, Transformer-based fusion — to produce a continuously updated pose and velocity estimate anchored to a global reference frame (Sections 4–5).
3. **Perception and Mapping** builds an environmental representation through SLAM and detects task-relevant or dynamic objects via deep-learning detectors such as YOLO (Sections 6, 8).
4. **Localisation** determines the vehicle's position relative to whichever reference is currently available — the GNSS-anchored estimate from stage 2, or the SLAM-relative position from stage 3 when GNSS is degraded or unavailable — and may feed a SLAM-derived correction back into the state-estimation filter (Section 6).
5. **Path Planning** computes a safe trajectory using global planning together with local, sampling-based replanning such as MPPI in response to newly perceived obstacles (Section 7).
6. **Control** converts the planned trajectory into torque/thrust commands via PID, SMC, or MPC (Section 3).

7. **Actuation** — the **Motor Mixer** — translates these torque commands into PWM signals sent to the ESCs, which drive the motors (Section 2), physically changing the vehicle's state.

This is a closed loop, not a one-directional pipeline: actuation changes the vehicle's physical state, which is re-measured by sensors and re-processed through every subsequent stage continuously. The notebook indicates that different stages operate at different rates — the IMU alone is sampled at 100–1000 Hz and GNSS at 1–10 Hz — but it does not specify the operating rate of the control, planning, or perception loops themselves, so this paper does not assign specific loop-rate figures beyond what is stated for the sensors. The FCU/companion-computer split from Section 2 maps onto this pipeline structurally: the FCU typically owns state estimation and control/actuation (stages 2, 6, 7), while the companion computer owns perception, mapping, and higher-level planning (stages 3–5), communicating via bridges such as uXRCE-DDS (Section 10).

---

13. Current Research Trends

Several trends recur across the notebook's synthesis of recent (2021–2025) sources:

- **Learning-based sensor fusion** — Transformer/Perceiver IO-inspired architectures as a candidate supplement to EKF-based fusion in GPS-degraded conditions (Shah, 2025).
- **Edge computing architectures explicitly designed around MPC-class controllers**, addressing the tension between control sophistication and onboard compute limits (Seisa et al., 2021).
- **Leaner, research-first autopilot stacks** (ROSflight 2.0) that push more autonomy logic to companion computers rather than embedding it in flight-controller firmware (Moore et al., 2025).
- **Empirical, flight-tested comparison of visual versus LiDAR SLAM** under institutional research programmes (Kannan et al., NASA ARC/AFRC).

A consistent thread across these trends is the movement of intelligence away from fixed, tightly coupled flight-controller firmware and toward more flexible, learning-capable, modular companion-computer software, while safety-critical, high-rate control remains comparatively conservative (PID/EKF-dominated) at the innermost loop. This observation is a synthesis across the four trends above rather than a claim made directly by any single source.

---

14. Limitations and Research Gaps

Drawn directly from the notebook's own assessment, the following are open problems rather than solved engineering questions:

- **Long-term SLAM stability.** Current open-source SLAM algorithms lack demonstrated reliability over extremely long-duration autonomous missions, particularly in military or highly dynamic environments.
- **AI certification.** No established standards currently exist for certifying deep-learning-based components (such as Transformer-based fusion) for use in safety-critical civilian airspace.
- **Swarm coordination.** Real-time multi-agent coordination via edge computing is described as a nascent field with limited established results, in contrast to comparatively mature single-UAV autonomy.
- **Cybersecurity of data links.** Spoofing and jamming resistance for data links and sensor inputs is unresolved (Section 11).
- **EKF vs. UKF for UAV state estimation.** As discussed in Section 5, the notebook does not provide a rigorous, UAV-specific comparative evaluation between these two filtering approaches; this is an explicit gap in the available evidence rather than an assumed conclusion.
- **Quantitative trade-off data.** Across control (Section 3), sensor fusion (Section 4), SLAM (Section 6), and edge/cloud computing (Section 9), the notebook provides qualitative trade-off descriptions but no quantified figures for accuracy, latency, or energy consumption. This limits how precisely any of these trade-offs can be engineered against in practice and is flagged as a gap applicable across multiple sections rather than repeated in each one.

---

15. Critical Comparison of Approaches

The recurring pattern across this paper is that **most design choices in autonomous UAV systems trade capability or accuracy against resource cost — compute, power, weight, or money — rather than one option being universally superior.**

Dimension	Lower-cost / lower-compute option	Higher-capability / higher-cost option	Nature of trade-off
Control	PID	SMC / MPC	Responsiveness and disturbance rejection vs. computational cost and (for SMC) actuator wear
State estimation	EKF	Transformer-based fusion	Established, lightweight, well-understood vs. emerging, unproven at scale (preprint-stage)
State estimation (filter design)	EKF	UKF	Lower per-step compute (single linearisation) vs. potentially better nonlinear accuracy (sigma-point sampling); UAV-specific evidence not available in the notebook
Localisation	Visual SLAM	LiDAR SLAM	Lower cost/weight/power vs. lighting robustness and denser mapping

Dimension	Lower-cost / lower-compute option	Higher-capability / higher-cost option	Nature of trade-off
Planning	Global-only	Global + local (MPPI)	Simplicity vs. real-time reactivity to dynamic obstacles
Compute	Cloud offload	Onboard edge AI	Lower onboard power/weight vs. lower latency and reduced network dependency
Autopilot stack	PX4 / ArduPilot	ROSflight	Turnkey completeness vs. research flexibility and transparency

Each row reflects a trade-off along accuracy, computational cost, latency, energy, robustness, or autonomy — and in every case except the EKF/UKF row, the notebook supports at least a qualitative basis for the comparison; the EKF/UKF row is included for completeness but explicitly marked as unresolved by the available evidence. The practical implication for a reader is that selecting a component in any of these rows should be driven by the specific constraints of the application (available power and compute, required accuracy, environmental conditions, and tolerance for research-stage risk) rather than by a search for a single best-in-class option.

---

16. Final-Year-Project (FYP) Relevant Implications

The following implications follow from the sourced material above and are presented as reasoned extrapolations for project planning, not as established findings:

- A project built on **PX4 or ArduPilot** benefits from mature tooling and community support but offers less flexibility for modifying core autonomy logic; a project built on **ROSflight** offers more room to experiment with custom estimation or control logic on the companion computer, at the cost of a less turnkey starting point (Moore et al., 2025).
- An FYP attempting real-time **MPC or MPPI** on constrained hardware should budget explicitly for companion-computer compute rather than assuming a microcontroller-class flight controller can absorb the additional load (Seisa et al., 2021).
- An FYP proposing **learned sensor fusion** (Transformer-based or otherwise) should be scoped with the understanding that this is preprint-stage work, not an established or widely reproduced technique in the reviewed literature (Shah, 2025) — a reasonable research direction, but not one to present as settled practice.
- An FYP proposing a **UAV-specific EKF vs. UKF comparison** would address a genuine, explicitly identified gap in the available material (Section 14) rather than duplicating existing sourced comparisons.
- Comparative SLAM work (vSLAM vs. LiDAR SLAM) is a well-precedented direction, with NASA ARC/AFRC-attributed research (Kannan et al.) offering a relevant methodological reference point.

- Any project involving multi-UAV coordination should be scoped conservatively, since swarm coordination via edge computing is explicitly identified as a nascent research area rather than one with mature, off-the-shelf solutions.

---

17. Conclusion

A modern autonomous UAV is best understood not as a single intelligent aircraft but as a tightly coupled pipeline — sensors, state estimation, perception and mapping, localisation, path planning, control, and actuation — executing in a continuous closed loop, distributed across a safety-critical flight controller and a more flexible companion computer. The literature synthesised in this paper shows agreement on certain foundations: the necessity of multi-sensor fusion, the inevitability of IMU drift without external correction, and the practical utility of ROS 2 as modular middleware. It shows genuine, unresolved trade-offs elsewhere: PID versus SMC versus MPC; visual versus LiDAR SLAM; PX4/ArduPilot versus ROSflight; cloud versus edge compute; and, notably, an evidentiary gap around EKF versus UKF performance specifically for UAV applications. Emerging techniques — particularly learned sensor fusion and sampling-based local planning — are promising but, in several cases, still at the preprint or early-research stage rather than established engineering practice. For students and researchers entering this field, the central lesson is that autonomous UAV design is an exercise in navigating explicit, context-dependent trade-offs — in accuracy, computation, latency, energy, robustness, and autonomy — rather than selecting universally "best" components.

Revision Summary

This revision addressed the critique of the previous draft as follows:

1. **Sourcing transparency.** Added an explicit Sourcing Note at the start disclosing that all citations are secondary (derived from the research notebook, not primary reading), that no DOIs were available, and that general background material added to strengthen technical explanations is clearly separated from notebook-derived claims throughout.
2. **Section 5 rewritten.** The Kalman Filtering section previously only explained the EKF despite its title promising KF, EKF, and UKF. It now explains the mechanism of the base KF (predict/update, Kalman gain), the EKF (linearisation via Jacobian/Taylor expansion), and the UKF (sigma-point sampling, unscented transform) — with background material explicitly labelled as such and clearly separated from the notebook's own (limited) claims. The "no comparative UAV-specific evidence" caveat is preserved rather than removed.
3. **Unsupported numeric claims removed.** Invented loop-rate language ("sub-millisecond attitude loop," "millisecond-scale control loops") from Sections 9 and 12 has been removed and replaced with explicit statements that the

notebook only specifies IMU/GNSS sampling rates, not control/planning/perception loop rates.

4. **Localisation reinstated as a distinct pipeline stage.** Section 6 now explicitly distinguishes localisation (position relative to a reference frame) from state estimation (Section 4/5) and mapping, and the pipeline in Section 12 now lists Localisation as its own stage, consistent with the requested Sensors → State Estimation → Perception/Mapping → Localisation → Path Planning → Control → Actuation structure.
5. **Section 11 expanded.** Added general-background material (failsafe behaviours, telemetry redundancy) clearly marked as not notebook-sourced, addressing the previous critique that this section was disproportionately thin relative to its title; the notebook-derived cybersecurity gap is retained and linked explicitly to its downstream effect on state estimation/localisation.
6. **Repetition reduced.** SWaP-C is now introduced once in Section 2 and referenced (not re-explained) in Sections 3, 6, and 9, rather than being re-justified at each occurrence.
7. **Comparative/analytical strengthening.** Section 15's trade-off table was expanded to explicitly include the EKF/UKF filter-design row (marked as evidence-gap) and to state directly which of the six requested trade-off dimensions (accuracy, computational cost, latency, energy, robustness, autonomy) each row addresses.
8. **FYP section tightened.** Added an explicit FYP suggestion targeting the EKF/UKF evidence gap identified in the critique, and preserved the "extrapolation, not finding" framing for all other implications.
9. **Tone and attribution discipline.** Throughout, claims are now consistently marked as (a) notebook-derived and attributed, (b) general background/textbook knowledge added for completeness, or (c) explicitly unresolved/requiring further research — no claim is presented ambiguously across these three categories.

- *Human Review:*
    - I need to verify the research gaps identified.
    - Review all facts and sources.
    - Verify if it is feasible for a Final Year Project.
    - Review and verify the vague claims.
- *Failure points:*
    - The sources are hard to verify as they are not explicitly mentioned where needed.
    - Some of the claims made are too vague.
    - Some of the points were oversimplified.
    - Proper explanations for advanced topics are not included where needed.
- Time Comparison
    - Manual: 2 hours
    - AI: 20 minutes
    - Time Improvement: 83.3% reduction in time taken